Aria Programming Manual

Version 1.1 — Based on Aria v0.16.11

Alternative Intelligence

April 2026

Contents

1 Part I: Quick Reference **1**
1.1 Language Specification Cheat Sheet . . . 1
1.2 Operators . . . 25
1.3 Functions . . . 33
1.4 Control Flow . . . 39
1.5 Memory Model . . . 45
1.6 Modules . . . 49
1.7 Collections . . . 53
1.8 Io System . . . 56
1.9 Standard Library . . . 59
1.10 Advanced Features . . . 61
1.11 Debugging . . . 82

2 Part III: Safety Walkthrough **83**

3 Part IV: Appendix — Reference **99**

1 Part I: Quick Reference

1.1 Language Specification Cheat Sheet

```
### Mandatory Functions ##########################################
func:main = int32(argc,argv)     ** program entry, must exit() **
func:failsafe = int32(tbb32:err) ** safety net, must exit(>0) **

### Keywords ####################################################
return Result<T>        ** exit function with Result<T> value **
ok maybeUnknownVal      ** allow passing unknown **
drop myFunc()           ** bypass Result system and drop Result **
raw myFunc()            ** bypass Result system and return Result.value **
defaults value          ** default if error in chain **
fall label              ** used in pick for fallthrough **
pass T:retVal           ** return Result<T> with value **
fail tbb8:errCode       ** return Result<T> with error **
exit int32:errCode      ** exit program (main/failsafe) **
wild                    ** unmanaged memory **
defer                   ** guarantees cleanup code runs when scope exits **
async                   ** declare function async **
```

```
const                   ** constant,  only in extern blocks **
fixed                   ** Aria version of const **
use                     ** import modules/files **
mod                     ** define module **
pub                     ** public visibility **
extern                  ** external C functions **
stack                   ** explicit stack allocation **
gc                      ** explicit GC allocation **
wildx                   ** executable memory allocation (for JIT) **
requires                ** Design by Contract: function preconditions **
ensures                 ** Design by Contract: function postconditions **
invariant               ** Design by Contract: loop invariants **
result                  ** return value in ensures clause **
limit<Rules>            ** value constraint checks **
break                   ** exit innermost loop (while, for, till, loop) **
continue                ** skip to next loop iteration **
macro                   ** compile-time macro **
comptime                ** compile-time evaluation **
await                   ** suspend execution until async operation completes **
catch                   ** error/exception handling keyword **
in                      ** membership test in for loops and collections **
as                      ** alias in imports: use "mod.aria".* as alias **
cfg                     ** conditional compilation attribute **
inline                  ** suggest function inlining to compiler **
noinline                ** prevent function inlining **
trait                   ** define a trait (interface) **
impl                    ** implement trait for type **
derive                  ** auto-derive trait impls **
opaque                  ** opaque type declaration (hides internal representation) **
Type                    ** generic/existential constraint **
Rules                   ** compile-time constraint ruleset used with limit<Rules> **
move                    ** explicit move semantics (transfer ownership) **
prove                   ** formal verification assertion (checked by Z3 backend) **
assert_static           ** compile-time assertion **
relaxed                 ** memory ordering: no synchronization guarantees **
acquire                 ** memory ordering: acquire **
release                 ** memory ordering: release **
acq_rel                 ** memory ordering: acq+rel **
seq_cst                 ** memory ordering: seq cst **
stream                  ** reserved: stream types **
pipe                    ** reserved keyword for pipe/IPC types **
process                 ** reserved keyword for process types **
debug                   ** reserved keyword for debug output mode **

### Builtin Helpers ##############################################
astack(capacity?)       ** init scope stack (default 256) **
apush(handle,value)     ** push value onto stack **
apop(handle)            ** pop top value from stack **
apeek(handle)           ** peek at top value **
acap(handle)            ** return size of stack in bytes of scope stack **
asize(handle)           ** return number of bytes used in scope stack **
afits(handle,val)       ** true if val fits on stack **
atype(handle)           ** type of item on top of stack **
ahash(cap)              ** create hash table, cap bytes **
```

```
ahsize(handle)          ** hash table size in bytes **
ahget(handle, key)      ** get value at key (or NIL) **
ahset(handle, key, val) ** set key to value **
ahcount(handle)         ** number of keys in table **
ahfits(handle, val)     ** 1 if val fits, 0 otherwise **
ahtype(handle, key)     ** type tag at key, -1 if none **
sys(CONST, args...)     ** safe syscall, Result<int64> **
sys!!(CONST, args...)   ** full syscall, Result<int64> **
sys!!!(expr, args...)   ** raw syscall, bare int64 **

### Special Values ##############################################
ERR                     ** Tbb type sticky error sentinel **
NIL                     ** No value / nothing, Aria equivalent of void **
NULL                    ** No reference / no address / 0x00 **
unknown                 ** special sentinel for undefined **

### Types ######################################################
Result<T>               ** Result type, all functions **
any                     ** Aria equivalent of C void* **
void                    ** in extern blocks only **
int[1..4096]            ** signed integers **
uint[1..4096]           ** unsigned integers **
tbb[8,16,32,64]                                     ** twisted balanced binary **
flt[32,64,128,256,512]                              ** floating point **
tfp[32,64]                                          ** twisted floating point **
fix256                                              ** fixed point 128.128 **
fix256<[Joules,Meters,...]>  ** dimensional types **
frac[8,16,32,64]        ** fraction type **
bool                                                ** boolean **
vec[2,3,9]              ** vector types **
dyn                     ** trait object dispatch **
obj                     ** dynamic dispatch values **
struct                                              ** standard struct **
enum                                                ** standard enum **
string                                              ** Aria string **
func                                                ** function defintion **
array                                               ** standard array **
trit                                                ** ternary equivalent to bit **
tryte                                               ** ternary equivalent to byte **
nit                                                 ** nonary equivalent to bit **
nyte                                                ** nonary equivalent to byte **
tensor                                              ** tensor type **
ttensor                                             ** ternary tensor type **
matrix                                              ** matrix type **
tmatrix                                             ** ternary matrix type **
binary                                              ** binary data type **
buffer                                              ** memory buffer type **
complex<T>              ** complex number type **
atomic<T>               ** lock-free atomic type **
simd<T,N>               ** SIMD vector type **
Handle<T>               ** safe arena pointer **

### Control Flow ################################################
if/else                 ** standard if/else construct **
```

```
while                   ** standard while loop for advanced uses **
for                     ** standard for for advanced uses **
till(limit,step)        ** loop with automatic iteration tracking **
loop(start,limit,step)  ** loop with automatic iteration tracking **
when/then/end           ** better while with state tracking **
pick                    ** Aria equivalent of switch/case constructs **

### Operators ##################################################
+                       ** add **
-                       ** subtract **
*                       ** multiply (also ptr in extern) **
/                       ** divide **
%                       ** modulo **
++                      ** increment **
--                      ** decrement **
=                       ** assignment **
==                      ** equality check **
<=>                     ** spaceship, comparison that returns -1/0/1 **
is                      ** ternary: is a > b : T : F **
+=                      ** add and assign **
-=                      ** subtract and assign **
*=                      ** multiply and assign **
/=                      ** divide and assign **
%=                      ** modulo and assign **
!                       ** logical not **
||                      ** logical or **
&&                      ** logical and **
&                       ** bitwise and **
|                       ** bitwise or **
^                       ** bitwise xor **
~                       ** bitwise not **
<<                      ** bitwise left shift **
>>                      ** bitwise right shift **
!!!                     ** failsafe(err) shorthand, example: '!!! err' **
->                      ** pointer-to / member access **
<-                      ** dereference (value FROM pointer) **
@                       ** address of **
=>                      ** cast **
$                       ** iteration var / Rules value **
<T>?                    ** Optional as in int64? **
?                       ** Safe unwrap **
??                      ** Null coalesce **
?!                      ** Emphatic unwrap, calls failsafe if error **
?.                      ** Safe navigation**
?|                      ** defaults shorthand **
_?                      ** drop shorthand **
_!                      ** raw shorthand **
|>                      ** Pipe forward **
<|                      ** Pipe backward **
..                      ** inclusive range **
...                     ** exclusive range **
:                       ** annotation in types, separator in ternary clause **
::<T>                   ** turbofish type annotation **
#                       ** pin **
```

```
$$i                     ** immutable borrow **
$$m                     ** mutable borrow **
""                      ** string literal **
''                      ** char literal **
``                      ** template literal **
&{ }                    ** template interpolation **
//                      ** line comment **
/*                      ** begin block comment **
*/                      ** end block comment **
\                       ** escape **

### Aria STREAMS ################################################
stdout                  ** text output stream **
stderr                  ** error output stream **
stddbg                  ** debug output stream **
stdin                   ** text input stream **
stddati                 ** data input stream **
stddato                 ** data output stream **

### Compiler Flags (ariac) ######################################

## Output
-o <file>               ** write output to <file> **
--emit-llvm             ** emit LLVM IR text (.ll) **
--emit-llvm-bc          ** emit LLVM bitcode (.bc) **
--emit-asm              ** emit assembly (.s) **
--emit-deps             ** emit JSON dependency manifest (for aria_make) **
--emit-ptx              ** emit PTX assembly for GPU execution **
--emit-wasm             ** compile to WebAssembly (.wasm) **

## Compilation Modes
-c                      ** compile library (no failsafe required) **
--shared                ** compile to shared library (.so) **
--static                ** link as fully static executable **

## Optimization & Debug
-O<0-3>                 ** optimization level (0=none, 3=max) **
-g                      ** emit DWARF debug info (for aria-dap) **
-v, --verbose           ** verbose compiler output **

## Verification (Z3 SMT Solver)
--verify                ** Z3 verification of Rules/limit **
--verify-report         ** emit detailed proof results (implies --verify) **
--verify-contracts      ** verify requires/ensures function contracts **
--verify-overflow       ** verify integer arithmetic cannot overflow **
--verify-concurrency    ** detect data races and deadlocks in concurrent code **
--verify-memory         ** detect use-after-free bugs in wild pointer code **
--smt-opt               ** SMT-guided optimizations **
--smt-timeout=N         ** Z3 timeout in ms (default 5000) **
--prove-report          ** Proven/Disproven per check **

## GPU Target (NVIDIA CUDA/PTX)
--target=<arch>         ** cpu, gpu, gpu+cpu, wasm32-* **
--gpu-arch=<sm>         ** CUDA CC (default sm_50) **
```

```
--gpu-opt=<0-3>         ** GPU optimization level (default: 3) **
--gpu-debug             ** embed debug info in PTX **

## Input & Linking
-I <dir>                ** add directory to module search path **
-l<library>             ** link against library (e.g., -lm, -lpthread) **
-L<path>                ** add library search path **
-Wl,<option>            ** pass option to linker **

## Warnings
-Wall                   ** enable all warnings **
-Werror                 ** treat warnings as errors **
-W<warning>             ** enable specific warning **
-Wno-<warning>          ** disable specific warning **

## Diagnostics
--ast-dump              ** dump AST and exit **
--tokens                ** dump tokens and exit **
-E                      ** preprocess only (output to stdout) **

## General
--version               ** show version **
--help, -h              ** show help text **

### MORE INFO ##################################################
aria-docs/guide```

\newpage

## Version History

| Version | Aria | Key Changes |
|---------|------|-------------|
| v1.1 | v0.16.11 | Layout fixes, content update |
| v1.0 | v0.13.7 | Initial release |

### Since v1.0

- **v0.14.x**: Expanded Z3 SMT solver coverage, bitvector-accurate
  proofs, Rules consistency checking, solver-guided optimizations
- **v0.15.x**: Self-hosting foundation - 5 compiler modules ported
  to Aria (3,070 lines), C-bridge FFI shim pattern
- **v0.16.x**: Code review series - dead code removal (~46K lines),
  TODO/FIXME audit, debug cleanup, guide and example fixes

\newpage

# Part II: Programming Guide


## Types


##### Dynamic Type - `any`
```

```
#### Overview

`any` is Aria's dynamically-typed value container. It can hold any type, with the actual
type tracked at runtime. Internally implemented as a fat pointer `{ptr, i64}` with a type tag.

> **Note:** `dyn` is NOT an alias for `any`. `dyn` is used exclusively for trait object
> dispatch (`dyn TraitName` in function parameters). See [traits](../advanced_features/traits.md).

#### Declaration

```aria
any:box = 42i64;
```

1.1.0.1 Access Methods

Use turbofish syntax to read, write, and resolve `any` values:

```
// Read with get::<T>()
int64:val = box.get::<int64>();

// Write with set::<T>(value)
box.set::<int64>(100i64);

// Resolve - consuming transform to concrete pointer
int64->:ptr = box.resolve::<int64>();
```

1.1.0.2 Casting

`any` does not use `as` for casting. Aria has three cast forms:

```
// Infix arrow cast
int32:num = some_value => int32;

// Checked builtin
int32:num = @cast<int32>(some_value);

// Unchecked builtin (wraps/truncates on overflow)
int8:truncated = @cast_unchecked<int8>(large_value);
```

For `any`, use `.get::<T>()` or `.resolve::<T>()` to extract the concrete value.

1.1.0.3 Heterogeneous Collections

```
any[5]:items;
items[0] = 42i64;
items[1] = "hello";
```

1.1.0.4 Performance

Dynamic type checking has runtime overhead. **Prefer static types** wherever possible. Use `any` only when truly needed (e.g., heterogeneous containers, plugin interfaces).

1.1.0.5 Related

- struct.md — static composite types
- result.md — type-safe error handling

1.1.0.5.1 Array — `array`

1.1.0.6 Overview

Standard contiguous arrays with compile-time sizing.
```

1.1.0.7 Declaration

```
int32[5]:numbers;
numbers[0] = 10i32;
numbers[1] = 20i32;
numbers[2] = 30i32;
numbers[3] = 40i32;
numbers[4] = 50i32;
```

1.1.0.8 Access

```
int32:first = numbers[0];     // zero-indexed
int32:last = numbers[4];
if (numbers[2] != 30i32) {
    // handle error
}
```

1.1.0.9 Slicing

```
int64[5]:nums;
int64[]:slice = nums[0..4];    // inclusive range
int64[]:slice2 = nums[0...2];  // exclusive range
```

1.1.0.10 Related

- types/vec.md — fixed-size vector types
- collections/astack.md — user stack (LIFO)
- collections/ahash.md — hash tables

1.1.0.10.1 Atomic Types — `atomic<T>`

1.1.0.11 Overview `atomic<T>` provides lock-free concurrent access to a wrapped value. Lock-free when T 64 bits; locked (mutex-backed) for larger types.

1.1.0.12 Declaration Two forms are supported:

```
stack atomic<int32>:counter;                        // uninitialized stack allocation
atomic<int32>:counter = atomic_new(0i32);            // initialized with value
stack atomic<bool>:flag;                             // boolean atomic
```

Type constraint: `atomic<T>` requires lock-free compatible types: `int8`–`int64`, `uint8`–`uint64`, `bool`, `tbb8`–`tbb64`. Floats, strings, and arrays are **rejected** at compile time.

1.1.0.13 Memory Ordering Default ordering is **Sequential Consistency** (SeqCst) — the safest option. Weaker orderings require `unsafe { }`:

Ordering	Safety	Use Case
SeqCst	Safe (default)	General purpose
Acquire/Release	Requires unsafe	Producer/consumer patterns
Relaxed	Requires unsafe	Counters where ordering doesn't matter

1.1.0.14 Operations

```
int32:val = counter.load();                          // read
counter.store(42i32);                                // write
int32:old = counter.swap(100i32);                    // exchange
bool:ok = counter.compare_exchange(99i32, 100i32);   // CAS
int32:prev = counter.fetch_add(1i32);                // atomic increment
int32:prev = counter.fetch_sub(1i32);                // atomic decrement
```

Weak CAS (`compare_exchange_weak`) may spuriously fail — use in a loop.

1.1.0.15 ERR Sentinel The TBB ERR sentinel value remains atomic — ERR propagation works through atomic operations.

1.1.0.16 Status Basic operations (load, store, swap, CAS, fetch_add, fetch_sub) are implemented. Advanced memory orderings are in progress.

1.1.0.17 Related

- advanced_features/concurrency.md — threads, channels
- tbb.md — TBB error propagation through atomics

1.1.0.17.1 Boolean — `bool`

1.1.0.18 Overview Standard boolean type storing `true` or `false`. 1 byte storage. Default value: `false`.

1.1.0.19 Declaration

```
bool:is_ready = true;
bool:done = false;
```

1.1.0.20 Logical Operations

```
bool:a = true;
bool:b = false;
bool:and_r = (a && b);   // false - short-circuit AND
bool:or_r  = (a || b);   // true  - short-circuit OR
bool:not_r = (!a);        // false - logical NOT
```

Both `&&` and `||` use **short-circuit evaluation** — the right operand is skipped if the left operand determines the result.

1.1.0.21 Important **No implicit int→bool conversion.** This is not C:

```
// WRONG: bool:flag = 1;     // compile error
// RIGHT: bool:flag = true;
```

1.1.0.22 Style

```
// DON'T: if (is_ready == true) { ... }
// DO:    if (is_ready) { ... }
```

1.1.0.22.1 Complex Numbers — `complex<T>`

1.1.0.23 Overview Generic complex number type: `complex<T>` where T is the component type.

```
complex<T> = { T:real, T:imag }
```

Common instantiations: `complex<flt64>`, `complex<fix256>`, `complex<tbb64>`.

1.1.0.24 Declaration

```
complex<flt64>:z = { real = 3.0, imag = 4.0 };  // 3 + 4i
```

1.1.0.25 Arithmetic Standard complex arithmetic: `+`, `-`, `*`, `/` with proper mathematical formulas.

```
complex<flt64>:a = { real = 1.0, imag = 2.0 };
complex<flt64>:b = { real = 3.0, imag = 4.0 };
complex<flt64>:sum = (a + b);  // {4.0, 6.0}
```

- `conjugate()` — implemented
- `magnitude()` — future
- `phase()` — future

1.1.0.26 ERR Propagation If either component is ERR (when using TBB component types), the entire complex number is tainted.

1.1.0.27 SIMD Interleaved layout for SIMD: `[real , imag , real , imag , ...]`

1.1.0.28 Status **[WARN] Not yet implemented in the compiler.** Consider the `aria-entangled` package.

1.1.0.29 Related

- flt.md — float component types
- fix256.md — deterministic component type
- tbb.md — error-propagating component type

1.1.0.29.1 Enum

1.1.0.30 Overview Enumerations define a set of named integer constants. Backed by `int64` at the ABI level. Since v0.2.39.

1.1.0.31 Declaration

```
enum:Color = { RED, GREEN, BLUE };                        // auto-numbered: 0, 1, 2
enum:HttpStatus = { OK = 200, NOT_FOUND = 404, ERROR = 500 };  // explicit values
enum:Mixed = { A, B = 10, C };                            // C = 11 (continues from last)
```

1.1.0.32 Usage

```
Color:my_color = Color.RED;
int64:val = Color.GREEN;     // enums convert to int64

pick (my_color) {
    (Color.RED)   { drop println("Red"); },
    (Color.GREEN) { drop println("Green"); },
    (Color.BLUE)  { drop println("Blue"); },
    (*) {}
}
```

Enum comparison:

```
Color:a = Color.RED;
Color:b = Color.BLUE;
if (a != b) {
    drop println("Different colors");
}
```

1.1.0.33 Rules

- Variant names must be unique within the enum
- Values must be valid `int64`
- Convention: variant names in UPPER_CASE
- Declared at top level (not inside functions)
- Supports `==` and `!=` comparison
- Enum-typed variables enforce same-enum assignment
- Integrates with `pick` statement exhaustiveness checking

1.1.0.34 Related

- struct.md — composite types
- control_flow/pick.md — switch/case with exhaustiveness

1.1.0.34.1 Exotic Number Types

1.1.0.35 Balanced Ternary

Type	Description	Notes
`trit`	Ternary digit: -1, 0, +1	~2 bits packed
`tryte`	9 trits	Ternary byte equivalent

Trit values: `T` (-1), `0` (0), `1` ($+1$).

1.1.0.36 Balanced Nonary

Type	Description	Notes
`nit`	Nonary digit: -4 to +4	Base-9 unit
`nyte`	Collection of nits	Nonary byte equivalent

1.1.0.37 Status

Basic initialization and comparison operations compile and pass tests:

```
trit:a = 1;
trit:b = 0;
tryte:ta = 1;
nit:n1 = 3;
nyte:ny1 = 7;
```

These types are functional for basic operations. Advanced arithmetic is limited.

1.1.0.38 Related

- tbb.md — twisted balanced binary (production error-propagating type)
- int.md — standard binary integers

1.1.0.38.1 Fixed Point — `fix256`

1.1.0.39 Overview `fix256` is a **Q128.128 deterministic fixed-point type**: 128-bit integer part + 128-bit fractional part = 256 bits total. It provides bit-exact identical results across all platforms (x86-64, ARM64, RISC-V, CUDA).

1.1.0.40 Specifications

Property	Value
Total size	256 bits (32 bytes)
Integer part	128 bits (signed)
Fractional part	128 bits
Precision	2^-128 2.9×10^-39
Range	±2^127 ±1.7×10^38
Storage	4 × 64-bit limbs, `limb[3]` is MSB with sign
Deterministic	Yes — zero drift, bit-exact across platforms

1.1.0.41 Declaration

```
fix256:a = fix256_from_int(10);        // from integer
fix256:b = fix256_from_float(3.0);     // from float
```

1.1.0.42 Arithmetic

```
fix256:sum  = a + b;
fix256:diff = a - b;
fix256:prod = a * b;
fix256:quot = a / b;
if (a != b) { /* comparison works */ }
```

1.1.0.43 Conversion Back

```
int64:val = fix256_to_int(sum);
```

1.1.0.44 Dimensional Analysis Variants `fix256` supports dimensional type annotations for physics/engineering:

```
fix256<Joules>:energy = fix256(100);
fix256<Meters>:distance = fix256(50);
fix256<Seconds>:time = fix256(10);
fix256<Newtons>:force = fix256(25);
fix256<Kelvin>:temp = fix256(300);
```

The compiler enforces dimensional consistency — you cannot add Joules to Meters.

1.1.0.45 Why Not Floats?

- **Float problem**: `0.1 + 0.2  0.3` (IEEE 754 representation error)
- **Float problem**: Results differ across platforms (x87 vs SSE, ARM vs x86)
- **Float problem**: Non-associative: `(a + b) + c  a + (b + c)`
- **fix256**: Same bits in → same bits out, every time, every platform

1.1.0.46 Use Cases

- Safety-critical systems (aerospace, medical, nuclear)
- Financial calculations requiring exact decimal representation
- Deterministic physics simulation
- Cross-platform reproducible scientific computation
- Post-quantum cryptographic operations

1.1.0.47 Related

- flt.md — IEEE 754 floating point (faster but non-deterministic)
- frac.md — exact rational arithmetic
- int.md — LBIM integers used internally

1.1.0.47.1 Floating Point — `flt` Widths: `flt32`, `flt64`, `flt128`, `flt256`, `flt512`

1.1.0.48 Overview IEEE 754 floating-point types for real number arithmetic. `flt64` is recommended for general use. For deterministic fixed-point arithmetic, see fix256.md.

1.1.0.49 Width Table

Type	Bytes	Precision	Range (approx)	Alias	Notes
flt32	4	~7 decimal digits	±3.4×10^38	f32	
flt64	8	~15 decimal digits	±1.8×10^308	f64	Recommended
flt128	16	~33 decimal digits	±1.2×10^4932	f128	LBIM
flt256	32	~70 decimal digits	Extended	f256	LBIM
flt512	64	~150 decimal digits	Extended	f512	LBIM

1.1.0.50 Declaration

```
flt64:pi = 3.14159265358979;
flt32:temp = 98.6f32;          // type suffix
flt64:big = 1.23e10;           // scientific notation
```

1.1.0.51 Special Values

- `NaN` — Not a Number (result of `0.0 / 0.0`)
- `+∞` / `-∞` — overflow results
- `+0` / `-0` — signed zero (IEEE 754)

`NaN != NaN` is **true** (standard IEEE behavior). Always compare with epsilon:

```
flt64:a = 0.1 + 0.2;
flt64:epsilon = 0.000001;
// DON'T: if (a == 0.3) { ... }
// DO:    if (abs(a - 0.3) < epsilon) { ... }
```

1.1.0.52 Conversions

```
flt64:from_int = x;            // int→float: safe widening
int32:from_flt = f => i32;     // float→int: truncates (no rounding)
flt64:from_f32 = small;        // flt32→flt64: safe widening
```

1.1.0.53 ABI Note `flt32` passes as `double` at the C ABI level. If writing C shims for extern functions, the C side must declare parameters as `double`, then cast internally. This is an Aria codegen convention, not a bug.

1.1.0.54 When NOT to Use Floats

- **Money**: Use cents as `int64`, or `fix256`
- **Deterministic simulation**: Use `fix256` (bit-exact across platforms)
- **Error-propagating math**: Use `tbb` types

1.1.0.55 Related

- fix256.md — deterministic fixed-point
- tfp.md — twisted floating point
- int.md — integer types

1.1.0.55.1 Fractions — `frac` Widths: `frac8`, `frac16`, `frac32`, `frac64`

1.1.0.56 Overview Fraction types store exact rational numbers as mixed fractions: whole part + numerator + denominator. Built on TBB internally, inheriting sticky ERR propagation.

1.1.0.57 Width Table

Type	Component Type	Storage	Notes
frac8	tbb8	3 bytes	`{tbb8:whole, tbb8:num, tbb8:denom}`
frac16	tbb16	6 bytes	
frac32	tbb32	12 bytes	
frac64	tbb64	24 bytes	

1.1.0.58 Representation A frac value `{whole, num, denom}` represents `whole + num/denom`:

- `{1, 1, 3}` = 1 (exactly)
- `{0, 1, 3}` = (exactly — not 0.333…)
- `{0, 0, 1}` = 0

1.1.0.59 Canonical Form Invariants After every operation, fractions auto-normalize to canonical form: - Denominator > 0 - Numerator 0 when whole 0 - Numerator < denominator - Reduced to lowest terms (GCD applied)

1.1.0.60 Why Fractions? Floating point cannot represent exactly. Fractions can:

```
flt64:  1/3 = 0.333333333333... (truncated, drift accumulates)
frac32: 1/3 = {0, 1, 3}         (exact, zero drift)
```

1.1.0.61 Status `frac8`, `frac16`, and `frac32` compile with `{whole, num, denom}` initializer syntax:

```
frac8:half = {0, 1, 2};       // 0 + 1/2
frac16:third = {0, 1, 3};     // 0 + 1/3
frac32:mixed = {1, 1, 3};     // 1 + 1/3
```

`frac64` can be declared but does not yet support initialization.

1.1.0.62 Related

- tbb.md — underlying component type
- fix256.md — deterministic fixed-point alternative
- flt.md — IEEE floating point

1.1.0.62.1 Signed Integers — `int` Widths: `int1`, `int2`, `int4`, `int8`, `int16`, `int32`, `int64`, `int128`, `int256`, `int512`, `int1024`, `int2048`, `int4096`

1.1.0.63 Overview Aria provides two's complement signed integers in powers of 2 from 1-bit to 4096-bit. `int32` (alias `i32`) is the **default integer type** — bare literals like `100` infer as `int32`.

1.1.0.64 Width Table

Type	Bytes	Range	Alias	Notes
int1	1	-1 to 0	i1	Sign bit only
int2	1	-2 to 1	i2	
int4	1	-8 to 7	i4	
int8	1	-128 to 127	i8	`abs(-128)` is undefined
int16	2	-32768 to 32767	i16	
int32	4	-2^31 to 2^31-1	i32	**Default**. Wraps on overflow
int64	8	-2^63 to 2^63-1	i64	
int128	16	-2^127 to 2^127-1	i128	LBIM — see below
int256	32	-2^255 to 2^255-1	i256	LBIM
int512	64	-2^511 to 2^511-1	i512	LBIM
int1024	128	-2^1023 to 2^1023-1	i1024	LBIM
int2048	256	-2^2047 to 2^2047-1	i2048	LBIM
int4096	512	-2^4095 to 2^4095-1	i4096	LBIM

1.1.0.65 Declaration

```
int32:x = 42;                  // explicit type
int64:big = 9999999i64;        // suffix selects width
int128:huge = 99999999999999999999i128;  // suffix required for large literals
```

1.1.0.66 Arithmetic

```
int32:a = 10;
int32:b = 3;
int32:sum  = (a + b);    // 13
int32:quot = (a / b);    // 3 (truncates toward zero)
int32:rem  = (a % b);    // 1
a++;                      // 11
b--;                      // 2
```

Integer division truncates: `1000000 / 2000000 = 0`. Overflow wraps (two's complement).

1.1.0.67 Conversions

```
int64:wide = a;            // widening: always safe
int32:narrow = big => i32; // narrowing: truncates high bits
int32:from_flt = f => i32; // float→int: truncates (no rounding)
```

Widening from any intN to intM (M > N) is always safe. Narrowing truncates.

1.1.0.68 LBIM Types (128 bit) Widths `int128` through `int4096` use the **Limb-Based Integral Model**: - Stored as arrays of 64-bit limbs - **Slower than int64** — use only when necessary - Passed via `sret/byval` at the ABI level (not in registers) - Type suffix required for literals: `42i128`, `99i1024` - Use cases: cryptography, post-quantum key material, UUID storage, high-precision financial

1.1.0.69 ABI Notes

Width	ABI Passing
int1–int64	Register (zero/sign-extended)
int128+	Pointer (sret/byval)

1.1.0.70 Related

- uint.md — unsigned integers
- tbb.md — twisted balanced binary (error-propagating integers)
- fix256.md — fixed-point using LBIM

1.1.0.70.1 Pointers — `wild`, `wildx`, `->`, `@`

1.1.0.71 Overview Aria has three memory modes. Pointers are only directly used in `wild` (unmanaged) mode and `extern` blocks.

1.1.0.72 Pointer Operators

Operator	Meaning
`@`	Address-of (get pointer to variable)
`->`	In type: pointer to T. In expression: member access through pointer
`<-`	Dereference (get value from pointer)
`?.`	Safe navigation (NIL-safe member access)

1.1.0.73 Wild Pointers

```
wild int32:x = 42;          // unmanaged allocation
int32->:ptr = @x;           // pointer to x
int32:val = <-ptr;           // dereference
```

`wild` memory is **not garbage collected**. You must manage it manually or use `defer` for cleanup.

1.1.0.74 Executable Memory — `wildx` `wildx` allocates executable memory for JIT compilation:

```
wildx uint8->:code = wildx_alloc(4096);  // 4KB executable page
```

1.1.0.75 Safe Alternative: Handle For arena-allocated data, prefer `Handle<T>` over raw pointers. See memory_model/handle.md.

1.1.0.76 Related

- memory_model/wild.md — wild allocation mode
- memory_model/handle.md — generational handles
- functions/extern.md — FFI pointer patterns

1.1.0.76.1 Result — Mandatory Error Handling

1.1.0.77 Overview **Every function in Aria returns `Result<T>`** (except extern functions). The compiler forces you to handle the result — unhandled Results are compile errors. This is the core of Aria's safety model at the function level.

1.1.0.78 Structure

```
Result<T> = { T|NULL:value, tbb8:error, bool:is_error }
```

- Success: `value` is set, `error` is NIL, `is_error` is false
- Error: `value` is NIL, `error` is set, `is_error` is true

1.1.0.79 Creating Results

Inside functions (not `main` or `failsafe`):

```
func:divide = flt64(int32:a, int32:b) {
    if (b == 0) {
        fail 1;              // return error Result
    }
    pass (a / b);            // return success Result
}
```

- `pass value` — return success, wraps value in Result
- `fail errcode` — return error, wraps tbb8 error code in Result
- Both support paren syntax: `pass(value)`, `fail(errcode)`

1.1.0.80 Handling Results

```
// Safe unwrap with fallback
flt64:val = divide(10, 3) ? 0.0;

// Null coalesce
flt64:val = divide(10, 0) ?? default_value;

// Emphatic unwrap - calls failsafe on error
flt64:val = divide(10, 3)?!;

// Defaults keyword - scoped fallback for chains
flt64:val = a() + b() + c() defaults 0.0;
// Shorthand: flt64:val = a() + b() + c() ?| 0.0;

// Drop (discard result entirely)
drop divide(10, 3);
// Shorthand: _? divide(10, 3);

// Raw (extract value, ignore error)
flt64:val = raw divide(10, 3);
// Shorthand: flt64:val = _! divide(10, 3);

// Discard assignment
_ = divide(10, 3);
```

1.1.0.81 Operator Summary

Operator	Keyword	Meaning
`?`	—	Safe unwrap with fallback value
`??`	—	Null coalesce
`?!`	—	Emphatic unwrap (failsafe on error)
`?|`	`defaults`	Scoped fallback for expression chains
`_?`	`drop`	Discard Result entirely
`_!`	`raw`	Extract value, bypass error checking

1.1.0.82 Auto-Unwrap in Argument Position When passing a Result as an argument to a function expecting T, it auto-unwraps:

```
func:add = int32(int32:a, int32:b) { pass (a + b); }
func:get_val = int32() { pass 42; }

// get_val() returns Result<int32>, but add() expects int32
// Result auto-unwraps in argument position
int32:sum = raw add(get_val(), get_val());
```

This does **not** apply to variable assignment — you must explicitly unwrap.

1.1.0.83 The `ok` Keyword `ok` allows passing a variable whose value may be `unknown`:

```
ok maybe_unknown_value;
```

1.1.0.84 main and failsafe `main` and `failsafe` are special — they use `exit` instead of `pass`/`fail`:

```
func:main = int32() {
    exit 0;   // exit program with code 0
}

func:failsafe = int32(tbb32:err) {
    exit 1;   // must exit with code > 0
}
```

1.1.0.85 Related

- tbb.md — TBB error propagation (value-level)
- functions/main_failsafe.md — mandatory functions
- functions/result_system.md — pass/fail patterns

1.1.0.85.1 SIMD Types — `simd<T,N>`

1.1.0.86 Overview Generic SIMD (Single Instruction, Multiple Data) vector type where T is the element type and N is the lane count (must be a power of 2).

```
simd<flt32, 4>:v = simd_new(1.0f32, 2.0f32, 3.0f32, 4.0f32);
simd<int32, 8>:vi = simd_splat(42);  // all lanes = 42
```

1.1.0.87 Lane Counts N must be a power of 2: 2, 4, 8, 16, 32, etc. Hardware support varies by platform: - x86-64 SSE: 4×f32, 2×f64 - x86-64 AVX2: 8×f32, 4×f64 - x86-64 AVX-512: 16×f32, 8×f64

1.1.0.88 Operations Component-wise arithmetic: `+`, `-`, `*`, `/` apply to all lanes simultaneously.

1.1.0.89 Related

- vec.md — fixed-size vector types
- tensor_matrix.md — tensor and matrix types

1.1.0.89.1 String — `string`

1.1.0.90 Overview Aria strings are **UTF-8 encoded, immutable, reference-counted, and length-tracked** (not null-terminated internally). String operations return new strings.

1.1.0.91 Declaration

```
string:name = "Alice";
string:greeting = "Hello, world!";
string:empty = "";
```

1.1.0.92 String Literals

Syntax	Type	Escape Processing
`"..."`	Regular string	Yes — `\n`, `\t`, `\\`, etc.
`` `...` ``	Raw string	No — backslashes are literal
`` `Hello, &{name}!` ``	Template literal	Interpolation with `&{expr}`

1.1.0.93 Interpolation

Use backtick strings with `&{expression}` for interpolation:

```
string:name = "Aria";
int32:version = 4;
println(`&{name} version &{version}`);  // "Aria version 4"
```

1.1.0.94 Operations

```
string:full = (first + " " + last);  // concatenation
int64:len = string_length(name);      // character count (UTF-8 aware)
string:sub = text[0..5];              // substring slice (inclusive range)
```

Built-in string functions: - `string_length(s)` — character count - `string_contains(s, substr)` — boolean search - `string_concat(a, b)` — concatenation (also `+` operator)

1.1.0.95 Performance

Don't concatenate in loops. String concatenation creates new strings each time. For repeated building, collect parts in an array and join.

1.1.0.96 ABI Notes

This is critical for FFI: - **Parameters**: `string` passes as `const char*` (single register, null-terminated) - **Returns**: `AriaString {char* data, int64_t length}` by value (%rax=data, %rdx=length) - C shims must use `const char*` for input params, `AriaString` struct for returns

1.1.0.97 Related

- operators/string_ops.md — string operators
- io_system/print.md — print/println

1.1.0.97.1 Struct

1.1.0.98 Overview

Structs are composite types that group named fields. Methods are defined via trait implementations.

1.1.0.99 Declaration

```
struct:Point = {
    int32:x;
    int32:y;
};

struct:Person = {
    string:name;
```

```
    int32:age;
};
```

1.1.0.100 Instantiation and Access

Struct literal syntax with named fields:

```
stack Point:p = Point{x: 10, y: 20};
println(p.x);    // 10
println(p.y);    // 20

stack Person:alice = Person{name: "Alice", age: 30};
```

Positional arguments (order must match declaration):

```
Point:p = Point{10i32, 20i32};
```

Constructor syntax (desugars to `Type_create(args)`):

```
Counter:c = raw instance<Counter>(10i32);
```

Fields can also be assigned after declaration:

```
Point:p;
p.x = 10i32;
p.y = 20i32;
```

1.1.0.101 Methods (via Trait Impls)

Methods are defined through trait implementations:

```
trait:HasArea = {
    func:area: int32(Rect2D:self);
};

struct:Rect2D = {
    int32:w;
    int32:h;
};

impl:HasArea:for:Rect2D = {
    func:area = int32(Rect2D:self) {
        pass self.w * self.h;
    };
};

// Call via UFCS (Uniform Function Call Syntax)
Rect2D:rect;
rect.w = 10i32;
rect.h = 5i32;
Result<int32>:a = rect.area();
```

1.1.0.102 Passing Structs

```
func:add_coords = int32(Point:p) {
    pass p.x + p.y;
};

Point:origin = Point{ x: 0, y: 0 };
int32:sum = raw add_coords(origin);
```

1.1.0.103 Related

- enum.md — enumeration types
- memory_model/handle.md — Handle for arena structs

1.1.0.103.1 Twisted Balanced Binary — `tbb` Widths: `tbb8`, `tbb16`, `tbb32`, `tbb64`

1.1.0.104 Overview TBB types are Aria's **error-propagating integers**. They use a symmetric range with a dedicated ERR sentinel value. Once a TBB value becomes ERR, it stays ERR through all subsequent operations — "sticky error propagation." This is the foundation of Aria's safety model.

Think of ERR as **NaN for integers**, but with a single well-defined value and deterministic comparison behavior.

1.1.0.105 Width Table

Type	Bytes	Valid Range	ERR Sentinel	Notes
tbb8	1	-127 to +127	-128 (0x80)	`abs(-127) = +127` always safe
tbb16	2	-32767 to +32767	-32768 (0x8000)	
tbb32	4	-2^31+1 to +2^31-1	-2^31 (0x80000000)	
tbb64	8	-2^63+1 to +2^63-1	-2^63 (0x8000000000000000)	

Note the **symmetric range** — unlike standard two's complement, the positive and negative maximums are equal in magnitude. The minimum value is reserved as the ERR sentinel.

1.1.0.106 Declaration

```
tbb32:value = 42;
tbb8:small = 100tbb8;     // optional type suffix
tbb32:error = ERR;         // ERR literal
```

Default initial value is `0`.

1.1.0.107 Sticky Error Propagation Any arithmetic involving ERR produces ERR:

```
tbb32:a = 10;
tbb32:b = ERR;
tbb32:c = (a + b);   // ERR - propagated from b
tbb32:d = (c * 100); // ERR - propagated from c
```

This lets you chain computations and check once at the end:

```
tbb32:result = step3(step2(step1(input)));
if (result == ERR) {
    // handle error - one check covers entire pipeline
}
```

1.1.0.108 What Triggers ERR

- Overflow (result exceeds valid range)
- Underflow (result below valid range)
- Division by zero
- Explicit assignment: `val = ERR;`
- Any operation with an ERR operand (sticky propagation)

1.1.0.109 ERR Comparison Unlike IEEE NaN, ERR has well-defined comparison behavior: - `ERR == ERR` is **true** - ERR compares as **less than all valid values**

1.1.0.110 TBB vs Standard Integers

Feature	int32	tbb32
Range	-2^31 to 2^31-1	-2^31+1 to 2^31-1
Overflow	Wraps	ERR
Div by zero	Undefined	ERR
Error propagation	None	Sticky
abs(min)	Undefined	Valid (+max)
Performance	Faster	Slight overhead

Use `tbb` when error propagation matters. Use `int` when you need maximum performance or the full range.

1.1.0.111 Related

- result.md — Result error handling (function-level)
- int.md — standard signed integers
- frac.md — fraction types built on TBB
- fix256.md — fixed-point built on limbs

1.1.0.111.1 Tensor & Matrix Types

1.1.0.112 tensor Multi-dimensional array type for numerical computing and AI workloads.

1.1.0.113 matrix 2D matrix type. Semantically a rank-2 tensor with matrix-specific operations.

1.1.0.114 Status **[WARN] Not yet implemented.** The types are registered in the parser and can be declared, but have no constructors or operations. Also registered as `tmatrix` and `ttensor`.

```
matrix:mat;   // compiles (declaration only)
tensor:tens;  // compiles (declaration only)
```

1.1.0.115 Related

- vec.md — fixed-size vector types
- simd.md — SIMD vector types

1.1.0.115.1 Twisted Floating Point — `tfp` Widths: `tfp32`, `tfp64`

1.1.0.116 Overview Twisted floating point types combine IEEE-like floating-point representation with TBB-style sticky error propagation. They are to floats what TBB is to integers.

Type	Bytes	Notes
tfp32	4	Single-precision with ERR sentinel
tfp64	8	Double-precision with ERR sentinel

1.1.0.117 Status `tfp32` and `tfp64` compile with `{exponent, mantissa}` initializer syntax:

```
tfp32:pi_approx = {1, 14159};
tfp64:e_approx = {2, 71828};
```

Basic initialization works. Full arithmetic operations are in progress.

1.1.0.118 Related

- flt.md — standard floating point
- tbb.md — twisted balanced binary (integer ERR propagation)

1.1.0.118.1 Unsigned Integers — `uint` Widths: `uint1`, `uint2`, `uint4`, `uint8`, `uint16`, `uint32`, `uint64`, `uint128`, `uint256`, `uint512`, `uint1024`, `uint2048`, `uint4096`

1.1.0.119 Overview

Unsigned integers store non-negative values only. Required for bitwise operations (`&`, `|`, `^`, `~`, `<<`, `>>`).

1.1.0.120 Width Table

Type	Bytes	Range	Alias	Notes
uint1	1	0 to 1	u1	Single bit
uint2	1	0 to 3	u2	
uint4	1	0 to 15	u4	
uint8	1	0 to 255	u8	Byte
uint16	2	0 to 65535	u16	
uint32	4	0 to 2^32-1	u32	
uint64	8	0 to 2^64-1	u64	
uint128	16	0 to 2^128-1	u128	LBIM
uint256	32	0 to 2^256-1	u256	LBIM
uint512	64	0 to 2^512-1	u512	LBIM
uint1024	128	0 to 2^1024-1	u1024	LBIM
uint2048	256	0 to 2^2048-1	u2048	LBIM
uint4096	512	0 to 2^4096-1	u4096	LBIM

1.1.0.121 Declaration

```
uint32:flags = 0xFF00u32;
uint64:addr = 0xDEADBEEFu64;
uint8:byte = 255u8;
```

1.1.0.122 Bitwise Operations

Bitwise ops require unsigned types:

```
uint32:a = 0xFF00u32;
uint32:b = 0x0FF0u32;
uint32:and_result = (a & b);   // 0x0F00
uint32:or_result  = (a | b);   // 0xFFF0
uint32:xor_result = (a ^ b);   // 0xF0F0
uint32:not_result = (~a);      // 0xFFFF00FF
uint32:shl = (a << 4u32);     // 0xF0000
uint32:shr = (a >> 4u32);     // 0x0FF0
```

1.1.0.123 LBIM Types (128 bit)

Same limb-based model as signed integers. See int.md for LBIM details. Passed via sret/byval at ABI level.

1.1.0.124 Related

- int.md — signed integers
- tbb.md — twisted balanced binary

1.1.0.124.1 Vector Types — `vec` Widths: `vec2`, `vec3`, `vec9`

1.1.0.125 Overview Fixed-size vector types for 2D, 3D, and 9-component math. SIMD-optimized where possible. Defaults to float components.

1.1.0.126 Width Table

Type	Components	Size (float)	Size (double)	Use Case
vec2	x, y	8 bytes	16 bytes	2D graphics, UV coords
vec3	x, y, z	12 bytes	24 bytes	3D graphics, physics
vec9	9 elements	36 bytes	72 bytes	3×3 matrices, tensors

1.1.0.127 Declaration

```
vec2:pos = vec2(1.0, 2.0);
vec3:color = vec3(0.5, 0.8, 1.0);
vec9:v = 0;                      // zero-init only (vec9 has no constructor yet)
```

Components are `flt64` by default. Access via `.x`, `.y`, `.z`:

```
flt64:px = pos.x;
flt64:py = pos.y;
```

1.1.0.128 Operations

```
vec2:a = vec2(1.0, 2.0);
vec2:b = vec2(3.0, 4.0);

vec2:sum  = (a + b);       // component-wise add: {4, 6}
vec2:diff = (a - b);       // component-wise sub: {-2, -2}
vec2:prod = (a * b);       // component-wise mul (Hadamard): {3, 8}
```

∗ between two vectors is component-wise (Hadamard product), NOT dot product.

1.1.0.129 Status **Partial compiler support.** `vec2` and `vec3` constructors and arithmetic work. `vec9` supports zero-init only — no constructor or indexing yet.

1.1.0.130 Related

- simd.md — generic SIMD types
- tensor_matrix.md — tensor and matrix types

1.2 Operators

1.2.0.0.1 Arithmetic Operators

1.2.0.1 Binary Operators

Operator	Operation	Example	Notes
`+`	Addition	`(a + b)`	Also string concatenation
`-`	Subtraction	`(a - b)`	
`*`	Multiplication	`(a * b)`	Also pointer/deref in extern
`/`	Division	`(a / b)`	Integer division truncates toward zero
`%`	Modulo	`(a % b)`	Remainder after division

1.2.0.2 Unary Operators

Operator	Operation	Example
`++`	Increment	`x++`
`--`	Decrement	`x--`

1.2.0.3 Compound Assignment

Operator	Equivalent	Example
`+=`	`x = (x + y)`	`x += 5;`
`-=`	`x = (x - y)`	`x -= 5;`
`*=`	`x = (x * y)`	`x *= 2;`
`/=`	`x = (x / y)`	`x /= 2;`
`%=`	`x = (x % y)`	`x %= 3;`

1.2.0.4 Example

```
int32:a = 10;
int32:b = 3;
int32:sum = (a + b);      // 13
int32:quot = (a / b);     // 3 (truncates)
int32:rem = (a % b);      // 1
a++;                       // 11
a += 5;                    // 16
```

1.2.0.5 Integer Division Integer division truncates toward zero: `1000000 / 2000000 = 0`. For float division, use `flt64` operands.

1.2.0.5.1 Assignment Operator

1.2.0.6 Basic Assignment

```
int32:x = 42;             // typed declaration with assignment
x = 100;                   // reassignment
```

1.2.0.7 Discard Assignment

```
_ = some_function();      // discard Result - acknowledges unused return
```

This is one way to handle nodiscard Results. See also `drop` and `raw`.

1.2.0.8 Compound Assignment

See arithmetic.md for `+=`, `-=`, `*=`, `/=`, `%=`. See bitwise.md for `<<=`, `>>=`.

1.2.0.9 Fixed Values

```
fixed int32:MAX = 100;    // Aria's const (use 'fixed', not 'const')
```

`const` is reserved for extern blocks. Use `fixed` for Aria constants.

Note: `fixed` is specified in the language but has limited compiler test coverage. Compute in a variable first, then assign to `fixed` if needed.

1.2.0.9.1 Bitwise Operators

Bitwise operations require unsigned types (`uint8`, `uint16`, `uint32`, `uint64`).

Operator	Operation	Example
`&`	Bitwise AND	`(a & b)`
`|`	Bitwise OR	`(a | b)`
`^`	Bitwise XOR	`(a ^ b)`
`~`	Bitwise NOT	`(~a)`
`<<`	Left shift	`(a << n)`
`>>`	Right shift	`(a >> n)`

1.2.0.10 Compound Assignment

Operator	Example
`<<=`	`flags <<= 1;`
`>>=`	`flags >>= 1;`

1.2.0.11 Example

```
uint32:flags = 0xFF00u32;
uint32:mask  = 0x0FF0u32;

uint32:result = (flags & mask);   // 0x0F00
uint32:all    = (flags | mask);   // 0xFFF0
uint32:diff   = (flags ^ mask);   // 0xF0F0
uint32:inv    = (~flags);         // 0xFFFF00FF
uint32:shl    = (flags << 4u32);  // 0xF0000
uint32:shr    = (flags >> 4u32);  // 0x0FF0
```

1.2.0.12 Common Patterns

```
// Set bit
flags = (flags | (1u32 << bit));

// Clear bit
flags = (flags & (~(1u32 << bit)));
```

```
// Toggle bit
flags = (flags ^ (1u32 << bit));

// Check bit
bool:is_set = ((flags & (1u32 << bit)) != 0u32);
```

1.2.0.12.1 Cast & Type Operators

Operator	Meaning	Example
`=>`	Cast / type conversion	`val => int32`
`@cast<T>`	Explicit checked cast	`@cast<int32>(val)`
`@cast_unchecked<T>`	Unchecked cast	`@cast_unchecked<int32>(val)`
`::<T>`	Turbofish type annotation	`func::<int32>()`

1.2.0.13 Cast Operator

```
flt64:f = 3.14;
int32:i = f => int32;          // truncates: 3
int64:wide = i => int64;       // widening: safe
```

1.2.0.14 Built-in Cast Functions

```
int32:val = @cast<int32>(f);              // checked
int32:val = @cast_unchecked<int32>(f);    // unchecked (no validation)
```

1.2.0.15 Optional Types — `<T>?`

```
int64?:maybe = get_value();    // may be NIL
```

1.2.0.16 Pin — `#`

```
#value;                  // pin value (prevent move)
```

1.2.0.17 Borrow Operators

Operator	Meaning
`$$i`	Immutable borrow
`$$m`	Mutable borrow

```
$$i int32:ref = value;         // immutable borrow
$$m int32:mut_ref = value;     // mutable borrow
```

1.2.0.17.1 Comparison Operators

Operator	Operation	Example	Result
`==`	Equal	`(a == b)`	`bool`
`!=`	Not equal	`(a != b)`	`bool`
`<`	Less than	`(a < b)`	`bool`
`>`	Greater than	`(a > b)`	`bool`
`<=`	Less or equal	`(a <= b)`	`bool`
`>=`	Greater or equal	`(a >= b)`	`bool`

Operator	Operation	Example	Result
`<=>`	Spaceship	`(a <=> b)`	`-1`, `0`, or `1`

1.2.0.18 Spaceship Operator Three-way comparison returning `-1` (less), `0` (equal), or `1` (greater). Result type is `int64`.

```
int64:cmp = (a <=> b);
pick (cmp) {
    (-1i64) { println("a < b"); },
    (0i64)  { println("a == b"); },
    (1i64)  { println("a > b"); },
    (*) {}
}
```

1.2.0.19 Ternary Conditional — `is` Aria uses `is` for ternary expressions:

```
int32:max = is (a > b) : a : b;
string:msg = is (count == 0) : "empty" : "has items";
```

Syntax: `is (condition) : true_value : false_value`

1.2.0.20 Float Comparison **Never use == with floats.** Use epsilon comparison:

```
flt64:epsilon = 0.000001;
bool:equal = (abs(a - b) < epsilon);
```

1.2.0.20.1 Logical Operators

Operator	Operation	Example
`&&`	Logical AND	`(a && b)`
`||`	Logical OR	`(a || b)`
`!`	Logical NOT	`(!a)`

All logical operators use **short-circuit evaluation**: - `&&` — skips right operand if left is false - `||` — skips right operand if left is true

1.2.0.21 Example

```
bool:valid = (age > 0 && age < 150);
bool:allowed = (is_admin || has_permission);
bool:denied = (!allowed);
```

1.2.0.22 In Control Flow

```
if (is_ready && has_data) {
    process();
}

when (!done) {
    work();
} then {
    // completed normally
} end {
```

```
    // condition was false initially
}
```

1.2.0.22.1 Member Access & Navigation

Operator	Meaning	Example
`.`	Field access / enum member	`person.name`, `Color.RED`
`->`	Pointer member access	`ptr->field`
`?.`	Safe navigation (NIL-safe)	`ptr?.field`

1.2.0.23 Field Access

```
stack Point:p = Point{x: 10, y: 20};
int32:px = p.x;
int32:py = p.y;
```

1.2.0.24 Pointer Access

```
stack Point->:ptr = @p;
int32:x_val = ptr->x;
ptr->x = 100;
```

1.2.0.25 Enum Member Access

```
Color:c = Color.RED;           // dot notation for enum members
int64:r = Color.RED;
```

1.2.0.26 Turbofish — `::<T>` Explicit type annotation for generic functions:

```
int32:val = parse::<int32>("42");
```

1.2.0.27 Safe Navigation

```
string:city = employee?.address?.city;  // NIL if any step is NIL
```

1.2.0.27.1 Pipe & Range Operators

1.2.0.28 Pipe Operators

Operator	Meaning	Example
`\\\|>`	Pipe forward	`data \\\|> transform \\\|> output`
`<\\\|`	Pipe backward	`output <\\\| transform <\\\| data`

Pipe forward passes the left-hand value as the first argument to the right-hand function:

```
result = data |> filter |> transform |> output;
// Equivalent to: output(transform(filter(data)))
```

1.2.0.29 Range Operators

Operator	Meaning	Example
`..`	Inclusive range	`1..10` (1 through 10)
`...`	Exclusive range	`1...10` (1 through 9)

Used in: - Array slicing: `arr[1..5]` - Loop bounds - Pattern matching

1.2.0.30 Failsafe Shorthand — `!!!`

```
!!! err;     // equivalent to calling failsafe(err)
```

1.2.0.31 Address-Of — `@`

```
int32->:ptr = @value;    // get address of value
```

1.2.0.32 Iterator Variable — `$`

The `$` symbol is the automatic iteration variable in `loop` and `till`:

```
loop(0, 10, 1) {
    println($);       // prints 0 through 9
}
```

1.2.0.32.1 Result Operators

These operators control how `Result<T>` values are handled. Every function call in Aria returns `Result<T>` — these operators let you unwrap, propagate, or discard results.

1.2.0.33 Operator Table

Operator	Keyword	Meaning	Example
`?`	—	Safe unwrap with fallback	`val = func() ? 0;`
`??`	—	Null coalesce	`val = func() ?? default;`
`?!`	—	Emphatic unwrap (failsafe on error)	`val = func()?!;`
`?|`	`defaults`	Scoped fallback for chains	`val = a() + b() ?| 0;`
`_?`	`drop`	Discard Result entirely	`_? func();`
`_!`	`raw`	Extract value, bypass error	`val = _! func();`

1.2.0.34 Safe Unwrap — `?`

Returns the value on success, or the fallback on error:

```
int32:val = get_number() ? 0;    // 0 if get_number fails
```

1.2.0.35 Null Coalesce — `??`

```
string:name = get_name() ?? "unknown";
```

1.2.0.36 Emphatic Unwrap — `?!`

Calls `failsafe` if the Result is an error:

```
int32:val = must_succeed()?!;    // aborts via failsafe on error
```

1.2.0.37 Defaults — `?|` / `defaults`

Provides a scoped fallback for an entire expression chain:

```
// Any error in a(), b(), or c() uses 0 as fallback
int32:val = a() + b() + c() defaults 0;

// Shorthand operator form
int32:val = a() + b() + c() ?| 0;

// Nested defaults
int32:val = func(a() + 2 + c(2 + d() defaults 3) defaults 2);
```

Defaults scopes to the nearest enclosing expression, not the entire statement.

1.2.0.38 Drop — `_?` / `drop`

Silently discards the Result (no error check, no value extraction):

```
drop setup();         // keyword form
_? setup();           // shorthand form
```

Only works on expressions, NOT on named variables.

1.2.0.39 Raw — `_!` / `raw`

Extracts the value field, ignoring any error:

```
int32:val = raw risky_func();   // keyword form
int32:val = _! risky_func();    // shorthand form
```

1.2.0.40 Paren Syntax

All keywords support optional parentheses (v0.4.6+):

```
drop(func());    // same as: drop func();
raw(func());     // same as: raw func();
pass(value);     // same as: pass value;
fail(code);      // same as: fail code;
exit(0);         // same as: exit 0;
```

1.2.0.41 Related

- types/result.md — Result type documentation
- functions/result_system.md — pass/fail patterns

1.2.0.41.1 String Operators

1.2.0.42 Concatenation

The `+` operator concatenates strings:

```
string:full = ("Hello" + ", " + "world!");
string:name = (first + " " + last);
```

Long chains (7+ operands) work correctly.

1.2.0.43 Interpolation — Template Literals

Use backtick strings with `&{expression}` for interpolation:

```
string:msg = `Hello, &{name}! You are &{age} years old.`;
string:calc = `Result: &{a + b}`;
int32:x = 42;
println(`x = &{x}`);
```

1.2.0.44 Comparison

Strings support `==` and `!=`:

```
if (name == "Alice") {
    println("Found Alice");
}
```

1.2.0.45 Comments (related syntax)

Syntax	Type
`//`	Line comment
`/* ... */`	Block comment

1.2.0.46 Escape Sequences (in `"..."` strings)

Escape	Character
`\n`	Newline
`\t`	Tab
`\\`	Backslash
`\"`	Double quote

Raw strings (`` `...` `` without `&{}`) do not process escapes.

1.3 Functions

1.3.0.0.1 Async Functions

1.3.0.1 Declaration

```
async func:fetch_data = string(string:url) {
    // ... async operations
    pass data;
}
```

The `async` keyword marks a function as asynchronous. Async functions return `Result<T>` like all other functions.

1.3.0.2 Await

```
string:data = raw fetch_data("https://example.com");
```

1.3.0.3 Related

- advanced_features/concurrency.md — threads, channels
- result_system.md — error handling in async context

1.3.0.3.1 Closures & Lambdas

1.3.0.4 Function Pointer Variables Lambda expressions are assigned to typed variables using function pointer syntax:

```
(int32)(int32):identity = int32(int32:x) { pass x; };

(int32)(int32, int32):add = int32(int32:a, int32:b) {
    int32:sum = a + b;
    pass sum;
};
```

The variable type is `(ReturnType)(ParamTypes)` — return type first, then parameter types.

1.3.0.5 Passing as Arguments Higher-order functions accept function pointers as parameters:

```
func:apply = int32((int32)(int32):f, int32:x) {
    int32:result = f(x) ? -1i32;
    pass result;
};

(int32)(int32):dbl = int32(int32:x) { int32:v = x + x; pass v; };
int32:r = apply(dbl, 21i32) ? -1i32;
```

Note: Lambdas must be assigned to a named variable first, then passed by name. There are no anonymous inline lambdas at call sites.

1.3.0.6 Related

- declaration.md — named function syntax
- generics.md — generic function syntax

1.3.0.6.1 Function Declaration

1.3.0.7 Syntax Aria functions use the `func:name = return_type(params)` syntax:

```
func:add = int32(int32:a, int32:b) {
    pass (a + b);
}

func:greet = NIL(string:name) {
    println(`Hello, &{name}!`);
    pass NIL;
}
```

1.3.0.8 Key Rules

- Function name follows `func:` — e.g., `func:calculate`
- Return type comes before the parameter list
- Parameters use `type:name` syntax
- `NIL` return type for functions that return nothing
- NIL-returning functions must end with `pass NIL;`
- ALL functions return `Result<T>` implicitly (except extern functions)

1.3.0.9 Calling Functions

```
int32:sum = raw add(10, 20);           // raw unwrap
int32:safe = add(10, 20) ? 0;          // safe unwrap with fallback
drop greet("Alice");                    // discard Result
```

1.3.0.10 Visibility

```
pub func:public_func = int32() {      // visible to other modules
    pass 42;
}

func:private_func = int32() {          // file-private (default)
    pass 99;
}
```

1.3.0.11 Type Inference for Literals Bare integer literals default to `int32`. Use suffixes for other widths:

```
func:process = NIL(int64:val) { pass(NIL); }
// WRONG: process(42);       // 42 is int32, function wants int64
// RIGHT: process(42i64);    // explicit int64 suffix
```

1.3.0.11.1 Design by Contract

1.3.0.12 Overview Aria supports formal contracts on functions: `requires` (preconditions), `ensures` (postconditions), and `invariant` (loop invariants). These are checked at compile time where possible and at runtime otherwise.

1.3.0.13 Requires — Preconditions

```
func:divide = flt64(int32:a, int32:b)
    requires b != 0
{
    pass (a / b);
}
```

Multiple conditions are comma-separated:

```
func:clamp_to_byte = int32(int32:val)
    requires val >= 0, val <= 255
{
    pass val;
}
```

1.3.0.14 Ensures — Postconditions

```
func:check_order = int32(int32:lo, int32:hi)
    requires lo >= 0, hi > lo
    ensures hi > 0
{
    pass hi - lo;
}
```

Contract clauses go between the closing `)` of the signature and the opening `{` of the body. Compiled with `--verify-contracts` flag (uses Z3 SMT solver).

1.3.0.15 Invariant Loop invariants are specified in the language design but have limited compiler test coverage.

1.3.0.16 Related

- advanced_features/rules.md — refinement types with `limit<>`
- functions/declaration.md — function syntax

1.3.0.16.1 Extern — FFI with C

1.3.0.17 Overview `extern` blocks declare C functions callable from Aria. Extern functions do **not** return `Result<T>` — they return raw values.

1.3.0.18 Syntax

```
extern "libm" {
    func:sin = double(double:x);
    func:cos = double(double:x);
    func:sqrt = double(double:x);
}
```

Or flat (no library block):

```
extern func:custom_func = int32(int32:a, int32:b);
```

1.3.0.19 Rules

1. **Extern blocks must be at file scope** — not inside functions
2. `extern "lib" { }` blocks have a limit of **7** declarations
3. Flat `extern func:` declarations have no per-file limit
4. Use `void` for no-return (not NIL): `func:exit = void(int32:code);`
5. Use `const` (not `fixed`) inside extern blocks

1.3.0.20 String ABI

Direction	Aria Type	C Type
Parameter	`string`	`const char*`
Return	`string`	`AriaString {char* data, int64_t length}`

C shims for string-returning functions must return the AriaString struct.

1.3.0.21 Float ABI

Aria's `flt32` passes as `double` at the C ABI level. C shims must use `double` params:

```
// C side
double my_func(double x) {    // NOT float!
    return (float)x * 2.0f;
}
```

1.3.0.22 Pointer Types in Extern

Pointer types in extern blocks may create `{i1, ptr}` optional wrappers that corrupt struct fields. Workaround: use `int64` for handle/pointer types (ABI-compatible on x86-64).

1.3.0.23 Example

```
extern "libc" {
    func:printf = int32(string:fmt);
    func:malloc = int64(int64:size);    // use int64 instead of ptr
}

func:main = int32() {
    drop printf("Hello from C!\n");
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.3.0.24 Related

- modules/use_import.md — importing Aria modules
- types/string.md — string ABI details

1.3.0.24.1 Generics

1.3.0.25 Generic Functions

```
func:identity = T(T:value) {
    pass value;
}

func:max = T(T:a, T:b) {
    if (a > b) {
        pass a;
    }
    pass b;
}
```

Generic type parameters are monomorphized at compile time — one specialized version per concrete type used.

1.3.0.26 Turbofish Syntax

Explicitly specify type arguments with `::<T>`:

```
int32:val = identity::<int32>(42);
flt64:pi = identity::<flt64>(3.14);
```

1.3.0.27 Generic Constraints

Constraints limit which types can be used via trait bounds:

```
func<T: Addable>:sum = int32(T:a, T:b) {
    pass (a + b);
};
```

See advanced_features/traits.md for trait definitions.

1.3.0.28 Related

- advanced_features/traits.md — trait constraints
- advanced_features/rules.md — refinement types

1.3.0.28.1 main and failsafe — Mandatory Functions

Every Aria program **must** define both `main` and `failsafe`.

1.3.0.29 main

Program entry point. Must call `exit()` (not `pass`/`fail`):

```
func:main = int32() {
    println("Hello, world!");
    exit 0;
}
```

- Signature: `func:main = int32()` — takes no arguments (argc/argv via sys)
- Must call `exit(code)` where code is an `int32`
- `exit 0` = success, nonzero = error
- `pass`/`fail` are **not valid** in main

1.3.0.30 failsafe

Mandatory unrecoverable error handler. Called when `?!` unwrap fails:

```
func:failsafe = int32(tbb32:err) {
    println("Fatal error occurred");
    exit 1;
}
```

- Signature: `func:failsafe = int32(tbb32:err)` — receives error code
- Must call `exit(code)` where code > 0
- `pass`/`fail` are **not valid** in failsafe
- Called automatically on emphatic unwrap (`?!`) failure
- Called by `!!! err;` shorthand
- This is Aria's last line of defense — it must always exit

1.3.0.31 Required in Every Program

Both functions must be present. A program without either will not compile.

1.3.0.32 Paren Syntax

`exit` supports both keyword and paren syntax:

```
exit 0;      // keyword form
exit(0);     // paren form
```

1.3.0.32.1 Result System — pass and fail

1.3.0.33 Overview

Inside regular functions (not `main` or `failsafe`), use `pass` and `fail` to return Results:

- `pass value` — return success Result containing value
- `fail errcode` — return error Result with tbb8 error code

1.3.0.34 Pass

```
func:square = int32(int32:x) {
    pass (x * x);
}

func:greet = NIL(string:name) {
    println(`Hello, &{name}!`);
    pass(NIL);           // NIL-returning functions must pass NIL
}
```

`pass` wraps the value in `Result<T>{value=val, error=NIL, is_error=false}`.

1.3.0.35 Fail

```
func:divide = flt64(int32:a, int32:b) {
    if (b == 0) {
        fail 1;          // error code 1 - division by zero
    }
    pass (a / b);
}
```

`fail` wraps the error code in `Result<T>{value=NIL, error=code, is_error=true}`.

1.3.0.36 Paren Syntax

Both support optional parentheses (v0.4.6+):

```
pass(42);        // same as: pass 42;
fail(1);         // same as: fail 1;
pass (x * x);   // parenthesized expression (always worked)
```

1.3.0.37 return Keyword

`return` is also available and expects a full `Result<T>` instance:

```
return Result{ error: err, value: NIL, is_error: true };
```

In practice, `pass`/`fail` are preferred — they're more concise.

1.3.0.38 Related

- types/result.md — Result type
- operators/result_operators.md — ?, ?!, ?|, drop, raw
- main_failsafe.md — exit instead of pass/fail

1.4 Control Flow

1.4.0.0.1 break and continue

1.4.0.1 break Exit the innermost loop immediately:

```
loop(0, 100, 1) {
    if ($ == 42) {
        break;
    }
}
```

1.4.0.2 continue Skip to the next iteration:

```
loop(0, 10, 1) {
    if ($ == 5) {
        continue;    // skip 5
    }
    println($);
}
```

1.4.0.3 Works With

- `for`
- `while`
- `loop` / `till`
- `when/then/end`

1.4.0.3.1 Error Flow

1.4.0.4 Layered Error Model Aria has a three-tier error system:

Tier	Mechanism	Scope	Recovery
Value-level	TBB ERR sentinel	Arithmetic	Check `== ERR`
Function-level	Result pass/fail	Function calls	`?`, `?!`, `?|`, `drop`, `raw`
Program-level	failsafe	Fatal errors	Must `exit(code > 0)`

1.4.0.5 Error Propagation Flow

`TBB ERR (value) → Result<T> fail (function) → failsafe (program)`

1. **TBB arithmetic** — overflow, div-by-zero → ERR sentinel, propagates through pipeline
2. **Function calls** — `fail errcode` → caller must handle Result
3. **Unrecoverable** — `?!` failure, `!!! err` → calls failsafe, program exits

1.4.0.6 The `ok` Keyword Allows passing a variable that may have value `unknown` down the call chain:

```
ok maybe_unknown_value;
```

1.4.0.7 The `!!!` Operator — Failsafe Shorthand The `!!!` operator immediately invokes `failsafe` with an error code. It is syntactic sugar:

```
!!! errcode;    →    failsafe(errcode);
```

1.4.0.7.1 Usage

```
func:validate = int32(int32:input) {
    if (input < 0) {
        !!! 42;     // calls failsafe(42) - program exits
    }
    pass input;
}

func:main = int32() {
    int32:val = raw validate(10);
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    // err == 42 if triggered by !!! above
    exit err => int32;
}
```

1.4.0.7.2 When To Use

Mechanism	When	Recovery
`fail errCode`	Recoverable error in a function	Caller handles Result
`?!` (emphatic unwrap)	Unwrap Result, failsafe if error	failsafe receives error
`!!! errCode`	Unrecoverable error, bail immediately	failsafe receives errCode

1.4.0.7.3 Interaction with `defer`

`defer` blocks still execute before `!!!` transfers control to failsafe:

```
func:cleanup_example = NIL() {
    defer {
        // This runs BEFORE failsafe is called
    };

    !!! 1;   // defer fires, then failsafe(1)
    pass NIL;
}
```

1.4.0.8 Related

- types/tbb.md — TBB sticky error propagation
- types/result.md — Result system
- functions/main_failsafe.md — failsafe function
- operators/result_operators.md — error handling operators

1.4.0.8.1 for Loop

1.4.0.9 C-Style For

```
for (int32:i = 0; i < 10; i++) {
    println(i);
}
```

Standard three-part syntax: init; condition; update.

1.4.0.10 Notes

- No trailing semicolon after closing brace
- For simple iteration with a counter, prefer `loop()` or `till()`
- `for` is best for complex iteration patterns

1.4.0.11 Related

- loop_till.md — loop/till with automatic `$` iterator
- while.md — condition-only loops

1.4.0.11.1 if / else

1.4.0.12 Syntax `if` requires parentheses around the condition:

```
if (x > 0) {
    println("positive");
} else if (x == 0) {
    println("zero");
} else {
    println("negative");
}
```

1.4.0.13 Rules

- Parentheses around condition are **mandatory**
- No trailing semicolon after the closing brace
- Braces are required (no single-statement if)

1.4.0.14 Common Patterns

```
// Guard clause
if (input == NIL) {
    fail 1;
}

// With logical operators
if (age >= 18 && has_id) {
    println("Allowed");
}
```

1.4.0.14.1 loop and till — Counted Iteration

1.4.0.15 loop(start, limit, step) Counted loop with automatic iteration variable `$`:

```
loop(0, 10, 1) {
    println($);       // prints 0, 1, 2, ..., 9
}

loop(0, 100, 2) {
    println($);       // prints 0, 2, 4, ..., 98
}
```

- `$` is the automatic iteration variable — no manual declaration needed

- `start` is the initial value
- `limit` is the exclusive upper bound
- `step` is the increment per iteration

1.4.0.16 till(limit, step)

Shorthand when starting from 0:

```
till(10, 1) {
    println($);       // prints 0, 1, 2, ..., 9
}
```

Equivalent to `loop(0, limit, step)`.

1.4.0.17 Using $ in Expressions

```
int32:sum = 0;
loop(1, 101, 1) {
    sum = (sum + $);   // sum of 1 to 100
}
println(sum);          // 5050
```

1.4.0.18 Notes

- No trailing semicolon after closing brace
- `$` is a **reserved symbol** — cannot use for other purposes inside loop body
- For complex iteration, use `for` instead

1.4.0.19 Related

- for.md — C-style for loops
- while.md — condition-only loops

1.4.0.19.1 pick — Switch/Case

1.4.0.20 Syntax

Aria's equivalent of switch/case:

```
pick (value) {
    (1i32) { println("one"); },
    (2i32) { println("two"); },
    (3i32) { println("three"); },
    (*) { println("other"); }
}
```

1.4.0.21 Rules

1. **`(*)` wildcard is required** for types with infinite domains (int32, string, etc.)
2. Arms use `(value) { body }` syntax — value in parentheses, then block
3. Arms are separated by commas
4. No implicit fallthrough (unlike C switch)

1.4.0.22 Fallthrough — `fall`

Explicit fallthrough to a labeled case:

```
pick (value) {
    (1i32) { println("one"); fall two; },
    two: (2i32) { println("two or fell from one"); },
    (*) { println("other"); }
}
```

Labels are placed before the match pattern with a colon: `label: (val) { }`. Use `fall label;` to jump to a named case.

1.4.0.23 With Enums Enums support exhaustiveness checking — if all variants are covered, `(*)` is optional:

```
enum:Color = { RED, GREEN, BLUE };
Color:c = Color.RED;

pick (c) {
    (Color.RED)   { println("Red"); },
    (Color.GREEN) { println("Green"); },
    (Color.BLUE)  { println("Blue"); }
}
```

1.4.0.24 Related

- if_else.md — simple branching
- types/enum.md — enums with exhaustiveness

1.4.0.24.1 when / then / end — State-Tracked Loop

1.4.0.25 Syntax

```
when (condition) {
    // body - executes while condition is true
} then {
    // runs when loop completed normally (condition became false)
} end {
    // runs when condition was initially false, or break was used
}
```

1.4.0.26 Example

```
int32:i = 0i32;
int32:result = 0i32;
when (i < 3i32) {
    i = i + 1i32;
} then {
    result = 1i32;     // loop completed normally
} end {
    result = 2i32;     // condition was initially false or break
}
```

1.4.0.27 vs while `when/then/end` is semantically similar to `while` but also tracks *how* the loop terminated. `then` runs on normal completion (condition became false). `end` runs if the condition was initially false or `break` was used. Use `while` for simple loops, `when` for state-driven loops where termination reason matters.

1.4.0.28 Notes

- The loop body goes in the `when (cond) { }` block
- `then { }` is the normal-completion handler
- `end { }` is the false-or-break handler

1.4.0.29 Related

- while.md — simple condition loops
- loop_till.md — counted iteration

1.4.0.29.1 while Loop

1.4.0.30 Syntax

```
while (condition) {
    // body
}
```

1.4.0.31 Example

```
int32:count = 0;
while (count < 10) {
    println(count);
    count++;
}
```

1.4.0.32 Notes

- No trailing semicolon after closing brace
- For state-tracked loops, prefer `when/then/end`
- For counted iteration, prefer `loop()` or `till()`

1.4.0.33 Related

- when.md — when/then/end (state-tracked while)
- loop_till.md — counted iteration

1.5 Memory Model

1.5.0.0.1 Borrow Semantics

1.5.0.1 Overview Aria has borrow operators for safe reference passing:

Operator	Meaning
`$$i`	Immutable borrow — read-only reference
`$$m`	Mutable borrow — read-write reference

1.5.0.2 Usage

```
$$i int32:ref = value;        // immutable borrow
$$m int32:mut_ref = value;    // mutable borrow
```

Multiple immutable borrows are allowed simultaneously. Only one mutable borrow at a time. The compiler enforces the 1-mut XOR N-immut rule.

1.5.0.3 Pin The `#` operator pins a value, preventing it from being moved:

```
#value;                 // pin - cannot be moved after this
```

1.5.0.4 Related

- overview.md — memory model overview
- types/pointer.md — pointer types

1.5.0.4.1 Garbage Collected Allocation

1.5.0.5 Declaration GC allocation is **implicit** — variables that are not marked `wild` or `stack` are garbage collected:

```
Point:p1;
p1.x = 10i32;
p1.y = 20i32;
```

The `gc` keyword exists in the lexer but is not used as a prefix in practice. Absence of `wild` = GC-managed.

1.5.0.6 When to Use

- Data that must outlive the creating function's scope
- Shared references across multiple parts of the program
- Complex object graphs with cycles

1.5.0.7 Notes `gc` is a contextual keyword — it can be used as a variable name outside allocation contexts.

1.5.0.8 Related

- overview.md — all allocation modes
- stack.md — stack allocation (prefer when possible)
- wild.md — unmanaged memory

1.5.0.8.1 Handle — Safe Arena Pointers

1.5.0.9 Overview `Handle<T>` is a generational index for arena-allocated data. It prevents use-after-free (CWE-416) by tracking generation counters.

1.5.0.10 Structure

```
Handle<T> = { uint64:index, uint32:generation }
// 12 bytes data + 4 padding = 16 bytes, 8-byte aligned
```

1.5.0.11 Usage

```
// Create arena and allocate
Handle<Point>:h = arena.alloc(Point{x: 1.0, y: 2.0});

// Access (checks generation - returns ERR if stale)
Point:p = arena.get(h) ? default_point;

// Update
arena.set(h, Point{x: 3.0, y: 4.0});

// Free (increments generation - future gets return ERR)
arena.free(h);

// Grow arena (increments ALL generation counters)
arena.grow();
```

1.5.0.12 Why Not Raw Pointers?

Problem	Raw Pointer	Handle
Use-after-free	Undefined behavior	ERR (generation mismatch)
Dangling reference	Silent corruption	Detected at access time
Double free	Undefined behavior	ERR on second free

1.5.0.13 Related

- wild.md — unmanaged memory (when Handles aren't appropriate)
- overview.md — memory model overview

1.5.0.13.1 Memory Model Overview

1.5.0.14 Three Allocation Modes

Aria provides explicit control over memory allocation:

Keyword	Mode	Management	Use Case
`stack`	Stack	Automatic (scope-based)	Default, fast, most variables
`gc`	Garbage Collected	Automatic (GC runtime)	Shared data, complex lifetimes
`wild`	Unmanaged	Manual	FFI, OS-level, performance-critical

1.5.0.15 Stack Allocation (Default)

```
int32:x = 42;           // stack-allocated (default)
stack int32:y = 99;      // explicit stack keyword
```

Stack variables are freed when their scope exits. This is the fastest allocation mode.

Note: `stack` and `gc` are **contextual keywords** — they can be used as variable names outside allocation contexts.

1.5.0.16 GC Allocation

```
string:data = "hello";     // no wild/stack prefix = GC-managed
```

GC-allocated values are tracked by the garbage collector and freed when no longer referenced. This is implicit — absence of `wild` or `stack` means GC mode.

1.5.0.17 Wild (Unmanaged)

```
wild int32:raw_val = 42;
wild int8->:buf = alloc(1024);
```

No automatic cleanup. Combine with `defer` for manual resource management.

1.5.0.18 Choosing a Mode

- **Default to stack** — fast, safe, no overhead
- **Use GC** when data must outlive its creating scope or be shared
- **Use wild** only for FFI, OS interaction, or performance-critical paths
- **Never use wild without defer** unless you have a specific reason

1.5.0.19 Related

- stack.md — stack allocation details
- gc.md — garbage collector
- wild.md — unmanaged memory
- handle.md — Handle safe arena pointers
- borrow.md — borrow semantics

1.5.0.19.1 Stack Allocation

1.5.0.20 Overview Stack allocation is the **default** in Aria. Variables are allocated on the function's stack frame and automatically freed when the scope exits.

1.5.0.21 Declaration

```
int32:x = 42;           // implicitly stack
stack int32:x = 42;      // explicitly stack (same effect)
```

1.5.0.22 Scope Rules Stack variables exist only within their declaring scope:

```
func:example = NIL() {
    int32:outer = 1;
    if (true) {
        int32:inner = 2;   // only exists in this if-block
    }
    // 'inner' is gone here
    pass NIL;
}
```

1.5.0.23 Performance Stack allocation is essentially free — just a pointer bump. No garbage collection overhead, no manual deallocation needed.

1.5.0.24 Related

- overview.md — all allocation modes
- gc.md — for data that must outlive scope

1.5.0.24.1 Wild — Unmanaged Memory

1.5.0.25 Declaration

```
wild int32:raw = 42;
wild int8->:buffer = alloc(1024);
wild int64->:ptr = alloc<int64>();
```

Wild memory is **not tracked** by stack cleanup or GC. You are responsible for cleanup.

1.5.0.26 Defer — Guaranteed Cleanup

Use `defer` to ensure cleanup runs when scope exits:

```
wild int8->:buffer = alloc(1024);
defer {
    free(buffer);
}

// ... use buffer ...
// free(buffer) runs automatically at scope exit
```

`defer` executes in LIFO (last-in, first-out) order for multiple defers.

1.5.0.27 Executable Memory — wildx

`wildx` allocates memory with execute permission for JIT compilation:

```
wildx uint8->:code = wildx_alloc(4096);   // 4KB executable page
defer {
    wildx_free(code);
}

// Write machine code into 'code' buffer
// Execute via function pointer cast
```

Note: `wildx` is specified but has no passing tests yet.

1.5.0.28 When to Use Wild

- FFI buffers passed to C functions
- OS-level operations (mmap, syscalls)
- JIT compilation (`wildx`)
- Performance-critical paths where GC pauses are unacceptable

1.5.0.29 Related

- overview.md — all allocation modes
- handle.md — safe alternative to raw pointers
- types/pointer.md — pointer syntax

1.6 Modules

1.6.0.0.1 mod — Defining Modules

1.6.0.1 Module Declaration

```
mod math {
    pub func:add = int32(int32:a, int32:b) {
        pass (a + b);
    };

    func:helper = int32(int32:x) {   // private to module
        pass (x * 2);
    };
}
```

External module declaration (separate file):

```
mod crypto;
```

1.6.0.2 Visibility

- `pub` — public, visible to importers
- Default — private, visible only within the module/file

1.6.0.3 Related

- use_import.md — importing modules

1.6.0.3.1 Packages

1.6.0.4 Package Structure

Aria packages follow a standard layout managed by `aria-make`:

```
my-package/
  src/
     main.aria
     lib.aria
  test/
     test_main.aria
  aria-make.toml
  README.md
```

1.6.0.5 aria-make.toml

```toml
[package]
name = "my-package"
version = "0.1.0"

[dependencies]
aria-string = "0.1.0"
```

1.6.0.6 Building

```
aria-make build        # compile
aria-make test         # run tests
aria-make run          # build and run
```

1.6.0.7 Package Registry Packages are hosted at `aria-packages` and `aria-packages-apt`: - Source packages: `REPOS/aria-packages/packages/` - APT packages: `REPOS/aria-packages-apt/`

1.6.0.8 Package Census (v0.11.3)

Category	Count	%
Pure Aria	17	24.6%
aria-libc backed	9	13.0%
Direct extern FFI	43	62.3%
Total (with TOML)	**69**	
Legacy/scaffold (no TOML)	27	
Grand total	**96**	

1.6.0.8.1 New in v0.11.3

Package	Category	Description
`aria-bitset`	Pure Aria (aria-libc mem)	Fixed-size bit sets with union, intersect, complement
`aria-result`	Pure Aria (aria-libc mem)	Extended Result combinators — unwrap, map_or, and/or
`aria-deque`	Pure Aria (aria-libc mem)	Double-ended queue with O(1) push/pop at both ends

1.6.0.9 Related

- use_import.md — importing from packages
- mod.md — module definitions

1.6.0.9.1 Standard Library Overview The standard library contains **59 modules** in `REPOS/aria/stdlib/`.

1.6.0.10 Import Pattern

```
use "stdlib_file.aria".*;    // bare filename - compiler searches stdlib/
```

1.6.0.11 Available Modules The standard library lives in `REPOS/aria/stdlib/` and provides:

1.6.0.11.1 String Utilities

- `string_length(s)` — character count (UTF-8 aware)
- `string_concat(a, b)` — concatenation
- `string_contains(s, substr)` — substring search

1.6.0.11.2 I/O

- `print(s)` — output without newline
- `println(s)` — output with newline
- `readFile(path)` — read file contents
- `writeFile(path, data)` — write file contents

1.6.0.11.3 Math

- Standard math functions via extern libm

1.6.0.11.4 Threading

- Thread pool, channels, atomics, mutexes, condvars, rwlocks, barriers, actors (via stdlib wrappers on aria-libc)

1.6.0.11.5 Concurrency Modules (v0.11.0)

- `thread.aria`, `mutex.aria`, `condvar.aria`, `rwlock.aria` — via aria_libc_process
- `channel.aria`, `actor.aria`, `thread_pool.aria` — built on above
- `shm.aria` — POSIX shared memory via aria_libc_posix

1.6.0.12 Related

- modules/use_import.md — import syntax
- io_system/print.md — print/println details

1.6.0.12.1 use — Importing Modules

1.6.0.13 Syntax

```
use "path/to/module.aria".*;           // wildcard import (all pub symbols)
use "stdlib_file.aria".*;              // stdlib import (bare filename)
```

1.6.0.14 Path Resolution

- **Relative paths**: resolved relative to the importing source file
- **Bare filenames**: compiler searches the stdlib directory
- Wildcard `.*` imports all `pub` declarations

1.6.0.15 Example

```
// file: math_utils.aria
pub func:square = int32(int32:x) {
    pass (x * x);
}

// file: main.aria
use "math_utils.aria".*;

func:main = int32() {
    int32:val = raw square(5);
    println(val);    // 25
    exit 0;
}

func:failsafe = int32(tbb32:err) { exit 1; }
```

1.6.0.16 Notes

- Only `pub func:` declarations are importable
- `Rules<T>` are file-scoped — not importable
- Negative constants imported via `use` may be zeroed at runtime (known codegen issue) — compute inline as `0i64 - value` instead

1.6.0.17 Related

- mod.md — defining modules
- packages.md — package structure

1.7 Collections

1.7.0.0.1 Hash Tables — ahash

1.7.0.1 Overview Per-scope hash tables mapping string keys to typed values. Heterogeneous (different value types per key). Auto-cleanup on scope exit. Optional byte-budget capacity.

1.7.0.2 Builtins

Builtin	Signature	Description
`ahash(cap)`	`(int64) → int64`	Create hash table with byte budget. 0 = unbounded. Returns handle.
`ahset(h, key, val)`	`(int64, str, T) → int32`	Set key to value. Returns 0 on success, -1 on overflow.
`ahget(h, key)`	`(int64, str) → T`	Get value at key. NIL if not found. Type mismatch → failsafe.
`ahcount(h)`	`(int64) → int64`	Number of keys in table.
`ahsize(h)`	`(int64) → int64`	Current size in bytes of all stored entries.
`ahfits(h, val)`	`(int64, T) → int64`	1 if val fits in remaining capacity, 0 otherwise. Always 1 if unbounded.
`ahtype(h, key)`	`(int64, str) → int32`	Type tag of value at key, -1 if key not found.

1.7.0.3 Example

```
func:main = int32() {
    int64:ht = ahash(0);                // unbounded hash table

    int32:rc1 = ahset(ht, "name", "Alice");
    int32:rc2 = ahset(ht, "age", 30);
    int32:rc3 = ahset(ht, "score", 98.6);

    string:name = ahget(ht, "name");
    int32:age = ahget(ht, "age");
    flt64:score = ahget(ht, "score");

    int64:count = ahcount(ht);          // 3

    println(name);
    println(age);
    exit 0;
}

func:failsafe = int32(tbb32:err) { exit 1; }
```

1.7.0.4 Fast-Path Mode When the Z3 solver can prove all values in a hash table are the same type at compile time, the compiler emits optimized **fast-path** code that skips runtime type checks. This is automatic — no user action required.

1.7.0.5 Capacity

- `ahash(0)` — unbounded (grows as needed)
- `ahash(1024)` — 1KB byte budget; `ahset` returns -1 when full

- `ahfits(h, val)` — check before inserting into bounded tables

1.7.0.6 Type Safety The receiving variable's type determines the expected type on `ahget`:

```
int32:val = ahget(ht, "age");     // expects int32 at "age"
string:s = ahget(ht, "age");      // type mismatch → failsafe!
```

Check with `ahtype(h, key)` before getting if unsure.

1.7.0.7 Auto-Cleanup Hash tables are automatically freed when the scope exits.

1.7.0.8 Related

- astack.md — user stack (LIFO)

1.7.0.8.1 User Stack — astack

1.7.0.9 Overview Per-scope implicit LIFO (last-in, first-out) scratch pad. Stores heterogeneous typed values. Auto-cleanup on scope exit. Fatal error model (exit(1) on error, not Result).

1.7.0.10 Builtins

Builtin	Signature	Description
`astack(cap)`	`(int64?) → int64`	Initialize stack with byte budget (default 256). Returns handle.
`apush(h, val)`	`(int64, T) → void`	Push typed value. Fatal on overflow.
`apop(h)`	`(int64) → T`	Pop top value. Type inferred from assignment. Fatal on mismatch/underflow.
`apeek(h)`	`(int64) → T`	Non-destructive read of top value. Same typed semantics as apop.
`acap(h)`	`(int64) → int64`	Total capacity in bytes.
`asize(h)`	`(int64) → int64`	Bytes currently used.
`afits(h, val)`	`(int64, T) → bool`	True if val fits without overflow.
`atype(h)`	`(int64) → int32`	Type tag of top element. Useful to avoid mismatch.

1.7.0.11 Example

```
func:main = int32() {
    int64:stk = astack 512;            // 512-byte stack (keyword form)
    // int64:stk = astack(512);        // paren form also works

    apush stk, 42;                      // push int32
    apush stk, "hello";                 // push string
    apush(stk, 3.14);                   // paren form

    string:s = apop stk;                // pop string: "hello" (LIFO)
    // string:s = apop(stk);            // paren form

    int32:n = apop(stk);                // pop int32: 42
    println(n);

    exit 0;
```

```
}

func:failsafe = int32(tbb32:err) { exit 1; }
```

1.7.0.12 Syntax Forms (v0.4.6+)

All astack builtins support both keyword and paren syntax:

```
// Statement-level (apush, astack)
apush stk, value;       // keyword
apush(stk, value);      // paren
astack 256;             // keyword
astack(256);            // paren

// Expression-level (apop, apeek, acap, asize, atype)
int32:v = apop stk;     // keyword
int32:v = apop(stk);    // paren

// Expression-level with argument (afits)
bool:ok = afits stk, value;   // keyword
bool:ok = afits(stk, value);  // paren
```

1.7.0.13 Error Behavior

Errors are **fatal** (not Result): - Stack overflow → `exit(1)` - Stack underflow → `exit(1)` - Type mismatch on pop/peek → `exit(1)`

Check before popping:

```
if (asize(stk) > 0) {
    int32:val = apop(stk);
}
```

1.7.0.14 Auto-Cleanup

The stack is automatically cleaned up when the scope exits (function return). No manual deallocation needed.

1.7.0.15 Related

- ahash.md — hash tables

1.8 Io System

1.8.0.0.1 File I/O

1.8.0.1 Reading Files

```
string:content = raw readFile("data.txt");
```

1.8.0.2 Writing Files

```
drop writeFile("output.txt", "Hello, file!");
```

1.8.0.3 File Operations

Function	Description
`readFile(path)`	Read entire file as string
`writeFile(path, data)`	Write string to file
`fileExists(path)`	Check if file exists
`fileSize(path)`	Get file size in bytes
`deleteFile(path)`	Delete a file

All file operations return `Result<T>`.

1.8.0.4 Streams

Stream	FD	Purpose
`stdin`	0	Standard input
`stdout`	1	Standard output
`stderr`	2	Standard error
`stddbg`	3	Debug stream (Hexstream)
`stddati`	4	Data input stream (Hexstream)
`stddato`	5	Data output stream (Hexstream)

1.8.0.5 Related

- print.md — print/println
- stdin.md — reading from stdin
- sys.md — direct syscalls

1.8.0.5.1 Print & Output

1.8.0.6 print / println

```
print("no newline");
println("with newline");
println(42);                        // prints integer
println(`Value: &{expression}`);    // interpolation
```

- `print(s)` — output without newline
- `println(s)` — output with newline

1.8.0.7 Streams

Stream	Purpose
`stdout`	Standard text output
`stderr`	Error output
`stddbg`	Debug output

1.8.0.8 Related

- types/string.md — string types and interpolation
- file_io.md — file operations

1.8.0.8.1 Standard Input

1.8.0.9 Reading from stdin

```
string:line = raw stdin_read_line();    // read one line
string:all = raw stdin_read_all();      // read until EOF
```

Both return `Result<string>`.

1.8.0.10 Related

- print.md — output
- file_io.md — file operations

1.8.0.10.1 sys() — Direct Syscalls

1.8.0.11 Three Tiers Aria provides direct syscall access at three safety levels:

Function	Tier	Syscalls Available	Return Type	TOS Layer
`sys(CONST, args...)`	Safe	~55 curated whitelist	`Result<int64>`	Layer 1
`sys!!(CONST, args...)`	Full	All syscalls	`Result<int64>`	Layer 2
`sys!!!(expr, args...)`	Raw	Any int64 expression	`int64` (bare)	Layer 3

1.8.0.12 Safe Tier — sys() Curated whitelist of ~55 safe syscalls. Returns `Result<int64>`:

```
int64:fd = sys(SYS_OPEN, path, flags, mode) ? -1i64;
```

1.8.0.13 Full Tier — sys!!() All syscalls available, but still returns Result:

```
int64:result = sys!!(SYS_MMAP, addr, length, prot, flags, fd, offset) ? -1i64;
```

1.8.0.14 Raw Tier — sys!!!() Accepts any int64 expression as syscall number. Returns bare int64 (no Result wrapper):

```
int64:result = sys!!!(231i64, 0i64);   // SYS_EXIT_GROUP
```

1.8.0.15 Known Issue Bare filename string literals in `sys()` calls may fail. Store in a variable first:

```
// WRONG: sys(SYS_OPEN, "file.txt", 0i64, 0i64);
// RIGHT:
string:path = "file.txt";
sys(SYS_OPEN, path, 0i64, 0i64);
```

1.8.0.16 Related

- file_io.md — higher-level file operations
- control_flow/error_flow.md — error handling tiers

1.9 Standard Library

1.9.0.0.1 Standard Library — Math

1.9.0.1 Functions Standard math functions available via extern libm:

Function	Description
`sin(x)`	Sine
`cos(x)`	Cosine
`sqrt(x)`	Square root
`abs(x)`	Absolute value
`pow(x, y)`	Power
`log(x)`	Natural logarithm
`floor(x)`	Floor
`ceil(x)`	Ceiling

1.9.0.2 Usage

```
extern "libm" {
    func:sin = double(double:x);
    func:cos = double(double:x);
    func:sqrt = double(double:x);
}

flt64:result = sin(3.14159);
```

1.9.0.3 Related

- types/flt.md — floating point types
- types/fix256.md — deterministic math alternative

1.9.0.3.1 Standard Library Overview

1.9.0.4 What's in stdlib/ The standard library provides common functionality as importable Aria modules. Located at `REPOS/aria/stdlib/`.

1.9.0.5 Import Pattern

```
use "module_name.aria".*;    // bare filename - compiler searches stdlib/
```

1.9.0.6 Available Categories

Category	Modules	See
String	string_utils	string_utils.md
Math	math functions via libm	math.md
I/O	print, readFile, writeFile	io_system/
Threading	thread pool, channels	advanced_features/concurrency.md

1.9.0.7 Built-in (No Import Needed) These are available without `use`: - `print()`, `println()` - `astack`, `apush`, `apop`, `apeek`, `acap`, `asize`, `afits`, `atype` - `ahash`, `ahset`, `ahget`, `ahcount`, `ahsize`, `ahfits`, `ahtype` - `sys()`, `sys!!()`, `sys!!!()`

1.9.0.8 Related

- modules/use_import.md — import syntax
- modules/packages.md — third-party packages

1.9.0.8.1 Standard Library — String Utilities

1.9.0.9 Functions

Function	Signature	Description
`string_length(s)`	`(string) → int64`	Character count (UTF-8 aware)
`string_concat(a, b)`	`(string, string) → string`	Concatenate two strings
`string_contains(s, sub)`	`(string, string) → bool`	Check if substring exists

1.9.0.10 Usage

```
use "string_utils.aria".*;

string:name = "Hello, World!";
int64:len = raw string_length(name);      // 13
bool:has = raw string_contains(name, "World");  // true
```

1.9.0.11 Related

- types/string.md — string type details
- operators/string_ops.md — string operators

1.10 Advanced Features

1.10.0.0.1 Concurrency

1.10.0.1 Overview Aria's concurrency model provides four layers of abstraction:

Layer	API	Use Case
Threads	`aria_shim_thread_*`	Direct POSIX thread control
Thread Pool	`aria_shim_pool_*`	Task-parallel work distribution
Channels	`aria_shim_channel_*`	Typed message-passing between threads
Actors	`aria_shim_actor_*`	Self-contained message-processing entities

All concurrency primitives are accessed via `extern` declarations to the runtime shim layer.

1.10.0.2 1. Threads

1.10.0.2.1 Spawning a Thread

```
extern func:aria_shim_thread_spawn = int64(int64:func_ptr, int64:arg);
extern func:aria_shim_thread_join = int32(int64:handle);
extern func:aria_shim_thread_detach = int32(int64:handle);
extern func:aria_shim_thread_yield = NIL();
extern func:aria_shim_thread_sleep_ms = NIL(int64:ms);
extern func:aria_shim_thread_hardware_concurrency = int32();
extern func:aria_shim_thread_current_id = int64();

func:worker = int64(int64:arg) {
    // Worker body - arg is user-provided data
    pass arg * 2i64;
}

func:main = int32() {
    int64:handle = aria_shim_thread_spawn(@worker, 42i64);
    drop aria_shim_thread_join(handle);
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.3 2. Thread Pool

```
extern func:aria_shim_pool_create = int64(int32:num_workers);
extern func:aria_shim_pool_submit = int32(
    int64:handle, int64:func_ptr, int64:arg);
extern func:aria_shim_pool_shutdown = int32(int64:handle);
extern func:aria_shim_pool_wait_idle = int32(int64:handle);
extern func:aria_shim_pool_active_tasks = int64(int64:handle);
extern func:aria_shim_pool_pending_tasks = int64(int64:handle);
```

```
func:task = int64(int64:arg) {
    pass arg + 1i64;
}

func:main = int32() {
    int32:cores = aria_shim_thread_hardware_concurrency();
    int64:pool = aria_shim_pool_create(cores);

    // Submit work
    loop(0i64, 100i64, 1i64) {
        drop aria_shim_pool_submit(pool, @task, $);
    }

    // Wait for all tasks to complete
    drop aria_shim_pool_wait_idle(pool);
    drop aria_shim_pool_shutdown(pool);
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.4 3. Channels Typed message-passing between threads. Three channel modes:

Mode	Creation	Behavior
Buffered	`channel_create(capacity)`	Send blocks when full, recv blocks when empty
Unbuffered	`channel_create_unbuffered()`	Send blocks until receiver is ready (rendezvous)
Oneshot	`channel_create_oneshot()`	Single send, single recv, then auto-closes

```
extern func:aria_shim_channel_create = int64(int32:capacity);
extern func:aria_shim_channel_send = int32(int64:handle, int64:value);
extern func:aria_shim_channel_recv = int64(int64:handle);
extern func:aria_shim_channel_try_send = int32(int64:handle, int64:value);
extern func:aria_shim_channel_try_recv = int64(int64:handle);
extern func:aria_shim_channel_close = int32(int64:handle);
extern func:aria_shim_channel_destroy = int32(int64:handle);
extern func:aria_shim_channel_count = int32(int64:handle);
extern func:aria_shim_channel_is_closed = int32(int64:handle);

func:producer = int64(int64:ch) {
    loop(1i64, 10i64, 1i64) {
        drop aria_shim_channel_send(ch, $);
    }
    drop aria_shim_channel_close(ch);
    pass 0i64;
}
```

```
func:consumer = int64(int64:ch) {
    int64:sum = 0i64;
    while (aria_shim_channel_is_closed(ch) == 0i32) {
        int64:val = aria_shim_channel_recv(ch);
        sum = sum + val;
    }
    pass sum;
}

func:main = int32() {
    int64:ch = aria_shim_channel_create(16i32);
    int64:prod = aria_shim_thread_spawn(@producer, ch);
    int64:cons = aria_shim_thread_spawn(@consumer, ch);

    drop aria_shim_thread_join(prod);
    drop aria_shim_thread_join(cons);
    drop aria_shim_channel_destroy(ch);
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.4.1 Channel Select Wait on multiple channels simultaneously:

```
extern func:aria_shim_channel_select2 = int32(
    int64:ch0, int64:ch1, int64:timeout_ms);

// Returns index of ready channel (0 or 1), or -1 on timeout
int32:ready = aria_shim_channel_select2(ch_a, ch_b, 5000i64);
```

1.10.0.5 4. Actors Self-contained entities that process messages from a mailbox (built on channels + threads):

```
extern func:aria_shim_actor_spawn = int64(int64:behavior_func);
extern func:aria_shim_actor_send = int32(int64:handle, int64:message);
extern func:aria_shim_actor_stop = int32(int64:handle);
extern func:aria_shim_actor_destroy = int32(int64:handle);
extern func:aria_shim_actor_is_alive = int32(int64:handle);
extern func:aria_shim_actor_pending = int32(int64:handle);
```

1.10.0.6 5. Synchronization Primitives

1.10.0.6.1 Mutexes

```
extern func:aria_shim_mutex_create = int64(int32:type);
extern func:aria_shim_mutex_lock = int32(int64:handle);
extern func:aria_shim_mutex_trylock = int32(int64:handle);
extern func:aria_shim_mutex_unlock = int32(int64:handle);
extern func:aria_shim_mutex_destroy = int32(int64:handle);
```

1.10.0.6.2 Condition Variables

```
extern func:aria_shim_condvar_create = int64();
extern func:aria_shim_condvar_wait = int32(int64:cv, int64:mutex);
extern func:aria_shim_condvar_timedwait = int32(
    int64:cv, int64:mutex, int64:timeout_ns);
extern func:aria_shim_condvar_signal = int32(int64:handle);
extern func:aria_shim_condvar_broadcast = int32(int64:handle);
extern func:aria_shim_condvar_destroy = int32(int64:handle);
```

1.10.0.6.3 Read-Write Locks

```
extern func:aria_shim_rwlock_create = int64();
extern func:aria_shim_rwlock_rdlock = int32(int64:handle);
extern func:aria_shim_rwlock_wrlock = int32(int64:handle);
extern func:aria_shim_rwlock_unlock = int32(int64:handle);
extern func:aria_shim_rwlock_destroy = int32(int64:handle);
```

1.10.0.7 6. Atomics

Low-level atomic operations with explicit memory ordering:

```
extern func:aria_shim_atomic_int64_create = int64(int64:initial);
extern func:aria_shim_atomic_int64_load = int64(int64:handle, int32:order);
extern func:aria_shim_atomic_int64_store = int32(
    int64:handle, int64:value, int32:order);
extern func:aria_shim_atomic_int64_fetch_add = int64(
    int64:handle, int64:value, int32:order);
extern func:aria_shim_atomic_int64_exchange = int64(
    int64:handle, int64:value, int32:order);
extern func:aria_shim_atomic_int64_destroy = int32(int64:handle);
```

Memory ordering constants (pass as `int32:order`):

Keyword	Value	Guarantee
`relaxed`	0	No synchronization
`acquire`	1	Reads after this see prior writes
`release`	2	Writes before this visible to acquirers
`acq_rel`	3	Combined acquire + release
`seq_cst`	4	Sequentially consistent (strongest)

1.10.0.8 Async/Await

The `async` keyword declares functions that can suspend and resume:

```
async func:fetch_data = string(string:url) {
    // Async operations
    pass data;
}
```

Use `await` to suspend until the async operation completes:

```
string:result = await fetch_data("https://example.com");
```

See functions/async.md for full async documentation.

1.10.0.9 Related

- functions/async.md — async function syntax
- verification.md — `--verify-concurrency` for data race detection
- safety_layers.md — safety levels for concurrent code

1.10.0.9.1 Advanced FFI Patterns

1.10.0.10 Large Struct Passing (LBIM) Types 128 bits are passed via `sret`/`byval` pointers, not registers:

```
// int128, int256, etc. are passed by pointer at ABI level
extern func:big_math = int128(int128:a, int128:b);
// C side receives: i128* sret, i128* byval, i128* byval
```

1.10.0.11 String Return from Extern C functions that return strings must return `AriaString`:

```
// C side
typedef struct { char* data; int64_t length; } AriaString;

AriaString my_string_func(const char* input) {
    AriaString result;
    result.data = strdup(input);
    result.length = strlen(input);
    return result;
}
```

1.10.0.12 Handle Pattern Use `int64` instead of pointer types in extern blocks to avoid `{i1, ptr}` wrapper issues:

```
extern func:create_context = int64();        // returns handle as int64
extern func:destroy_context = NIL(int64:h);  // takes handle as int64
```

1.10.0.13 Store Before Pass When passing extern function results directly to `pass`, store in a variable first:

```
// WRONG: pass(extern_func());
// RIGHT:
int32:val = extern_func();
pass val;
```

1.10.0.14 Related

- functions/extern.md — basic extern usage
- types/string.md — string ABI
- types/int.md — LBIM types

1.10.0.14.1 JIT Compilation in Aria

1.10.0.15 Overview Aria includes a built-in JIT assembler for runtime code generation. The JIT subsystem builds on two foundations:

- **WildX** (`wildx_alloc.cpp`) — W X memory management with ASLR, guard pages, code signing, and quota enforcement
- **Assembler** (`assembler.cpp`) — x86-64 instruction encoder with label backpatching, register allocation, peephole optimization, and instruction selection

The JIT is accessible from Aria code through the `jit` stdlib package, which provides FFI bindings to the C++ assembler API.

1.10.0.16 Architecture

```
              Aria Source Code
        use jit;  use wildx;

          jit.aria FFI Bindings
   81 bindings + 17 helpers + constants

          Assembler Pipeline

   IR Queue  →  Peephole  →  Liveness
   (lazy)       Optimizer    Analysis

       ↓             ↓            ↓

   Linear    →  Insn     →  Machine
   Scan RA     Selection     Code Emit


       WildX - W X Memory Manager
  ASLR | Guard Pages | Code Signing | Quota

             x86-64 Hardware
```

1.10.0.17 Instruction Set (v0.7.2+)

The assembler supports 45+ x86-64 instructions across multiple categories:

1.10.0.17.1 Integer

- **Data movement:** `MOV r64, imm64` / `MOV r64, r64`
- **Arithmetic:** `ADD`, `SUB`, `IMUL` (r64,r64 and r64,imm32)
- **Bitwise:** `XOR`, `AND`, `OR`, `NOT`, `NEG`
- **Shifts:** `SHL`, `SHR`, `SAR` with imm8
- **Compare:** `CMP r64, r64` / `CMP r64, imm32`
- **Stack:** `PUSH`, `POP`
- **Flow:** `JMP`, `JE/JNE/JL/JLE/JG/JGE/JB/JBE/JA/JAE`, `RET`
- **Call:** `CALL r64`, `CALL label`, `CALL abs`

1.10.0.17.2 Floating-Point (SSE2)

- `MOVSD` (reg-reg, load, store), `ADDSD`, `SUBSD`, `MULSD`, `DIVSD`, `UCOMISD`
- XMM0–XMM15 registers

1.10.0.17.3 SIMD (SSE)

- `MOVAPS` (reg-reg, aligned load/store), `ADDPS`, `MULPS`
- Packed float32 (4x f32) operations

1.10.0.17.4 Memory

- `MOV r64, [base+offset]` (load), `MOV [base+offset], r64` (store)
- `LEA r64, [base+offset]` (address computation)
- `store_local` / `load_local` (RBP-relative stack frame)

1.10.0.18 Register Allocator (v0.7.3+) The JIT includes a linear scan register allocator for automatic register assignment:

```
use jit;

fn jit_example() {
    let a = jit.asm_create();
    let v0 = jit.asm_vreg_new_gpr(a);  // virtual register
    let v1 = jit.asm_vreg_new_gpr(a);

    jit.asm_mov_r64_imm64(a, v0, 10);
    jit.asm_mov_r64_imm64(a, v1, 32);
    jit.asm_add_r64_r64(a, v0, v1);
    jit.asm_mov_r64_r64(a, jit.REG_RAX, v0);
    jit.asm_ret(a);

    let guard = jit.asm_finalize(a);
    let result = jit.asm_execute(guard);
    // result == 42
}
```

Features: - 12 allocatable GPRs (RAX, RCX, RDX, RSI, RDI, R8, R9, RBX, R12-R15) - 14 allocatable XMMs (XMM0-XMM13) - Automatic spill/reload when registers are exhausted - Auto prologue/epilogue when callee-saved registers are needed - Mixed physical + virtual register support

1.10.0.19 Peephole Optimizer (v0.7.4) The JIT runs a peephole optimization pass on the IR before register allocation:

Pattern	Optimization	Bytes Saved
`MOV r, 0`	`XOR r, r`	6–7
`MOV r, r`	eliminated	3–4
`ADD r, 0` / `SUB r, 0`	eliminated	7
`SHL r, 0` / `SHR r, 0`	eliminated	4
`MOV r, X; MOV r, Y`	dead store eliminated	10
`MOV r, 2^n; IMUL d, r`	`SHL d, n`	~7
`XOR r, r; ADD r, s`	`MOV r, s`	3–4

Statistics available via `aria_asm_peephole_stats()`.

1.10.0.20 Instruction Selection (v0.7.4) During code emission, the allocator selects optimal machine encodings:

IR Instruction	Selected Encoding	Bytes Saved
`MOV_IMM64` (value 0xFFFFFFFF)	`MOV r32, imm32`	4–5
`CMP r, 0`	`TEST r, r`	4
`ADD r, 1`	`INC r`	4
`SUB r, 1`	`DEC r`	4

IR Instruction	Selected Encoding	Bytes Saved
`ADD/SUB r, imm8`	imm8 form	3

Statistics available via `aria_asm_insn_sel_stats()`.

1.10.0.21 Profiling Integration (v0.7.4) JIT code can be registered with Linux `perf` for profiling:

```
jit.asm_perf_map_register(code_ptr, code_size, "my_jit_function");
// Now visible in: perf record -p <pid> && perf report
```

This writes to `/tmp/perf-<pid>.map` in the format expected by `perf`.

1.10.0.22 WildX Security (v0.7.1) All JIT code runs through WildX's security pipeline: - **ASLR:** Random mmap hints for JIT pages - **Guard pages:** `PROT_NONE` sentinels around executable regions - **Code signing:** FNV-1a hash verified before every execution - **W X:** Strict WRITABLE → EXECUTABLE state machine (never both) - **Quota:** Default 64MB, configurable via `aria_wildx_set_quota()` - **Audit logging:** `--wildx-audit` flag for ALLOC/SEAL/EXEC/FREE events

1.10.0.23 Multi-Architecture (v0.7.4) Architecture detection and abstraction:

```
let arch = jit.asm_get_arch();       // ASM_ARCH_X86_64 or ASM_ARCH_AARCH64
let ok = jit.asm_arch_supported(arch); // true on x86-64
```

AArch64 backend is stubbed for future implementation. The architecture abstraction layer supports querying the current target and checking support before code generation.

1.10.0.24 Safety JIT compilation is a **Layer 3 (raw)** operation — it bypasses all safety guarantees. Executable memory is: - Not bounds-checked - Not type-checked - A security risk if used with untrusted input

Always use WildX guards and code signing for JIT code.

1.10.0.25 Related

- memory_model/wild.md — unmanaged memory
- types/pointer.md — pointer operations
- safety_layers.md — safety layer definitions

1.10.0.25.1 Macros & Compile-Time Evaluation

1.10.0.26 Overview Aria provides three mechanisms for compile-time code generation and evaluation:

1. **AST macros** — Template-based code generation invoked with `name!(args)`
2. **Compile-time evaluation** — `comptime(expr)` for constant folding at compile time
3. **Derive macros** — Auto-generate trait implementations (see Derive Macros)

1.10.0.27 AST Macros

1.10.0.27.1 Declaration Macros use `macro:name = (params) { body; };` syntax, consistent with `func:name`:

```
macro:double_it = (x) {
    x + x;
};
```

1.10.0.27.2 Invocation Invoke a macro with `name!(args)`:

```
func:main = int32() {
    int32:val = double_it!(5i32);
    // val == 10 (expanded to: 5i32 + 5i32)
    exit val;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.27.3 Multiple Parameters

```
macro:add3 = (a, b, c) {
    a + b + c;
};

func:main = int32() {
    int32:result = add3!(1i32, 2i32, 3i32);
    // result == 6
    exit result;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.27.4 How It Works

1. The parser sees `macro:name = (params) { body; };` and stores the AST template
2. When `name!(args)` is encountered, arguments are substituted into the template
3. Expansion happens at parse time — the macro body becomes inline code
4. Type checking runs on the expanded code, not the macro definition

1.10.0.27.5 Key Differences from C Macros

Feature	C Preprocessor	Aria Macros
Expansion level	Text substitution	AST-level substitution
Type safety	None (text)	Full (post-expansion type check)
Hygiene	None	Scoped to expansion site
`#` operator	Stringify	**Pin** (borrow checker)
`##` operator	Token paste	**Does not exist**
Syntax	`#define NAME(x)`	`macro:name = (x) { };`

Important: Aria's `#` operator is the **pin** operator for the borrow checker, NOT a stringify operator.

1.10.0.28 Compile-Time Evaluation

1.10.0.28.1 comptime Expression Evaluate an expression at compile time:

```
func:main = int32() {
    int32:size = comptime(4 * 1024);
    // size == 4096, computed at compile time
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.28.2 comptime Function Mark an entire function for compile-time evaluation:

```
comptime func:factorial = int64(int64:n) {
    if (n <= 1i64) {
        pass 1i64;
    }
    pass n * raw factorial(n - 1i64);
}

func:main = int32() {
    int64:val = comptime(raw factorial(10i64));
    // val == 3628800, computed entirely at compile time
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.28.3 pub comptime Function Export a compile-time function for use in other modules:

```
pub comptime func:align_up = int64(int64:val, int64:align) {
    pass ((val + align - 1i64) / align) * align;
}
```

1.10.0.28.4 comptime Block Execute a block of statements at compile time:

```
func:main = int32() {
    comptime {
        // All statements here execute at compile time
        int32:x = 10;
        int32:y = x * 2;
    };

    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.29 Derive Macros

Auto-generate trait implementations for structs using `#[derive(...)]`:

```
#[derive(Eq, Ord, Clone, ToString, Debug, Hash)]
struct:Point = {
    int32:x;
    int32:y;
};

func:main = int32() {
    Point:a = Point{ x: 1, y: 2 };
    Point:b = Point{ x: 3, y: 4 };

    bool:equal = raw a.eq(b);
    string:repr = raw a.to_string();

    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.29.1 Available Derive Traits

Trait	Method	Description
`Eq`	`eq(other)`	Field-by-field equality
`Ord`	`less_than(other)`	Lexicographic comparison
`Clone`	`clone()`	Shallow copy
`ToString`	`to_string()`	String representation
`Debug`	`debug()`	Debug output with type info
`Hash`	`hash()`	FNV-1a hash

Derived methods return `Result<T>` — use `raw` to unwrap the value.

1.10.0.30 Attribute Macros

Attributes modify declarations at compile time:

```
#[inline]
func:hot_path = int32(int32:x) {
    pass x * 2;
}

#[noinline]
func:cold_path = int32(int32:x) {
    pass x / 2;
}

#[align(64)]
struct:CacheLine = {
    int64:data;
};
```

1.10.0.30.1 Available Attributes

Attribute	Target	Description
`#[inline]`	Functions	Suggest inlining
`#[noinline]`	Functions	Prevent inlining
`#[align(N)]`	Structs/Vars	Set byte alignment
`#[derive(...)]`	Structs	Generate trait impls
`#[gpu_kernel]`	Functions	Mark as GPU kernel
`#[gpu_device]`	Functions	GPU device function
`#[comptime]`	Functions	Compile-time evaluation

1.10.0.31 See Also

- MACRO_AUTHORING_GUIDE.md — Full derive and attribute reference
- Traits — Trait system documentation
- Generics — Generic functions and turbofish

1.10.0.31.1 Rules & Limit — Refinement Types

1.10.0.32 Rules

Rules define compile-time and runtime constraints on types:

```
Rules<int32>:r_positive = {
    $ > 0
};

Rules<int32>:r_small_positive = {
    limit<r_positive>,
    $ < 100
};
```

Multi-type rules (value validated against multiple widths):

```
Rules<int32,int64>:r_positive_wide = {
    $ > 0
};
```

1.10.0.33 limit

Apply Rules as a type annotation:

```
func:process = NIL(limit<r_positive> int32:count) {
    // 'count' is guaranteed > 0
    pass NIL;
}
```

1.10.0.34 Member Access in Rules — $.field

```
Rules<Person>:r_adult = {
    $.age >= 18
};
```

1.10.0.35 Array Rules

```
Rules<int32[]>:arr_first_positive = { $[0] > 0 };
Rules<int32[]>:arr_min_length = { $.length >= 4 };
```

1.10.0.36 Notes

- `$` is the value being checked
- `$.field` accesses struct members
- `$[n]` accesses array elements
- Rules are file-scoped — not importable via `use`

1.10.0.37 Related

- traits.md — behavioral constraints
- functions/design_by_contract.md — requires/ensures

1.10.0.37.1 Safety Layers — TOS (Terms of Safety)

1.10.0.38 Overview Aria's safety model uses explicit layers. Each layer grants more power and less safety:

Layer	Name	Access	Safety
0	Safe	Default Aria code	Full — Result, bounds checks, type safety
1	Controlled	`sys()` safe syscalls	Curated syscall whitelist
2	Supervised	`sys!!()` all syscalls	All syscalls, still returns Result
3	Raw	`sys!!!()`, `wild`, `wildx`	No safety net — you own it

1.10.0.39 TOS Safety Vocabulary Explicit bypass keywords that escalate safety level:

Keyword	Action	Layer
`raw` / `_!`	Extract value, ignore error	1+
`drop` / `_?`	Discard Result entirely	1+
`ok`	Pass potentially unknown value	1+
`?!`	Emphatic unwrap (failsafe on error)	0
`wild`	Unmanaged memory allocation	3
`wildx`	Executable memory allocation	3
`sys!!!()`	Raw syscall	3

1.10.0.40 Philosophy Every safety bypass is **visible in code**. There are no hidden undefined behaviors. When reading Aria code, the `raw`, `wild`, `drop`, `sys!!!` keywords immediately identify where safety guarantees are intentionally relaxed.

1.10.0.41 Related

- control_flow/error_flow.md — error model overview
- types/result.md — Result safety
- io_system/sys.md — syscall tiers

1.10.0.41.1 Traits

1.10.0.42 Overview Traits define shared behavior that types can implement. They serve as constraints for generic type parameters.

```
trait:Describable = {
    func:describe = int32(int32:self);
};
```

1.10.0.43 Implementation

```
impl:Describable:for:int32 = {
    func:describe = int32(int32:self) {
        pass self + 100;
    };
};
```

For struct types:

```
trait:Measurable = {
    func:area = int32(Rect2D:self);
};

impl:Measurable:for:Rect2D = {
    func:area = int32(Rect2D:self) {
        pass self.w * self.h;
    };
};
```

1.10.0.44 As Generic Constraints

```
func<T: Addable>:apply_trait = int32(T:val) {
    pass val;
};
```

1.10.0.45 Dynamic Dispatch — `dyn`

```
func:process = int32(dyn Describable:item) {
    pass item.describe();
};
```

1.10.0.46 Borrow Receivers — `$$i` and `$$m` Trait method parameters can use borrow qualifiers to express ownership semantics:

Qualifier	Meaning	Aliasing Rule
`$$i`	Immutable borrow (shared)	Multiple `$$i` borrows allowed simultaneously
`$$m`	Mutable borrow (exclusive)	Only one `$$m` borrow at a time, no concurrent `$$i`

1.10.0.46.1 Syntax Borrow qualifiers appear before the type in trait method parameter lists:

```
trait:Transformable = {
    func:scale = int32($$i Transformable:self, int32:factor);
    func:mutate = int32($$m Transformable:self, int32:value);
};
```

- `$$i Transformable:self` — borrows `self` immutably (read-only access)

- `$$m Transformable:self` — borrows `self` mutably (read-write access)

1.10.0.46.2 Implementation

```
impl:Transformable:for:Rect2D = {
    func:scale = int32($$i Rect2D:self, int32:factor) {
        // self is immutable - cannot modify fields
        pass self.w * factor;
    };

    func:mutate = int32($$m Rect2D:self, int32:value) {
        // self is mutable - can modify fields
        pass value;
    };
};
```

1.10.0.46.3 When To Use Each

Use `$$i` when	Use `$$m` when
Method only reads data	Method modifies the receiver
Want to allow shared access	Need exclusive access
Pure computation / getters	Setters / state mutation

1.10.0.46.4 Interaction with the Borrow Checker The compiler enforces borrow rules at compile time:

```
// OK: multiple immutable borrows
$$i Rect2D:ref1 = ...;
$$i Rect2D:ref2 = ...;  // fine, both are $$i

// ERROR: mutable + immutable at same time
$$m Rect2D:ref1 = ...;
$$i Rect2D:ref2 = ...;  // compile error: cannot borrow while $$m exists
```

1.10.0.46.5 The # Pin Operator The pin operator `#` guarantees a variable's memory address stays stable. This prevents the GC from relocating pinned data — essential for zero-copy FFI:

```
#my_struct;  // pin: address will not move
```

Pinning is orthogonal to borrowing: a pinned value can still be borrowed with `$$i` or `$$m`, but its address is guaranteed stable for the pin's lifetime.

1.10.0.47 Status Traits are specified in the language design. Implementation status varies — see the RFC at `META/ARIA/TRAITS_AND_BORROW_SEMANTICS_RFC.md`.

1.10.0.48 Related

- rules.md — refinement types (`Rules<T>`, `limit<>`)
- functions/generics.md — generic functions

1.10.0.48.1 Verification in Aria — Formal Proofs with Z3

1.10.0.49 Overview Aria integrates the Z3 SMT solver to verify program properties at compile time. Verification is opt-in via `--verify` flags and covers six domains: Rules/limit constraints, function contracts, integer overflow, concurrency safety, memory safety, and user-driven proofs.

When a property is **Proven**, the compiler can eliminate redundant runtime checks. When **Disproven**, a compile-time diagnostic is emitted with a counterexample. When **Unknown** (solver timeout), the runtime check is preserved.

1.10.0.50 Compiler Flags

Flag	Purpose
`--verify`	Enable all verification phases
`--verify-contracts`	Verify `requires`/`ensures` contracts
`--verify-overflow`	Prove arithmetic won't overflow, eliminate checks
`--verify-concurrency`	Detect data races and deadlocks
`--verify-memory`	Detect use-after-free bugs
`--smt-opt`	Enable SMT-guided optimizations (implies `--verify`)
`--smt-timeout=N`	Per-query Z3 timeout in milliseconds (default: 5000)
`--prove-report`	Proven/Disproven/Unknown per check

Use `--verify` alone to enable everything, or combine individual flags:

```
ariac program.aria -o program \
    --verify-contracts --verify-overflow --prove-report
```

1.10.0.51 1. Rules & Limit Verification Rules define value constraints. The compiler uses Z3 to verify that every `limit<>` binding satisfies its Rules at compile time.

```
Rules<int32>:r_positive = { $ > 0 };
Rules<int32>:r_bounded  = { $ >= 0, $ <= 1000 };

func:main = int32() {
    limit<r_positive> int32:x = 42;      // Proven: 42 > 0
    limit<r_bounded>  int32:y = 500;     // Proven: 500 >= 0 AND 500 <= 1000
    limit<r_positive> int32:z = -1;      // Disproven: -1 > 0 is false
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

Compile with `--verify --prove-report`:

```
[limit] Proven:  'x' satisfies r_positive
[limit] Proven:  'y' satisfies r_bounded
[limit] Disproven: 'z' violates r_positive - counterexample: $ = -1
```

1.10.0.51.1 Rules Propagation When **all** callers of a function pass `limit<>`-constrained arguments, the compiler propagates those constraints to the callee and eliminates redundant checks:

```
Rules<int32>:r_safe = { $ >= 0, $ <= 100 };

func:process = int32(int32:val) {
    pass val * 2;  // overflow check eliminated
```

```
                // if all callers use limit<r_safe>
}

func:main = int32() {
    limit<r_safe> int32:a = 50;
    int32:result = raw process(a);
    exit 0;
}

func:failsafe = int32(tbb32:err) {
    exit 1;
}
```

1.10.0.51.2 Rules Narrowing If `r_child` constraints are a subset of `r_parent`, the compiler proves subsumption:

```
Rules<int32>:r_positive       = { $ > 0 };
Rules<int32>:r_small_positive = { limit<r_positive>, $ < 100 };

// Passing r_small_positive where r_positive is expected: Proven safe
```

1.10.0.52 2. Function Contracts `requires` (preconditions) and `ensures` (postconditions) are verified across call sites using Z3.

```
func:safe_divide = flt64(int32:a, int32:b)
    requires b != 0
{
    pass (a / b);
};

func:clamp = int32(int32:val, int32:lo, int32:hi)
    requires lo <= hi
    ensures result >= lo
{
    if (val < lo) { pass lo; };
    if (val > hi) { pass hi; };
    pass val;
};
```

1.10.0.52.1 Cross-Function Contract Propagation When function A calls function B, the compiler checks that A's context satisfies B's `requires` clause:

```
func:make_positive = int32(int32:x)
    requires x > 0
    ensures result > 0
{ pass x; };

func:double_positive = int32(int32:y)
    requires y > 0
{
    int32:pos = raw make_positive(y);  // Proven: y > 0 satisfies requires x > 0
    pass pos * 2;
};
```

1.10.0.52.2 Loop Invariants

```
func:sum_to_n = int32(int32:n) requires n >= 0 {
    int32:s = 0;
    int32:i = 0;
    while (i < n) invariant s >= 0, i >= 0 {
        s = s + i;
        i = i + 1;
    };
    pass s;
};
```

1.10.0.53 3. Overflow Elimination With `--verify-overflow` or `--smt-opt`, the compiler uses Rules constraints to prove arithmetic is within bounds and eliminates overflow checks:

```
Rules<int32>:SmallPos = { $ >= 0, $ <= 1000 };

func:safe_add = int32() {
    limit<SmallPos> int32:a = 500;
    limit<SmallPos> int32:b = 300;
    int32:result = a + b;    // Proven safe: max 1000+1000 = 2000 < INT32_MAX
    pass result;             // overflow check ELIMINATED
};

func:risky_mul = int32() {
    limit<SmallPos> int32:m = 999;
    limit<SmallPos> int32:n = 999;
    int32:big = m * n;  // Unknown: can't prove all
    pass big;           // overflow check KEPT
};
```

1.10.0.54 4. Concurrency Verification With `--verify-concurrency`, the compiler detects data races on shared variables and deadlocks from inconsistent lock ordering.

1.10.0.54.1 Data Race Detection

```
int32:counter = 0i32;
extern func:aria_shim_mutex_lock = int32(int64:mtx);
extern func:aria_shim_mutex_unlock = int32(int64:mtx);

// UNSAFE - shared write without lock
func:unsafe_writer = int64(int64:arg) {
    counter = counter + 1i32;          // Disproven: data race
    pass 0i64;
};

// SAFE - mutex-guarded write
func:safe_writer = int64(int64:arg) {
    drop aria_shim_mutex_lock(global_mtx);
    counter = counter + 1i32;          // Proven: race-free
    drop aria_shim_mutex_unlock(global_mtx);
    pass 0i64;
};
```

1.10.0.54.2 Deadlock Detection

```
// UNSAFE - cyclic lock ordering
func:lock_ab = NIL() {
    lock(mtx_a); lock(mtx_b);
    unlock(mtx_b); unlock(mtx_a);
};
func:lock_ba = NIL() {
    lock(mtx_b); lock(mtx_a);
    unlock(mtx_a); unlock(mtx_b);
};
// Disproven: potential deadlock - A→B then B→A

// SAFE - consistent ordering
func:lock_ab2 = NIL() {
    lock(mtx_a); lock(mtx_b);
    unlock(mtx_b); unlock(mtx_a);
};
func:lock_ab3 = NIL() {
    lock(mtx_a); lock(mtx_b);
    unlock(mtx_b); unlock(mtx_a);
};
// Proven: deadlock-free
```

1.10.0.55 5. Memory Safety With `--verify-memory`, the compiler tracks allocation/free state of `wild` pointers and detects use-after-free bugs.

```
// UNSAFE - use after free
func:test_uaf = NIL() {
    wild ?->:buf = malloc(64i64);
    free(buf);
    wild ?->:alias = buf;     // Disproven: buf is freed
};

// SAFE - alias before free
func:test_safe = NIL() {
    wild ?->:buf = malloc(64i64);
    wild ?->:alias = buf;     // Proven: buf is live
    free(buf);
};

// CONDITIONAL - path-dependent UAF
func:test_cond_uaf = NIL(int32:flag) {
    wild ?->:buf = malloc(64i64);
    if (flag > 0i32) { free(buf); };
    wild ?->:use = buf;       // Disproven: buf might be freed on flag > 0 path
};
```

1.10.0.56 6. prove and assert_static User-driven compile-time proof directives:

```
Rules<int32>:Percentage = { $ >= 0, $ <= 100 };

func:example = NIL() {
    limit<Percentage> int32:pct = 50;

    prove(pct >= 0);           // Proven at compile time → erased from binary
    prove(pct <= 100);         // Proven at compile time → erased from binary
```

```
    assert_static(pct > 10);   // Proven → erased
    assert_static(pct > 99);   // Unknown → warning, kept as runtime assert
};
```

Directive	On Proven	On Disproven	On Unknown
`prove(expr)`	Erased	Compile error	Compile error
`assert_static(expr)`	Erased	Warn+assert	Warn+assert

1.10.0.57 SMT-Guided Optimizations (–smt-opt) When `--smt-opt` is passed, the compiler uses Z3 proofs to perform additional optimizations beyond safety checking:

- **Overflow check elimination** — proven-safe arithmetic skips runtime overflow guards
- **Null check elimination** — `Optional` from non-null sources skips nil checks
- **Loop-invariant hoisting** — expressions proven constant across iterations
- **Defaults fallback elimination** — infallible functions skip `?|` fallback code
- **Rules propagation** — inter-procedural constraint propagation

1.10.0.57.1 Performance Impact Typical overhead and speedup from the benchmark suite:

Metric	Value
Compile-time overhead (per flag)	9–19%
Compile-time overhead (–verify-memory)	~114% (small files)
Runtime speedup (–smt-opt, Rules)	~3%
Runtime speedup (–smt-opt, overflow)	~16%
Runtime speedup (–smt-opt, user stack)	~25%

1.10.0.58 Timeout Tuning The default per-query timeout is 5000ms. Reduce for faster compilation at the cost of more Unknown results; increase for complex proofs:

```
ariac program.aria -o program \
    --verify --smt-timeout=2000   # faster
ariac program.aria -o program \
    --verify --smt-timeout=10000  # slower
```

1.10.0.59 Notes

- Verification requires the compiler built with `-DARIA_HAS_Z3=ON`
- The Z3 context is created once per compilation and shared across all queries
- Each verification query creates a lightweight solver from the shared context
- `--prove-report` prints a summary of all Proven/Disproven/Unknown verdicts
- Rules, contracts, and `prove`/`assert_static` compose — use them together
- Disproven results include counterexamples when available

1.10.0.60 Best Practices

1. **Start with contracts** — Add `requires`/`ensures` to public-facing functions first. They document intent and catch bugs at call sites.
2. **Use `limit<Rules>` at boundaries** — Apply Rules constraints where data enters the system (user input, file reads, network data). The solver propagates them inward.
3. **Prefer `prove` for critical invariants** — If a property MUST hold, use `prove()`. A compile error on failure is better than a runtime surprise.

4. **Use `assert_static` for desirable invariants** — When a property should hold but isn't critical, `assert_static` gives a warning and keeps a runtime fallback.
5. **Build with `--smt-opt` in release** — Let proven-safe checks be eliminated. The ~3–25% runtime speedup is free after paying the compile-time cost.
6. **Tune timeouts per project** — Small projects: `--smt-timeout=2000`. Large projects with complex proofs: `--smt-timeout=10000` or higher.
7. **Use `--prove-report` during development** — See exactly what's Proven vs Unknown. Target Unknown results for optimization or stronger annotations.
8. **Compose constraints** — Rules, contracts, and prove/assert_static work together. A `limit<r>` binding inside a function with `requires` gives the solver more to work with.

1.10.0.61 Related

- rules.md — Rules & Limit syntax
- ../functions/design_by_contract.md — requires/ensures
- safety_layers.md — Aria's safety model
- concurrency.md — threading and mutexes

1.10.0.61.1 WebAssembly Target

1.10.0.62 Overview

Aria can target WebAssembly (WASM) via LLVM's wasm32 backend.

1.10.0.63 Limitations

Current WASM limitations: - No direct syscall access (`sys()` not available) - No `wildx` (executable memory) - No POSIX threads (single-threaded only) - Limited file I/O (browser sandbox) - No FFI with native C libraries

1.10.0.64 Building for WASM

```
ariac --target wasm32 source.aria -o output.wasm
```

1.10.0.65 Related

- functions/extern.md — FFI (not available in WASM)
- io_system/sys.md — syscalls (not available in WASM)

1.11 Debugging

1.11.0.0.1 Debugging

1.11.0.1 Debug Output Use the `stddbg` stream (FD 3) for debug output that doesn't mix with stdout/stderr.

1.11.0.2 Print Debugging

```
println(`DEBUG: x = &{x}, y = &{y}`);
```

1.11.0.3 Type Inspection

- `atype(handle)` — type tag of top astack element
- `ahtype(handle, key)` — type tag of ahash entry

1.11.0.4 Common Issues See `META/ARIA/KNOWN_ISSUES.md` for current known issues and workarounds.

1.11.0.4.1 Constant Arithmetic Bug `fixed` with arithmetic expressions evaluates to 0. Compute in a variable:

```
// WRONG: fixed int64:NEG1 = 0i64 - 1i64;   // → 0
// RIGHT: int64:neg1 = (0i64 - 1i64);
```

1.11.0.4.2 String Literal in sys() Store in variable first:

```
// WRONG: sys(SYS_OPEN, "file.txt", 0i64, 0i64);
// RIGHT: string:path = "file.txt";
//        sys(SYS_OPEN, path, 0i64, 0i64);
```

2 Part III: Safety Walkthrough

2.0.0.1 Safety-Critical Development in Aria: A Walkthrough

Version: 0.3.4
Date: 2026-04-10
Status: Living document

2.0.0.2 Table of Contents

1. Introduction
2. The Application: Medical Infusion Pump
3. Feature 1: failsafe — The Mandatory Safety Net
4. Feature 2: Result<T> — No Silent Failures
5. Feature 3: TBB — Sticky Error Propagation
6. Feature 4: limit<Rules> — Z3 Verification
7. Feature 5: Borrow Semantics
8. Feature 6: Emergency Operators — ?! and !!!
9. Feature 7: wild — Controlled Unsafe Access
10. Feature 8: sys() — Tiered Syscall Safety
11. Feature 9: *? and* ! Shorthand
12. Complete Pump Controller
13. Z3 Verification in Practice
14. Comparison with Other Approaches
15. Summary

2.0.0.3 1. Introduction

Safety-critical software — code that controls medical devices, aircraft, nuclear reactors, and autonomous vehicles — has a fundamental problem: **the consequences of a bug are not a stack trace, they are injury or death.**

Traditional approaches to this problem fall into two camps:

1. **Restricted subsets** (MISRA C, SPARK/Ada): take an existing language, ban the dangerous parts, and add external analysis tools.
2. **Formal methods bolted on** (Frama-C, CBMC): write normal C, then attempt to prove properties after the fact with separate tools.

Both approaches share a weakness: **safety is an afterthought.** The language itself does not enforce safety — external tools and coding standards do. A developer who forgets to run the checker, or who disables a warning, is back to writing unsafe code.

Aria takes a different approach: **safety is structural.** The compiler itself refuses to produce a binary that lacks mandatory error handling, that uses unchecked results, or that violates declared constraints. You cannot forget to handle errors because the compiler will not let you. You cannot silently overflow a safety-critical integer because the type system traps it.

This walkthrough demonstrates Aria's safety features through a realistic application — a medical drug infusion pump controller — building up from individual features to a complete, compilable program.

2.0.0.4 Who This Document Is For

- Safety engineers evaluating Aria for safety-critical domains
- Language researchers comparing error-handling and verification models
- Developers curious about Aria's safety guarantees and how they differ from Rust, Ada/SPARK, or traditional C/C++ approaches

2.0.0.5 Prerequisites

- The `ariac` compiler (v0.3.4+)
- Basic familiarity with imperative programming
- No prior Aria experience required

2.0.0.6 Companion Files Each section references a standalone, compilable example in the `examples/` directory:

File	Feature
`01_failsafe_basics.aria`	Mandatory failsafe handler
`02_result_handling.aria`	Result<T> with pass/fail
`03_tbb_propagation.aria`	TBB sticky error types
`04_limit_rules.aria`	limit<Rules> constraints
`05_borrow_safety.aria`	Borrow semantics (i/m)
`06_emergency_operators.aria`	?! and !!! operators
`07_infusion_pump.aria`	Complete pump controller
`08_syscall_safety.aria`	sys() tiered syscall safety

2.0.0.7 2. The Application: Medical Infusion Pump A drug infusion pump is a device that delivers fluids — typically medication — into a patient's body at a controlled rate. Infusion pump software failures have caused real patient deaths and are a frequent subject of FDA recalls.

Common failure modes include:

- **Overdose:** software calculates too high a flow rate
- **Silent sensor failure:** a disconnected sensor returns stale data
- **Arithmetic overflow:** a rate calculation wraps around to a plausible but wrong value
- **Memory corruption:** a pointer bug overwrites the dosage register
- **Missing error handling:** an error return is ignored, execution continues with garbage data

Our example pump controller will demonstrate how each of Aria's safety features addresses one or more of these failure modes.

The controller performs a single infusion cycle:

```
1. Load and validate patient data
2. Read sensors (temperature, pressure, flow rate)
3. Calculate weight-based dosage using TBB arithmetic
4. Apply limit<Rules> to constrain the final dose
5. Deliver or enter safe mode based on error count
```

2.0.0.8 3. Feature 1: failsafe — The Mandatory Safety Net

2.0.0.9 The Problem In C, error-handling functions like `atexit()` or signal handlers are optional. A developer can write a program with no error handling at all, and the compiler will happily produce a binary. In safety-critical systems, this is unacceptable.

2.0.0.10 Aria's Solution Every Aria program **must** define a `failsafe` function. The compiler checks for its existence and refuses to compile if it is missing. This is enforced at the analysis phase (BUG-005 fix, v0.3.4), not by convention or linting.

2.0.0.11 Syntax

```
func:failsafe = int32(tbb32:err) {
    // err: the TBB error code that triggered this handler
    // Body MUST call exit() - failsafe cannot "resume"
    exit(1);
};
```

2.0.0.12 Key Properties

Property	Detail
Mandatory	Compiler error if missing
Signature	`int32(tbb32:err)` — fixed, not configurable
Error code	`tbb32` — uses TBB type so it cannot be accidentally corrupted
Return	Must call `exit(code)` — this is a terminal handler
Invocation	Called automatically on unhandled errors, or explicitly via `!!!`

2.0.0.13 Example *(See `examples/01_failsafe_basics.aria`)*

```
func:failsafe = int32(tbb32:err) {
    print("EMERGENCY: failsafe triggered\n");
    exit(1);
};

func:main = int32() {
    print("System online - failsafe registered.\n");
    int32:status = 0;
    if (status == 0) {
        print("All subsystems nominal.\n");
    }
    exit(0);
};
```

2.0.0.14 Why This Matters

In traditional safety-critical C (IEC 62304, DO-178C), external coding standards mandate error handling — but the compiler itself does not enforce them. Aria makes the guarantee structural: **no failsafe, no binary.**

2.0.0.15 4. Feature 2: Result<T> — No Silent Failures

2.0.0.16 The Problem

In C, functions signal errors through return codes, `errno`, or null pointers. All of these can be silently ignored:

```
FILE *f = fopen("data.bin", "r");  // f might be NULL
fread(buf, 1, 100, f);             // UB if f is NULL - no compiler warning
```

In C++, exceptions can be unhandled. In Go, errors are values that can be discarded with `_`.

2.0.0.17 Aria's Solution

Functions that can fail return `Result<T>` implicitly. The caller **must** check `.is_error` before accessing `.value` — the compiler enforces this rule (called "no-checky-no-val"). Accessing `.value` without a prior `.is_error` check is a compile error.

2.0.0.18 Syntax

```
// Declaring a function that can fail:
func:read_sensor = int32() {
    pass(42i32);    // success - wraps 42 in Result<int32>
    fail(1i32);     // error   - wraps 1 as error code
};

// Calling and checking:
Result<int32>:r = read_sensor();
if (r.is_error) {
    int32:code = r.error;    // safe - checked first
} else {
    int32:val = r.value;     // safe - checked first
}
```

2.0.0.19 Error Codes

Aria defines standardized error code ranges:

Range	Meaning	Examples
`ERR`	Unknown/unclassified	TBB min sentinel
`0`	No error / success	—
`-1` to `-16`	System errors	-1 SYS_GENERAL, -3 SYS_IO, -7 SYS_OVERFLOW
`1` to `10`	User-defined errors	1 USR_GENERAL, application-specific

2.0.0.20 Example

(See `examples/02_result_handling.aria`)

```
func:read_temperature = int32() {
    pass(365i32);    // 36.5 °C in tenths
};

func:read_pressure = int32() {
    fail(1i32);     // sensor offline
};

func:main = int32() {
    Result<int32>:temp = read_temperature();
    if (temp.is_error) {
        print("Temperature sensor failed\n");
    } else {
        int32:t = temp.value;
        print("Temperature OK\n");
    }

    Result<int32>:pres = read_pressure();
    if (pres.is_error) {
        int32:code = pres.error;
        print("Pressure sensor offline\n");
    } else {
        int32:p = pres.value;
        print("Pressure nominal\n");
    }
```

```
    exit(0);
};
```

2.0.0.21 Why This Matters The compiler guarantees at the type level that **every error path is explicitly handled**. There is no mechanism to silently discard an error — no `_`, no unchecked return codes, no unhandled exceptions.

2.0.0.22 5. Feature 3: TBB — Sticky Error Propagation

2.0.0.23 The Problem In IEEE 754 floating-point, `NaN` propagates through arithmetic: `NaN + 1 = NaN`. This is valuable — it prevents corrupted data from producing plausible results. But integers have no equivalent. In C, `INT_MAX + 1` is undefined behavior. In most languages, integer overflow silently wraps to a negative number.

This is catastrophic in safety-critical systems. Imagine a dosage calculation:

```
dose = patient_weight * rate_per_kg
```

If `patient_weight` overflows to a small positive number due to a sensor bug, the dose might appear valid but be dramatically wrong.

2.0.0.24 Aria's Solution TBB (Trusted Balanced Bitfield) types are integers with a **sentinel error value** at the type minimum. Once a TBB variable enters the ERR state — through overflow, underflow, or division by zero — it **stays** in ERR through all subsequent arithmetic.

Type	ERR Sentinel
`tbb8`	-128
`tbb16`	-32768
`tbb32`	-2147483648
`tbb64`	-9223372036854775808

2.0.0.25 Sticky Behavior The critical property is that **errors never heal:**

```
ERR + 1     = ERR      (not a valid number)
ERR * 0     = ERR      (not zero - errors don't annihilate!)
ERR - ERR   = ERR      (not zero - errors don't cancel!)
1 + ERR     = ERR      (commutativity preserved)
```

This is fundamental. In C, `INT_MIN * 0 = 0`, which means an overflow error can silently disappear. TBB guarantees this cannot happen.

2.0.0.26 Example *(See `examples/03_tbb_propagation.aria`)*

```
func:main = int32() {
    // Normal arithmetic works as expected
    tbb32:sensor_a = 100;
    tbb32:sensor_b = 200;
    tbb32:sum = sensor_a + sensor_b;    // 300

    // Overflow produces ERR
    tbb32:big = 2147483647;
    tbb32:one = 1;
    tbb32:overflow = big + one;          // ERR (-2147483648)
```

```
    // ERR is sticky - it NEVER heals
    tbb32:still_bad = overflow + sensor_a;  // ERR
    tbb32:zero = 0;
    tbb32:mul_zero = overflow * zero;       // ERR (not zero!)
    tbb32:self_sub = overflow - overflow;   // ERR (not zero!)

    exit(0);
};
```

2.0.0.27 Application: Dosage Calculation In the infusion pump controller, the dose calculation uses TBB arithmetic:

```
tbb32:w = weight_kg;
tbb32:base = rate_per_kg;
tbb32:dose = w * base;

tbb32:err_sentinel = -2147483648;
if (dose == err_sentinel) {
    fail(-7);   // SYS_OVERFLOW - propagate error up
}
```

If **any** input is corrupted, the output is guaranteed to be ERR. The pump cannot deliver a corrupted dose because the ERR sentinel will be caught by the check, or by the `limit<Rules>` constraint (which rejects ERR values since they are negative and outside the valid dose range).

2.0.0.28 6. Feature 4: limit<Rules> — Value Constraints with Z3 Verification

2.0.0.29 The Problem Safety-critical systems have **value constraints** that must never be violated: a drug dose must not exceed the maximum, a motor speed must stay below the redline, a reactor temperature must remain within safe bounds.

In traditional languages, these constraints live in comments, documentation, or runtime assertions that can be forgotten, disabled, or bypassed.

2.0.0.30 Aria's Solution Aria's `Rules` declarations define named constraints. The `limit<RuleName>` annotation on a variable means the compiler and runtime enforce those constraints at every assignment. With Z3 verification (`--verify`), the compiler can **prove** constraints at compile time — before the program ever runs.

2.0.0.31 Syntax

```
// Scalar constraint: dose must be 1-5000 mg
Rules:r_safe_dose = {
    $ > 0,
    $ <= 5000
};

// Struct field constraint: patient must be adult
Rules<Patient>:r_adult = {
    $.age_years >= 18,
    $.weight_kg >= 30
};
```

```
// Cascading: pediatric dose inherits safe dose + adds upper bound
Rules:r_pediatric = {
    limit<r_safe_dose>,
    $ <= 500
};

// Apply to a variable:
limit<r_safe_dose> int32:dose = 250;   // OK - 250 is in [1, 5000]
limit<r_safe_dose> int32:bad  = 9999;  // Runtime error → failsafe
```

2.0.0.32 Z3 Verification When compiled with `--verify`, the compiler translates Rules constraints into Z3 SMT assertions and attempts to prove them statically:

```
ariac --verify 04_limit_rules.aria
```

If the compiler can prove a constraint is always satisfied, no runtime check is needed. If it finds a counter-example, it reports the violating values. If it can neither prove nor disprove, a runtime check is emitted as a safety net.

2.0.0.33 Example *(See `examples/04_limit_rules.aria`)*

```
Rules:r_safe_dose_mg = {
    $ > 0,
    $ <= 5000
};

Rules:r_pediatric_dose = {
    limit<r_safe_dose_mg>,
    $ <= 500
};

struct:Patient = {
    int32:weight_kg;
    int32:age_years;
};

Rules<Patient>:r_adult_patient = {
    $.weight_kg >= 30,
    $.age_years >= 18
};

func:main = int32() {
    limit<r_safe_dose_mg> int32:dose = 250;              // OK
    limit<r_pediatric_dose> int32:child_dose = 100;       // OK (cascade)

    stack Patient:pat = Patient{weight_kg: 80, age_years: 35};
    limit<r_adult_patient> Patient:adult = pat;           // OK
    exit(0);
};
```

2.0.0.34 Application: Multi-Layer Dosage Protection In the infusion pump, constraints form a multi-layer defense:

1. **Weight constraint** (`r_valid_weight`): patient weight 1–300 kg
2. **Dose constraint** (`r_safe_dose`): final dose 1–5000 mg

3. **Patient eligibility** (`r_eligible_patient`): age 18, weight 30 kg
4. **Flow constraint** (`r_safe_flow`): delivery rate 1–1000 mL/hr

A dosage that somehow passes the calculation check, passes the patient-max-dose clamp, but falls outside 1–5000 mg would be caught by `limit<r_safe_dose>` and trigger the failsafe handler.

2.0.0.35 Why This Matters Unlike runtime assertions in C (`assert(dose <= 5000)`) which can be compiled out with `NDEBUG`, Aria's constraints are part of the type system. They cannot be disabled. With Z3, they can be verified at compile time — providing the strongest possible guarantee without runtime cost.

2.0.0.36 7. Feature 5: Borrow Semantics — Compile-Time Memory Safety

2.0.0.37 The Problem Memory corruption is the leading cause of security vulnerabilities (per CISA, NSA, and Microsoft). In safety-critical systems, a corrupted pointer can overwrite a dosage register, crash a flight controller, or cause a reactor SCRAM.

C and C++ provide no structural protection against: - Use-after-free - Data races from concurrent aliased mutation - Dangling pointers

2.0.0.38 Aria's Solution Aria uses explicit borrow annotations with two rules enforced at compile time:

Annotation	Meaning	Rule
`$$i Type:ref = var;`	Immutable borrow	Any number may coexist
`$$m Type:ref = var;`	Mutable borrow	At most one; no $$i borrows on same variable

This prevents the fundamental aliasing problem: you can never have a mutable reference coexisting with any other reference to the same data.

2.0.0.39 Example *(See `examples/05_borrow_safety.aria`)*

```
func:main = int32() {
    int32:sensor_reading = 42;

    // Multiple immutable borrows - ALLOWED
    $$i int32:view_a = sensor_reading;
    $$i int32:view_b = sensor_reading;
    int32:sum = view_a + view_b;   // 84

    // Single mutable borrow - write-through
    int32:actuator_position = 0;
    $$m int32:control = actuator_position;
    control = 100;
    // actuator_position is now 100

    exit(0);
};
```

2.0.0.40 Compile-Time Rejection The compiler rejects conflicting borrows:

```
int32:x = 10;
$$i int32:reader = x;
$$m int32:writer = x;   // COMPILE ERROR: $$i borrow active on x
```

This is checked **at compile time** — no runtime overhead, no possibility of the check being disabled.

2.0.0.41 Application: Safe Patient Data Access In the infusion pump, patient data is accessed through immutable borrows:

```
$$i int32:pat_weight = safe_weight;
$$i int32:pat_max = checked_patient.max_dose_mg;
```

This guarantees that the dosage calculation cannot accidentally modify patient data. If a future developer tries to write through these references, the compiler will reject the code.

2.0.0.42 8. Feature 6: Emergency Operators — ?! and !!!

2.0.0.43 The Problem Error handling in safety-critical systems must be both **convenient** (so developers actually use it) and **auditable** (so reviewers can trace every error path). Exception-based languages make error handling convenient but difficult to audit. Return-code languages make error handling auditable but tedious.

2.0.0.44 Aria's Solution Aria provides two operators that form a **two-level error escalation system**:

Operator	Name	Level	Behavior
`?!`	Emphatic unwrap	Local	Unwrap Result; use default on error
`!!!`	Direct failsafe	Global	Immediately invoke failsafe handler

Level 1 — `?!` (local recovery): "I can handle this with a safe default."

```
Result<int32>:sensor = read_sensor();
int32:value = sensor ?! safe_default;
// On success: value = sensor.value
// On error:   value = safe_default
```

Level 2 — `!!!` (global escalation): "This is catastrophic. Emergency stop."

```
!!! -2;   // Immediately calls failsafe(-2), program terminates
```

2.0.0.45 Example *(See `examples/06_emergency_operators.aria`)*

```
// Level 1: Local recovery with ?!
Result<int32>:ok_reading = read_sensor_ok();
int32:val_ok = ok_reading ?! 50i32;    // val_ok == 98 (unwrapped)

Result<int32>:bad_reading = read_sensor_fail();
int32:val_bad = bad_reading ?! 50i32;  // val_bad == 50 (default)

// Level 2: Escalate after exhausting local recovery
if (consecutive_failures >= 3) {
```

```
    !!! -2;   // trigger failsafe - emergency shutdown
}
```

2.0.0.46 Application: Sensor Fallback Cascade In the infusion pump, ?! provides graceful degradation:

```
// Temperature: use body-temp default if sensor fails
int32:temperature = temp_result ?! 370i32;

// Pressure: use 100 mmHg default if sensor fails
int32:pressure = pres_result ?! 100i32;

// Flow rate: use 0 (safe stop) if sensor fails
int32:flow = flow_result ?! 0i32;
```

Each sensor has a domain-specific safe default. If too many sensors fail, the controller enters safe mode rather than delivering an uncertain dose.

2.0.0.47 Auditability Both operators are syntactically distinctive — `?!` and `!!!` stand out in code review. A security auditor can grep for these tokens to find every error recovery and escalation point in the codebase.

2.0.0.48 9. Feature 7: wild — Controlled Unsafe Access

2.0.0.49 The Problem Safety-critical systems sometimes need to interact with hardware registers, DMA buffers, or foreign-function interfaces that require raw pointer access. In Rust, this is `unsafe {}`. In Ada, it is `pragma Import`. The challenge is making unsafe code **visible and auditable** while still allowing it when needed.

2.0.0.50 Aria's Approach The `wild` keyword declares raw, unmanaged pointers that bypass the borrow checker:

```
wild void->:hw_reg = @extern("get_hardware_register", 0x40001000);
```

`wild` pointers are: - **Explicitly marked** — they cannot be confused with safe references - **Grep-able** — every use of `wild` in a codebase is an audit point - **Unrestricted** — they support pointer arithmetic, casts, and FFI

The philosophy is the same as Rust's `unsafe`: make the dangerous parts visible so reviewers know exactly where to focus.

2.0.0.51 When To Use wild

Use Case	Example
Hardware register access	`wild uint32->:reg = ...;`
FFI with C libraries	`extern func:malloc = wild void->(int64:size);`
DMA buffer management	`wild uint8->:dma_buf = ...;`
Performance-critical paths	Manual memory layouts

In the infusion pump controller, `wild` would be used for the actual hardware interface layer — reading ADC registers, controlling valve GPIOs, and communicating over CAN bus. The example code uses simulated functions instead to keep the demonstrations portable and compilable without hardware.

2.0.0.52 10. Feature 8: sys() — Tiered Syscall Safety *(See `examples/08_syscall_safety.aria`)*

Direct syscalls bypass libc — one wrong argument can corrupt memory or kill a process. Aria applies its TOS escalation model to make syscall danger visible and intentional.

2.0.0.53 The Three Tiers

```
sys(WRITE, 1i64, "safe\n", 5i64);      // Safe tier - curated whitelist
sys!!(FORK);                            // Full tier - all syscalls allowed
sys!!!(1i64, 1i64, "raw\n", 4i64);     // Raw tier - bare int64, no wrapping
```

Tier	Return	Safety	Use Case
`sys()`	`Result<int64>`	Compile-time whitelist (~55 safe syscalls)	Library code
`sys!!()`	`Result<int64>`	All syscalls, named constants required	Process control
`sys!!!()`	`int64`	No validation, any expression	Bare metal

2.0.0.54 Compile-Time Enforcement

The key innovation: `sys()` only accepts named constants from a curated whitelist. You cannot sneak a dangerous syscall through with a variable:

```
sys(WRITE, fd, buf, len);    // OK - WRITE is in safe set
sys(FORK);                   // COMPILE ERROR: FORK not in safe set
int64:nr = 57i64;
sys(nr, 0i64);               // COMPILE ERROR: variables not allowed
```

The error message guides you to the right tier: > `error: 'FORK' is not in the safe syscall set. Use sys!!('FORK', ...) for full access.`

2.0.0.55 Why This Matters for Safety-Critical Systems

1. **Auditable** — `aria-safety` flags every `sys!!` and `sys!!!` usage
2. **Visible** — `grep 'sys!!!'` finds all raw syscall paths in a codebase
3. **Intentional** — you cannot accidentally use the dangerous tiers
4. **Lazy-safe** — the easiest path (`sys()`) is also the safest

For the infusion pump: sensor reading and file logging use safe-tier `sys()`. If hardware control needed `IOCTL` (which *is* in the safe set), no escalation is required. Only truly dangerous operations like process spawning would need `sys!!` — and that would be immediately visible in review.

2.0.0.56 11. Feature 9: *?* *and* *!* — Ergonomic Drop & Raw Shorthand *(See `examples/09_drop_raw_shorthand.aria`)*

Aria v0.4.0 adds prefix shorthand operators for the two TOS-bypass builtins:

Shorthand	Desugars to	Operator Family
`_?expr`	`drop(expr)`	`?` family (safe/default)
`_!expr`	`raw(expr)`	`!` family (danger/force)

These are **pure parser sugar** — `_?` and `_!` are converted to `drop()` and `raw()` call nodes during parsing. The type checker and codegen see the same IR as the verbose forms. No new semantics, no new risks.

2.0.0.57 Why Both Forms? Aria keeps verbose and terse forms so developers can choose the right one for context:

```
// In a long function where readability matters most:
drop(writeFile("audit.log", entry));
int32:val = raw(verified_result);

// In tight imperative code where the intent is obvious:
_?println("tick");
int32:x = _!safe_add(a, b);
```

2.0.0.58 The ? and ! Families The shorthand operators fit naturally into Aria's existing operator families:

? family (safe / provides fallback / discards): - `expr ? default` — unwrap with fallback - `expr??` — null-coalesce - `_?expr` — drop result entirely

! family (danger / force / bypass): - `expr?!` — emphatic unwrap (calls failsafe on error) - `expr!!` — force cast - `!!!` — direct failsafe - `_!expr` — raw extract without check

2.0.0.59 Safety-Critical Perspective For the infusion pump, `_?` and `_!` are flagged by `aria-safety` just like their verbose counterparts. The tool reports `_!` as `[RAW]` severity — the shorthand does not let you sneak past code review:

```
[RAW]  Line 42: _! raw extraction shorthand - bypasses error check (desugars to raw())
[DROP] Line 58: _? drop shorthand - discards Result without checking (desugars to drop())
```

Both `drop()` and `_?` are fully interchangeable. Both `raw()` and `_!` are fully interchangeable. The choice is purely stylistic.

2.0.0.60 12. Putting It Together: The Complete Pump Controller *(See `examples/07_infusion_pump.aria`)*

The complete infusion pump controller combines all features into a coherent program:

2.0.0.61 Architecture

```
                  failsafe()
          Last line of defense
   Closes valves, sounds alarm, logs error

                    invoked by !!! or unhandled error

                   main()


  Load Patient  →  Read Sensors  →  Calculate
    Data             (Result<T>)      Dosage
                                      (TBB)
  limit<Rules>     ?! defaults
  $$i borrows



                                    Constrain
```

```
limit<Rules>

Deliver or
Safe Mode
```

2.0.0.62 Safety Layers in the Code

Layer	Feature	What It Prevents
1	`failsafe()`	Unhandled errors escaping the program
2	`Result<T>`	Silent sensor failures
3	`?!` operator	Sensor failure without safe default
4	TBB arithmetic	Overflow producing plausible wrong dosage
5	`limit<Rules>`	Out-of-range dosage reaching the pump
6	`$$i` borrows	Accidental mutation of patient data
7	Error counting	Delivering dose with too many unknowns

2.0.0.63 Walkthrough of Key Sections **Patient data validation:**

```
stack Patient:patient = Patient{
    id: 1042, weight_kg: 72, age_years: 45, max_dose_mg: 2000
};
limit<r_eligible_patient> Patient:checked_patient = patient;
limit<r_valid_weight> int32:safe_weight = checked_patient.weight_kg;

$$i int32:pat_weight = safe_weight;
int32:max_dose_val = checked_patient.max_dose_mg;
$$i int32:pat_max = max_dose_val;
```

Three constraints are applied before any calculation begins: the patient must be eligible (age/weight), the weight must be in human range, and subsequent access is read-only through immutable borrows.

Sensor reading with local recovery:

```
Result<int32>:temp_result = read_temperature();
int32:temperature = temp_result ?! 370i32;   // default: 37.0 °C

Result<int32>:flow_result = read_flow_rate();
int32:flow = flow_result ?! 0i32;            // default: stop delivery
```

Each sensor has a domain-specific default. The flow sensor defaults to 0 (no delivery) — the safest possible fallback for a failed flow measurement.

TBB-safe dosage calculation:

```
tbb32:w = weight_kg;
tbb32:base = base_dose_per_kg;
tbb32:dose = w * base;

tbb32:err_sentinel = -2147483648;
if (dose == err_sentinel) {
    fail(-7);   // SYS_OVERFLOW
}
```

If either input is corrupted, the TBB multiplication produces ERR, which is caught by the sentinel check. The error cannot propagate silently.

Final constraint enforcement:

```
limit<r_safe_dose> int32:final_dose = clamped;
```

Even after clamping to the patient's maximum, the dose must still pass the global safe-dose rule (1–5000 mg). This is the last automated check before delivery.

Delivery decision:

```
if (errors >= 2) {
    // Multiple sensor failures - do NOT deliver
    exit(0);   // safe mode
}
```

The controller counts errors throughout the cycle. Two or more failures trigger safe mode — no drug is delivered.

2.0.0.64 12. Z3 Verification in Practice Aria v0.3.4 includes a built-in Z3 SMT solver integration that can verify limit<Rules> constraints and function contracts at compile time.

2.0.0.65 Usage

```
#### Verify all constraints
ariac --verify 07_infusion_pump.aria

#### Generate a detailed verification report
ariac --verify --verify-report pump_report.txt 07_infusion_pump.aria

#### Verify specific categories
ariac --verify-contracts 07_infusion_pump.aria     # requires/ensures
ariac --verify-overflow  07_infusion_pump.aria      # integer overflow
```

2.0.0.66 What Z3 Proves The Z3 verifier operates in three phases:

1. **Phase 1 — Constraint Verification:** Checks that every `limit<Rules>` assignment satisfies its constraints. For constant values, this is a direct proof. For computed values, Z3 searches for counter-examples.
2. **Phase 2 — Contract Verification:** Checks `requires` (preconditions) and `ensures` (postconditions) on function signatures.
3. **Phase 3 — Overflow Analysis:** Checks integer arithmetic for potential overflow, especially on safety-critical types.

2.0.0.67 Verification Report Example

```
=== Z3 Verification Report ===
File: 07_infusion_pump.aria

[PASS] limit<r_safe_dose> at line 156: dose   [1, 5000] - PROVED
[PASS] limit<r_valid_weight> at line 134: weight   [1, 300] - PROVED
[PASS] limit<r_eligible_patient> at line 131: age >= 18, weight >= 30 - PROVED
[PASS] limit<r_safe_flow> - no violations found

Constraints verified: 4/4
Counter-examples: 0
Verification time: 0.023s
```

2.0.0.68 Comparison with External Tools

Approach	When	Integrated	Incremental
SPARK/Ada	Compile time	[Y]	[Y]
Frama-C/WP	Post-hoc	[N]	[N]
CBMC	Post-hoc	[N]	[N]
Aria + Z3	**Compile time**	**[Y]**	**[Y]**

Aria's Z3 integration runs as part of the normal compilation pipeline. No separate tool invocation, no configuration files, no annotation language different from the source language.

2.0.0.69 14. Comparison with Other Approaches

2.0.0.70 How Aria Compares to Safety-Critical Alternatives

Feature	C (MISRA)	Ada/SPARK	Rust	**Aria**
Mandatory error handler	[N] Convention	[N] Optional	[N] Optional	**[Y] Compiler-enforced**
Checked results	[N] Return codes	[Y] Exceptions	[Y] Result<T>	**[Y] Result<T> + no-checky-no-val**
Integer overflow safety	[N] UB	[Y] Constraint_Error	[Y] Panic (or wrap)	**[Y] TBB sticky sentinel**
Value constraints	[N] assert()	[Y] Subtypes	[N] (crate-level)	**[Y] limit<Rules> + Z3**
Memory safety	[N] Manual	[Y] (no pointers)	[Y] Borrow checker	**[Y] Borrow (i/m) + wild**
Formal verification	External (Frama-C)	SPARK prover	External (Kani)	**Built-in Z3**
Unsafe escape hatch	Entire language	`pragma Import`	`unsafe {}`	`wild` **keyword**

2.0.0.71 Key Differentiators

1. **failsafe is mandatory, not optional.** No other mainstream language refuses to compile a program that lacks a top-level error handler.
2. **TBB has no equivalent.** Rust panics on overflow (in debug) or wraps (in release). Ada raises Constraint_Error (which can be caught and ignored). TBB errors are **sticky and inescapable** — they propagate through all arithmetic forever.
3. **limit<Rules> with Z3 is first-class.** Ada subtypes are similar in spirit but lack SMT verification. Rust has no equivalent — constraint checking requires external crates and runtime assertions.
4. **Two-level error escalation (?! / !!!) is distinct from try/catch.** The operators are syntactically loud, making error paths visible in code review.

2.0.0.72 15. Summary

Aria's safety model is built on a simple insight: **safety features that can be forgotten will be forgotten.** Every feature described in this walkthrough is either mandatory (failsafe), compiler-enforced (Result checking, borrow rules), or structurally integrated (TBB, limit<Rules>, Z3).

2.0.0.73 The Safety Stack

```
Layer 7:  limit<Rules> + Z3    Provable value constraints
Layer 6:  $$i / $$m borrows    Compile-time memory safety
Layer 5:  TBB types        Sticky error propagation
Layer 4:  ?! operator      Local recovery with safe defaults
Layer 3:  Result<T>        Mandatory explicit error handling
Layer 2:  !!! operator      Global emergency escalation
Layer 1:  failsafe()       Mandatory last line of defense
```

Each layer catches failures that escape the layer above it. A corrupted sensor reading might be caught by `?!` (Layer 4); if not, by TBB propagation (Layer 5); if not, by `limit<Rules>` (Layer 7); if not, by `failsafe` (Layer 1).

2.0.0.74 Compiling the Examples

```
#### Build the compiler
cd aria/build && cmake .. && make -j$(nproc)

#### Compile and run individual examples
./ariac ../../aria-docs/safety-walkthrough/examples/01_failsafe_basics.aria
./01_failsafe_basics

#### Compile with Z3 verification
./ariac --verify ../../aria-docs/safety-walkthrough/examples/04_limit_rules.aria

#### Compile the complete pump controller
./ariac ../../aria-docs/safety-walkthrough/examples/07_infusion_pump.aria
./07_infusion_pump
```

2.0.0.75 Next Steps

- **For safety engineers:** Review the pump controller code and compare it to your current development practices. Consider which failure modes Aria prevents that your current toolchain does not.
- **For language researchers:** The formal comparison in Section 12 is intended as a starting point. We welcome analysis from the verification and safety communities.
- **For developers:** Clone the Aria repository, build the compiler, and try compiling and modifying the example files. Break things. See what the compiler catches.

This document is part of the Aria documentation suite. For language specification, see `specs/aria_specs.txt`. For getting started, see `GETTING_STARTED.md`.

3 Part IV: Appendix — Reference

3.0.0.1 The Aria Philosophy - In the Creator's Words

3.0.0.2 The Origin Story "One thing that really inspired me a lot was my month or two self-teaching x86-64 asm and the nasm macro system. Sure, anything complex is a pain in the ass, BUT, the simplicity always drew me in. The instructions for the most part say exactly what they do: **MOV, RET, JMP, JNE, PUSH, POP, CMP**… I actually loved that part."

3.0.0.3 What I Wanted

3.0.0.4 From Assembly

- **Clarity**: Instructions say what they do. No hunting through docs to understand if `<*>?::<turboBobDiesel>++j` adds integers or fries your GPU
- **Simplicity**: The core should be simple and understandable
- **NASM macro power**: Metaprogramming that actually works

3.0.0.5 From the Ecosystem

- **C interop**: The entire world runs on C when you get down to it
- **No forced tradeoffs**: I didn't want to choose between convenience/safety and power/performance
- **Per-operation control**: Move back and forth as needed, from web app to kernel mod
- **Rust's borrow checker without the year-long learning curve or frustration spiral**
- **JavaScript's functional system without the rest of the insanity**

3.0.0.6 For My Brain (ADHD-Friendly)

- **Guided decisions**: Compiler steers you toward good choices
- **Override when needed**: When guidance doesn't match reality for the current task, you can choose
- **Clear symbols**: One symbol, one meaning (except `*` for C interop necessity)
- **Better I/O organization**:
  - Binary and text don't need to be mixed
  - Debug info goes to its own log
  - Errors go to their own stream
  - I/O goes to its destination
  - **stdOut should only have standard output in text form**

3.0.0.7 Better Foundations

- **More native types**: The ones we have are piss poor for many use cases
- **TBB types**: Reuse -128 and other sentinel values to handle errors AND enable cmov optimizations
- **Balanced types**: Ternary logic isn't just academic - it's useful
- **Arbitrary precision**: int1024, int2048, int4096 for real-world crypto/quantum needs

3.0.0.8 But Here's The Truth "When it came down to it, **every decision got made based on safety for the model and those around it** and supporting the model performance wise as well."

3.0.0.9 Safety First, Everything Else is Gravy The fact that TBB's -128 sentinel enables: - Better error handling (sticky errors that don't cause UB) - Performance optimization (cmov instead of branches)

That's **gravy**. A happy accident from good foundational decisions.

The fact that other features mesh well together? **More gravy**.

3.0.0.10 The Core Priority Safety for the AI model and the developers using it.

That's not negotiable. That's not a tradeoff. That's the north star every decision orbits around.

3.0.0.11 Design Principles That Emerged

3.0.0.12 1. Symbols Say What They Do

Like ASM instructions. No mystery operators. No context-dependent meanings (except `*` for C interop).

```
MOV → move
RET → return
JMP → jump

// In Aria:
Result<T>  // It's a result that wraps T
.is_error  // Boolean: is this an error?
.value     // The value (if not error)
pass()     // Pass (return success)
fail()     // Fail (return error)
```

3.0.0.13 2. No False Dichotomies

You don't choose between: - Safety OR performance - Convenience OR control - Beginners OR experts - High-level OR low-level

You get all of them. You choose per operation.

3.0.0.14 3. ADHD-Friendly Guardrails

- Compiler guides you (default Result wrapping)
- But lets you override (`@nochecky` if you know what you're doing)
- Clear error messages with examples
- No hunting through docs to understand behavior

3.0.0.15 4. Better I/O Separation

Different streams for different purposes: - Standard output → text only - Standard error → errors only
- Debug log → debug info - Binary output → binary data

No mixing. No confusion.

3.0.0.16 5. Safety Through Design, Not Restriction

Bad approach: "You can't do X because it's unsafe"
Aria approach: "You can do X, but the compiler will make sure you do it safely, or ask you to explicitly bypass (auditability)"

Examples: - Result enforcement: Must check, but simple to do - Borrow checker: Safe references without fighting the compiler - wild modifier: Unsafe operations are visible and auditable

3.0.0.17 The Gravy List

Things that turned out great but weren't the primary goal:

3.0.0.18 TBB Types with Sentinel Values

- **Goal:** Better error handling (sticky errors)
- **Gravy:** Enables cmov optimizations, avoiding branches
- **More gravy:** -128 causes bugs in two's complement, so banning it improves safety too

3.0.0.19 One Symbol One Meaning

- **Goal:** Clarity (don't hunt through docs)
- **Gravy:** Makes the language easier to teach
- **More gravy:** Reduces cognitive load for ADHD
- **Even more gravy:** Better tooling (unambiguous parsing)

3.0.0.20 Result Enforcement

- **Goal:** Make sure AI model checks errors
- **Gravy:** Teaches developers good error handling
- **More gravy:** Eliminates whole classes of bugs
- **Even more gravy:** Self-documenting code (you see the checks)

3.0.0.21 Generic System

- **Goal:** Type safety with reusable code
- **Gravy:** Result, HashMap<K,V>, etc. just work
- **More gravy:** Zero runtime cost (monomorphization)
- **Even more gravy:** Clear syntax (no trait soup)

3.0.0.22 Why This Matters for AI Safety When an AI is writing code: 1. **Simpler is safer**: Less ambiguity = fewer misunderstandings 2. **Explicit is verifiable**: Can check the AI's work 3. **Guided is consistent**: AI learns patterns that are already safe 4. **Auditable bypasses**: When AI goes unsafe, it's visible

This isn't about protecting developers from themselves.
This is about creating a language where AI-generated code is naturally safer.

3.0.0.23 The Bottom Line

> "I wanted to not have to go hunt through man pages or online tutorials every five minutes to figure out what operators do. I wanted to know what was happening."

Aria: The language where things mean what they say.

- `MOV` moves
- `Result<T>` is a result
- `.is_error` is a boolean
- `pass()` passes
- `wild` means "I'm going off the rails, audit this"

Safety through clarity.
Performance through good foundations.
Everything else?

Gravy.

This philosophy shapes every design decision in Aria. When in doubt, ask: "Does this make the AI model safer? Does this make meaning clearer? Does this reduce hunting through docs?" If yes to any, consider it. If yes to all, it's probably the right choice.

Remember: The goal is safety. The gravy is that safety turns out to be powerful, performant, and pleasant to use.

3.0.0.24 Aria Compiler Architecture

Purpose: The Aria language compiler - transforms Aria source code into executable binaries via LLVM.

3.0.0.25 Philosophy

3.0.0.26 Design Principles

1. **Errors Are Values** - No exceptions. Result<T,E> everywhere.
2. **Explicit Over Implicit** - No hidden behavior, no magic.
3. **Memory Safety Without Runtime Cost** - Compile-time borrow checking.
4. **Performance Matters** - Zero-cost abstractions, direct LLVM integration.
5. **Balanced Ternary Computing** - Native support for -1/0/+1 computation via TBB types.

3.0.0.27 Non-Negotiables

- **Spec Compliance** - aria_ecosystem/specs/ documents are law
- **No Simplification** - Difficulty is not a reason to deviate
- **No Bloat** - Every feature must serve a clear purpose
- **Test Coverage** - Features without tests don't merge
- **LLVM Integration** - No custom backends, LLVM is the target

---

3.0.0.28 High-Level Architecture

```
Aria Source Code (.aria)
    ↓
[Preprocessor] - Handles #include, conditional compilation
    ↓
[Lexer] - Tokenization (src/frontend/lexer/)
    ↓
[Parser] - AST construction (src/frontend/parser/)
    ↓
[Semantic Analysis] - Type checking, borrow checking (src/frontend/sema/)
    → Type Checker
    → Borrow Checker
    → Exhaustiveness Analyzer
    → Symbol Table
    → Module Resolver
    ↓
[IR Generation] - LLVM IR emission (src/backend/ir/)
    ↓
[LLVM Optimization Passes]
    ↓
[Code Generation] - Platform-specific assembly
    ↓
[Linker] - Final executable
```

3.0.0.29 Component Breakdown

3.0.0.30 Frontend (src/frontend/)

3.0.0.31 Lexer (lexer/)

- **Purpose**: Convert source text into tokens
- **Key Files**:
  - `lexer.cpp` - Main tokenization logic
  - `token.cpp` - Token definitions
- **Responsibilities**:
  - UTF-8 handling
  - Keyword recognition
  - Number literal parsing (including TBB values)
  - String literal processing (including template literals)
  - Position tracking for error messages

3.0.0.32 Parser (parser/)

- **Purpose**: Build Abstract Syntax Tree (AST) from tokens
- **Key Files**:
  - `parser.cpp` - Main parsing logic (~3000 lines)
  - Statement parsing (pick, when, defer, etc.)
  - Expression parsing (precedence climbing)
- **Responsibilities**:
  - Syntax validation
  - AST node construction
  - Error recovery
  - Operator precedence
  - Pattern matching for pick statements

3.0.0.33 AST (ast/)

- **Purpose**: Data structures representing program structure
- **Key Files**:
  - `ast_node.h` - Base AST node class
  - `expr.h` - Expression nodes
  - `stmt.h` - Statement nodes
  - `type.h` - Type nodes
- **Node Types**:
  - Literals, identifiers, binary/unary ops
  - Function calls, array indexing, member access
  - Variable declarations, function declarations
  - Control flow (when, pick, defer)
  - Pick patterns (ranges, wildcards, ERR)

3.0.0.34 Semantic Analysis (sema/)

- **Purpose**: Validate program correctness, infer types, check ownership

3.0.0.34.1 Type Checker (type_checker.cpp)

- Type inference for expressions
- Type compatibility validation
- Must-use result enforcement (nodiscard)
- Pick statement exhaustiveness checking
- Generic type resolution

3.0.0.34.2 Borrow Checker (borrow_checker.cpp)

- Ownership tracking (move semantics)
- Lifetime validation
- Pinning reference analysis
- Appendage Theory enforcement
- Use-after-move detection

3.0.0.34.3 Exhaustiveness Analyzer (exhaustiveness.cpp)

- **NEW** - Pattern coverage analysis for pick statements
- Type domain calculation (TBB ranges, bool values, enum variants)
- Interval arithmetic for gap finding
- ERR sentinel enforcement for TBB types

3.0.0.34.4 Symbol Table (symbol_table.cpp)

- Variable scoping
- Function signatures
- Type definitions
- Module imports

3.0.0.34.5 Module Resolver (module_resolver.cpp)

- Import path resolution
- Dependency graph construction
- Circular dependency detection

3.0.0.35 Backend (src/backend/)

3.0.0.36 IR Generation (ir/)

- **Purpose**: Emit LLVM IR from validated AST
- **Key Files**:
  - `ir_generator.cpp` - Main IR emission
  - `codegen_expr.cpp` - Expression codegen
  - `codegen_stmt.cpp` - Statement codegen
  - `tbb_codegen.cpp` - TBB-specific optimizations
  - `ternary_codegen.cpp` - Ternary operator lowering

3.0.0.37 Code Generation Strategy

1. **Function-at-a-time** - Process each function independently
2. **SSA Form** - Static Single Assignment via LLVM
3. **Type Lowering** - Aria types → LLVM types
4. **Optimization** - Leverage LLVM's optimization passes

3.0.0.38 Runtime (src/runtime/)

3.0.0.39 Memory Management

- **GC** - Garbage collector for default allocations
- **Arena** - Bump allocator for temporary data
- **Pool** - Fixed-size object pools
- **Slab** - Variable-size slab allocator
- **Wild** - Manual memory (opt-out of GC)

3.0.0.40 Streams (streams/)

- Six-stream I/O implementation
- FD 3-5 initialization (stddbg, stddati, stddato)
- Platform-specific stream handling

3.0.0.41 Async Runtime (async/)

- M:N threading scheduler (planned)
- Work-stealing queue (planned)
- Coroutine state management (planned)

3.0.0.42 Dependencies

3.0.0.43 External

- **LLVM 20.1.2** - IR generation and optimization
- **CMake 3.20+** - Build system
- **C++17** - Compiler implementation language

3.0.0.44 Internal (Aria Ecosystem)

- **aria_lang_specs** - Language specification (source of truth)
- **aria_ecosystem** - Architecture documentation
- **AriaBuild** - Build system integration (planned)
- **AriaX** - Package management (planned)

3.0.0.45 Build System

3.0.0.46 CMakeLists.txt Structure

```
#### Phase 1: Lexer
LEX_SOURCES = lexer.cpp, token.cpp

#### Phase 2: Parser
PARSER_SOURCES = parser.cpp

#### Phase 3: Semantic Analysis
SEMA_SOURCES = type_checker.cpp, borrow_checker.cpp, exhaustiveness.cpp, ...

#### Phase 4: IR Generation
IR_SOURCES = ir_generator.cpp, codegen_expr.cpp, codegen_stmt.cpp, ...

#### Phase 5: Runtime
RUNTIME_SOURCES = gc.cpp, streams.cpp, ...
```

```
#### Main Executable
ariac = LEX + PARSER + SEMA + IR + RUNTIME + LLVM
```

3.0.0.47 Build Commands

```
#### Configure
cmake -B build -DCMAKE_BUILD_TYPE=Debug

#### Build compiler
cmake --build build --target ariac

#### Run tests
cmake --build build --target test

#### Install
cmake --build build --target install
```

3.0.0.48 Code Organization

```
aria/
  include/              # Public headers
     frontend/
        ast/         # AST node definitions
        lexer/       # Lexer interface
        parser/      # Parser interface
        sema/        # Semantic analysis interfaces
     backend/
        ir/          # IR generation interface
     runtime/         # Runtime library headers

  src/                 # Implementation files
     frontend/
        lexer/
        parser/
        ast/
        sema/
     backend/
        ir/
     runtime/
        gc/
        streams/
        async/
     main.cpp         # Compiler entry point

  tests/               # Test suite
     unit/           # Unit tests (C++)
     integration/    # Integration tests (Aria code)
     safety/         # Safety feature tests
     runtime/        # Runtime library tests

  aria_ecosystem/      # Documentation (submodule)
     specs/          # Language specifications
```

```
    MASTER_ROADMAP.md

docs/               # Generated documentation
```

3.0.0.49 Data Flow Example

```
func:main = int8() {
    tbb8:result = 42;
    pick(result) {
        (0..127) { stddbg << "Valid\n"; },
        (ERR) { stddbg << "Error\n"; }
    }
    pass(0);
};
```

3.0.0.50 Compilation Pipeline

1. **Lexer Output** (tokens):

   ```
   FUNC, COLON, IDENT(main), EQUAL, IDENT(int8), LPAREN, RPAREN, LBRACE,
   IDENT(tbb8), COLON, IDENT(result), EQUAL, NUMBER(42), SEMICOLON,
   PICK, LPAREN, IDENT(result), RPAREN, LBRACE, ...
   ```

2. **Parser Output** (AST):

   ```
   FuncDeclStmt
       name: "main"
       returnType: int8
       body: BlockStmt
           VarDeclStmt (tbb8:result = 42)
           PickStmt
               selector: IdentifierExpr("result")
               cases:
                   PickCase (0..127)
                   PickCase (ERR)
   ```

3. **Type Checker**:

   - Infers `result` type as `tbb8`
   - Validates pick selector is tbb8
   - Calls ExhaustivenessAnalyzer
   - Verifies both 0..127 and ERR cover full tbb8 domain

4. **Borrow Checker**:

   - Tracks `result` ownership
   - No moves detected (value semantics for tbb8)
   - No lifetime issues

5. **IR Generation** (LLVM IR):

   ```
   define i8 @main() {
   entry:
     %result = alloca i8
     store i8 42, i8* %result
     %0 = load i8, i8* %result
     %1 = icmp eq i8 %0, -128  ; ERR check
     br i1 %1, label %err_case, label %range_case
   ```

```
range_case:
  call void @stddbg_print(i8* "Valid\n")
  br label %exit

err_case:
  call void @stddbg_print(i8* "Error\n")
  br label %exit

exit:
  ret i8 0
}
```

6. **LLVM Optimization**:
   - Dead code elimination
   - Constant propagation (`%result` is always 42)
   - Branch elimination (never takes ERR path)
7. **Code Generation**:
   - Platform-specific assembly
   - Link against Aria runtime
   - Produce executable binary

3.0.0.51 Key Algorithms

3.0.0.52 Exhaustiveness Checking

1. Calculate type domain (min/max values, symbols)
2. Extract coverage from pick cases (ranges, literals, ERR)
3. Sort and merge overlapping intervals
4. Find gaps using sweep algorithm
5. Check ERR coverage if TBB type
6. Generate helpful error message

3.0.0.53 Borrow Checking (Appendage Theory)

1. Assign scope depth to each variable
2. Track moves and pinning references
3. Validate lifetime rules (child cannot outlive parent)
4. Detect use-after-move
5. Verify drop order (children first, parents last)

3.0.0.54 Type Inference

1. Bottom-up traversal of expression tree
2. Literal types from token (42 → int64, 3.14 → double)
3. Operator result types from operands
4. Function return types from declaration
5. Generic instantiation via unification

3.0.0.55 Testing Strategy

3.0.0.56 Unit Tests (C++)

- Test individual components (lexer, parser, etc.)
- Mock dependencies
- Fast feedback (~1 second)

3.0.0.57 Integration Tests (Aria)

- Test actual Aria programs
- Verify end-to-end compilation
- Check runtime behavior

3.0.0.58 Safety Tests

- Test error detection (must-use, exhaustiveness, borrow checking)
- Verify helpful error messages
- Ensure bugs are caught at compile-time

3.0.0.59 Performance Characteristics

Component	Complexity	Bottleneck
Lexer	O(n)	UTF-8 decoding
Parser	O(n)	Memory allocation for AST
Type Checker	$O(n \cdot m)$	Generic instantiation
Borrow Checker	$O(n \cdot d)$	Scope depth tracking
Exhaustiveness	$O(k \cdot \log k)$	Interval sorting (k = cases)
IR Generation	O(n)	LLVM API calls

3.0.0.60 Future Directions

3.0.0.61 v0.1.0 (Released February 2026)

- Complete core language features
- Stable ABI
- Basic standard library
- Documentation generator

3.0.0.62 v0.2.0 (Released March 2026)

- Self-hosting compiler (5 modules, 220 tests)
- Improved error diagnostics with suggestions
- License change to Apache 2.0
- 7 critical codegen bug fixes

3.0.0.63 v0.2.1+ Goals

- Full async/await runtime
- Incremental compilation
- Debugger integration (LLDB)
- Cross-compilation support

3.0.0.64 For Contributors **Read This First**: 1. CONTRIBUTING.md - Rules and expectations 2. TASKS.md - Available work 3. aria_ecosystem/specs/ - Language specifications

Key Questions: - **Where do I start?** Read TASKS.md for beginner-friendly work. - **How do I test?** `cmake --build build && ./build/ariac tests/your_test.aria` - **What's the code style?** Match existing code. We don't bikeshed formatting. - **Can I add a feature?** Only if it's in the specs. Want a new feature? Discuss in aria_ecosystem first.

Common Pitfalls: - Not reading the spec before implementing - Simplifying because "it's easier this way" - Adding dependencies without discussion - Not testing edge cases - Ignoring error handling

---

This document evolves with the project. Last major update: 2024-12-24

3.0.0.65 Aria Type System Design

Date: 2025-02-07
Status: Design Phase - Ready for Implementation

3.0.0.66 Philosophy

"A little functional, a little OOP, a lot of imperative"

3.0.0.67 What We Have

- [DONE] **Functional**: Closures, lambdas, first-class functions
- [DONE] **OOP-lite**: UFCS method syntax (zero-cost)
- [DONE] **Imperative**: Core strength - explicit, predictable control flow

3.0.0.68 What We're Adding

Type system - Organized, encapsulated composable units with zero overhead

NOT OOP - No inheritance, no vtables, no runtime dispatch, no magic

3.0.0.69 Core Principle: Zero Context Switching

Aria's Consistency Law: `type:name = value;` for EVERYTHING

```
// Variables
int32:x = 42;

// Functions
func:add = int32(int32:a, int32:b) { pass(a + b); };

// Lambdas (just functions without names)
int32(int32:x) { pass(x * 2); };  // Can execute immediately with ()

// Structs
struct:Point = { int32:x; int32:y; };

// Types (new!)
Type:Counter = { /* regular Aria syntax inside */ };
```

No special cases. No syntax changes. Just organization.

3.0.0.70 Type Declaration Syntax

3.0.0.71 Complete Example

```
Type:Counter = {
    // Constructor - just a regular Aria function
    func:create = Counter(int32:initial) {
        Counter:c = {
            secret_ptr = allocate_storage(),
            count = initial
        };
        pass(c);
    };

    // Destructor - just a regular Aria function
    func:destroy = void(Counter:self) {
        free(self.secret_ptr);
    };
```

```
    // Methods - just regular Aria functions with self parameter
    func:increment = void(Counter:self) {
        self.count = self.count + 1;
    };

    func:get_value = int32(Counter:self) {
        pass(self.count);
    };

    // Private members - only accessible within Type methods
    struct:internal = {
        int32->:secret_ptr;
        int64:internal_counter;
    };

    // Public members - accessible everywhere
    struct:interface = {
        int32:count;
        int8:flags;
    };

    // Static/type-level members
    struct:type = {
        int32:MAX_VALUE = 1000;
        int32:MIN_VALUE = 0;
    };
};
```

3.0.0.72 Usage

```
// Instantiate - desugars to Counter_create(0)
Counter:myCounter = instance<Counter>(0);

// Method calls - UFCS desugars to Counter_increment(myCounter)
myCounter.increment();
int32:val = myCounter.get_value();

// Access public fields
myCounter.count = 5;

// Access static members
int32:max = Counter.MAX_VALUE;

// Cleanup - desugars to Counter_destroy(myCounter)
myCounter.destroy();
```

3.0.0.73 Semantic Transformation (Compile-Time Only!)

3.0.0.74 Input: Type Declaration

```
Type:Counter = {
    func:create = Counter(int32:initial) { /* ... */ };
    func:destroy = void(Counter:self) { /* ... */ };
    func:increment = void(Counter:self) { /* ... */ };
```

```
    struct:internal = { int32->:secret_ptr; };
    struct:interface = { int32:count; };
    struct:type = { int32:MAX_VALUE = 1000; };
};
```

3.0.0.75 Output: Desugared Code

```
// Step 1: Merge internal + interface into actual struct
struct:Counter = {
    // From internal (private, but no runtime enforcement yet)
    int32->:secret_ptr;

    // From interface (public)
    int32:count;
};

// Step 2: Prefix all functions with TypeName_
pub func:Counter_create = Counter(int32:initial) {
    // Original body unchanged
};

pub func:Counter_destroy = void(Counter:self) {
    // Original body unchanged
};

pub func:Counter_increment = void(Counter:self) {
    // Can access both internal and interface fields
    // (Access control is semantic analysis concern)
};

// Step 3: Static members become module-level constants
pub func:Counter_MAX_VALUE = int32() { pass(1000); };

// Step 4: instance<Counter>(0) desugars to Counter_create(0)
Counter:myCounter = Counter_create(0);

// Step 5: myCounter.increment() uses existing UFCS
// Already implemented in codegen_expr.cpp:2727-2800!
```

3.0.0.76 Implementation Phases

3.0.0.77 Phase 1: Parser (Syntax Recognition) **File**: `src/frontend/parser/parser.cpp`

Add case for `Type:` declarations:

```
if (current_token.type == TOKEN_TYPE && peek().type == TOKEN_COLON) {
    return parseTypeDecl();
}
```

New AST Node:

```
class TypeDeclStmt : public ASTNode {
public:
    std::string typeName;

    // Required functions
```

```
    ASTNodePtr createFunc;   // Constructor
    ASTNodePtr destroyFunc;  // Destructor (optional)

    // Struct definitions
    ASTNodePtr internal;     // struct:internal
    ASTNodePtr interface;    // struct:interface
    ASTNodePtr typeMembers;  // struct:type

    // Additional methods
    std::vector<ASTNodePtr> methods;
};
```

3.0.0.78 Phase 2: Semantic Analysis (Desugaring)

File: `src/frontend/sema/type_checker.cpp`

```
void TypeChecker::checkTypeDecl(TypeDeclStmt* stmt) {
    // 1. Validate required components
    if (!stmt->createFunc) {
        addError("Type '" + stmt->typeName +
                 "' missing required 'create' function");
    }

    // 2. Merge internal + interface into combined struct
    auto combinedStruct = mergeStructs(stmt->internal, stmt->interface);

    // 3. Register combined struct as actual type
    typeSystem->registerStruct(stmt->typeName, combinedStruct);

    // 4. Validate all methods
    for (auto& method : stmt->methods) {
        validateMethodSignature(method, stmt->typeName);
    }

    // 5. Export public symbols (interface + methods)
    exportTypeSymbols(stmt);
}
```

3.0.0.79 Phase 3: Code Generation (No Changes Needed!)

Existing code handles it all: - Struct codegen already works [DONE] - Function codegen already works [DONE]
- UFCS already implemented [DONE] - Just need to emit desugared AST nodes

3.0.0.80 Phase 4: Access Control (Future Enhancement)

Optional: Enforce `internal` vs `interface` at semantic analysis

```
void TypeChecker::validateMemberAccess(MemberAccessExpr* expr) {
    // Check if accessing internal member from outside Type methods
    if (isInternalMember(expr->member) && !isWithinTypeMethod(currentFunction)) {
        addError("Cannot access internal member '" + expr->member +
                 "' outside of Type methods");
    }
}
```

3.0.0.81 Naming Strategy: OOP Deflection

3.0.0.82 Terminology Mapping

Traditional OOP	Aria Type System	Rationale
`class`	`Type:`	Different word = different thing
`constructor`	`func:create`	It's just a function you write
`destructor`	`func:destroy`	Explicit, not magic
`new Class()`	`instance<Type>()`	Shows it's calling `create`
`private`	`internal`	Implementation internals
`public`	`interface`	What you expose
`static`	`type`	Type-level, not instance-level
`this`	`self`	More functional-style
`method`	UFCS function	It's a function with prefix!

3.0.0.83 Defense Mechanisms

Critic: "This is just OOP with different names!"

Response: "Not at all. Look at the generated code - it's structs and functions. No inheritance, no vtables, no dynamic dispatch. Just organized composition with zero overhead. If you want OOP, use C++ or Java. Aria gives you the ergonomics without the cost."

Critic: "Why not use `class` keyword?"

Response: "Because it's not a class. It's a Type declaration that organizes related functions and data. The word matters - it sets expectations correctly."

Critic: "You're missing inheritance!"

Response: "We left that out on purpose :-) Use composition. Put one Type as a field in another. Studies show composition is more maintainable anyway. Favor composition over inheritance, right?"

Critic: "This is incomplete OOP!"

Response: "Well of course it isn't. Who told you it was? We explicitly avoided OOP complexity while keeping the useful organizational patterns. Zero-cost abstractions over convenient abstractions."

3.0.0.84 Example: Type-Based Atomic Counter

```
// stdlib/concurrent/atomic_counter.aria

Type:AtomicCounter = {
    // Constructor
    func:create = AtomicCounter(int32:initial) {
        wild int8->:mem = malloc(4);
        int32->:ptr = mem;
        <- ptr = initial;
        AtomicCounter:ac = { storage = ptr, last_value = initial };
        pass(ac);
    };

    // Destructor
    func:destroy = void(AtomicCounter:self) {
        free(self.storage);
    };

    // Methods
    func:load = int32(AtomicCounter:self, int32:order) {
        int32:val = __atomic_load_4(self.storage, order);
        pass(val);
    };
```

```
    func:store = void(AtomicCounter:self, int32:val, int32:order) {
        __atomic_store_4(self.storage, val, order);
    };

    func:increment = int32(AtomicCounter:self) {
        int32:old = __atomic_fetch_add_4(self.storage, 1, 5);
        pass(old);
    };

    // Private - implementation detail
    struct:internal = {
        int32->:storage;
    };

    // Public - visible state
    struct:interface = {
        int32:last_value;
    };

    // Static members
    struct:type = {
        int32:RELAXED = 0;
        int32:ACQUIRE = 2;
        int32:RELEASE = 3;
        int32:SEQ_CST = 5;
    };
};

// Usage
use "stdlib/concurrent/atomic_counter.aria";

func:main = int32() {
    AtomicCounter:counter = instance<AtomicCounter>(0);

    // Method syntax (UFCS)
    counter.store(42, AtomicCounter.SEQ_CST);
    int32:prev = counter.increment();
    int32:current = counter.load(AtomicCounter.RELAXED);

    // Access public field
    counter.last_value = current;

    // Cleanup
    counter.destroy();

    pass(current);
};
```

3.0.0.85 Composition Over Inheritance

```
// No inheritance needed - just compose!

Type:Logger = {
```

```
    func:create = Logger() { /* ... */ };
    func:log = void(Logger:self, int8->:msg) { /* ... */ };
    struct:interface = { int8->:name; };
};

Type:Database = {
    func:create = Database(Logger:logger) {
        Database:db = { logger = logger };
        pass(db);
    };

    func:query = void(Database:self, int8->:sql) {
        self.logger.log("Executing query");  // Delegate!
        // ... execute query
    };

    struct:interface = {
        Logger:logger;  // Composition!
    };
};

// Usage
func:main = void() {
    Logger:log = instance<Logger>();
    Database:db = instance<Database>(log);

    db.query("SELECT * FROM users");

    db.destroy();
    log.destroy();
};
```

3.0.0.86 Advantages Over Traditional OOP

3.0.0.87 Zero-Cost Abstractions [DONE]

```
// Your Aria code:
counter.increment();

// Compiles to (LLVM IR):
call @AtomicCounter_increment(%struct.AtomicCounter %counter)

// NOT (OOP with vtable):
%vtable = load ptr, ptr %counter.vtable
%fn_ptr = getelementptr %vtable, i64 3
%fn = load ptr, ptr %fn_ptr
call void %fn(ptr %counter)  // Indirect call!
```

3.0.0.88 Predictable Memory Layout [DONE]

```
Type:Point = {
    struct:interface = { int32:x; int32:y; };
};

// Memory layout (exactly):
```

```
// [x: 4 bytes][y: 4 bytes]
// Total: 8 bytes (no vtable pointer!)
```

3.0.0.89 FFI Compatible [DONE]

```c
// C code can use Aria Types directly
struct Point {
    int32_t x;
    int32_t y;
};

void use_point(struct Point* p) {
    // Works perfectly!
}
```

3.0.0.90 Compile-Time Resolution [DONE]

- All method calls resolved at compile time
- No runtime type checking
- LLVM can inline everything
- Cache-friendly (no pointer chasing)

3.0.0.91 Current Implementation Status

3.0.0.92 [DONE] Already Working

- Struct definitions
- Function definitions
- UFCS method syntax
- Module exports
- Public visibility

3.0.0.93 [WIP] Need to Implement

- `Type:` parser
- Type AST node
- Desugaring logic
- `instance<T>()` syntax
- Static member access (`Type.MEMBER`)

3.0.0.94 [NOTE] Future Enhancements

- Access control enforcement (`internal` checking)
- Better error messages for Type usage
- Type-aware editor completion
- Automatic `destroy()` calls (RAII)

3.0.0.95 Next Steps

1. **Draft parser changes** - Add Type: recognition
2. **Create AST node** - TypeDeclStmt structure
3. **Implement desugaring** - Transform to structs + functions
4. **Test with atomic example** - Verify zero-cost
5. **Document patterns** - Stdlib development guide

Motto: "Composition over inheritance, explicitness over magic, zero-cost over convenience"

Tagline: "It's not OOP, it's organized imperative programming :-)"

3.0.0.96 Aria Function Specification

3.0.0.97 Declaration Syntax

All function declarations require a trailing semicolon.

```
func:nameOfFunc = <ReturnType>(<ArgType>:argName){
    // Always returns Result<ReturnType> implicitly - main and failsafe are exceptions
    // Can return Result object, Result literal, or use pass/fail sugar functions
    // exit() is NOT valid from normal functions
};
```

3.0.0.98 Examples: pass/fail/return usage

```
func:thisFails = uint8(int32:arg) {
    tbb8:err = 1;
    // stuff happens
    fail(err); // sugar for: return Result{error:err, value:NIL, is_error:true};
};

func:thisPasses = uint8(int32:arg) {
    uint8:retVal = 33;
    // stuff happens
    pass(retVal); // sugar for: return Result{error:NIL, value:retVal, is_error:false};
};

func:longForm = flt32() {
    flt32:retVal = 1.2;
    // do stuff
    return Result{error:NIL, value:retVal, is_error:false};
};
```

3.0.0.99 main and failsafe — Both MANDATORY to compile an executable

3.0.0.100 Examples: exit() usage

```
func:main = int32(int32:argc, int8[]->:argv) {
    // pass()/fail() are INVALID in main
    // exit() call required; normal control flow semantics apply otherwise
    exit(0);
};

func:main = int32() {
    // zero-argument form also valid
    exit(0);
};

func:failsafe = int32(tbb32:err) {
    // Handle graceful shutdown
    // pass()/fail() are INVALID in failsafe
    // exit() call required; err code must be > 0
    exit(1);
};
```

Note: `int8[]->` is Aria's equivalent to C's `char**` for argv compatibility.

3.0.0.101 Exit Code Conventions

- Traditional conventions apply: 0 = no error, > 0 = error

3.0.0.102 FAILSAFE ERROR CODES (for `err` in `failsafe(tbb32:err)`)

The failsafe parameter is tbb32, giving range [-2147483647, +2147483647] with ERR sentinel.

3.0.0.103 Special Values

Code	Meaning
ERR	Unknown/unclassifiable error (tbb32 sentinel, min value)
0	No error / testing / forced failsafe with no fault

3.0.0.104 System Error Codes (negative range: -1 to -2147483647)

Reserved for runtime, OS, and hardware-level errors. These are set by the Aria runtime or OS signal handlers — user code should not emit these.

Code	Name	Meaning
-1	SYS_GENERAL	General system error (unspecified)
-2	SYS_OUT_OF_MEMORY	Memory allocation failed
-3	SYS_STACK_OVERFLOW	Stack overflow detected
-4	SYS_SEGFAULT	Segmentation fault / access violation
-5	SYS_ABORT	Abort signal (SIGABRT)
-6	SYS_FPE	Floating-point exception (SIGFPE)
-7	SYS_BUS_ERROR	Bus error (SIGBUS)
-8	SYS_ILLEGAL_INSN	Illegal instruction (SIGILL)
-9	SYS_PIPE	Broken pipe (SIGPIPE)
-10	SYS_ALARM	Alarm/timer expired (SIGALRM)
-11	SYS_TERMINATED	Process terminated by signal (SIGTERM)
-12	SYS_IO_ERROR	I/O error (disk, device)
-13	SYS_PERMISSION	Permission denied
-14	SYS_DEADLOCK	Deadlock detected
-15	SYS_THREAD_PANIC	Thread panic / unrecoverable thread error
-16	SYS_RESOURCE_LIMIT	System resource limit reached (file descriptors, etc.)
-17 to -99	(reserved)	Reserved for future Aria runtime system codes
-100 to -999	(reserved)	Reserved for OS/platform-specific system codes
-1000 to -2147483647	(unassigned)	Available for future system-level extensions

3.0.0.105 User Error Codes (positive range: 1 to 2147483647)

For application-level errors triggered by `!!! code` (failsafe invocation).

Code	Name	Meaning
1	USR_GENERAL	General application error (unspecified)
2	USR_INVALID_CONFIG	Configuration invalid or missing
3	USR_INIT_FAILED	Initialization/startup failed
4	USR_DATA_CORRUPT	Data corruption detected
5	USR_DEPENDENCY	Required dependency unavailable

Code	Name	Meaning
6	USR_NETWORK	Network error (connection lost, timeout)
7	USR_AUTH	Authentication/authorization failure
8	USR_HARDWARE	Hardware device error (sensor, GPU, etc.)
9	USR_TIMEOUT	Critical operation timed out
10	USR_ASSERTION	Assertion / invariant violation
11 to 49	(reserved)	Reserved for future Aria standard user codes
50 to 99	(reserved)	Reserved for framework/library standard codes
100+	(user-defined)	Application-specific error codes

3.0.0.106 Usage Notes

- System codes are **negative**, user codes are **positive** — easy to distinguish in handlers
- Runtime sets system codes automatically on signals; user code calls `!!! code` with positive values
- ERR (tbb sentinel) means "we don't know what happened" — always handle it
- Code 0 should never reach failsafe in normal operation, but handle it gracefully
- Libraries should document their error codes in the 100+ range and avoid collisions

3.0.0.107 User Stack Builtins (v0.4.3) Per-scope implicit LIFO scratch pad for type-safe intra-function value passing. Separate from the hardware call stack and the `stack` memory allocation keyword.

3.0.0.108 `astack(capacity?)` Initialize one implicit stack per function scope. - `capacity`: optional `int64`, default 256 slots if omitted - No handle returned — the stack is implicit to the current scope - Must be called before `apush`/`apop`/`apeek` in the same scope

```
astack(64i64);   // 64-slot stack
astack();        // 256-slot default
```

3.0.0.109 `apush(value)` Push a typed value onto the implicit stack. - Accepts: int8, int16, int32, int64, flt32, flt64, bool, string, pointer - Type tag stored automatically (no manual tag management) - Fatal `exit(1)` with diagnostic to stderr on overflow - No return value

```
apush(42i64);
apush(3.14f64);
apush(true);
```

3.0.0.110 `apop()` Pop the top value. Destination type inferred from assignment context. - Zero arguments - Runtime type-tag checking: fatal `exit(1)` on mismatch or underflow - Returns value directly (not wrapped in `Result<T>`)

```
int64:n = apop();    // pops as int64
flt64:f = apop();    // pops as flt64
bool:b = apop();     // pops as bool
```

3.0.0.111 `apeek()` Non-destructive read of the top value. - Same context-typed semantics as `apop()` - Value remains on stack

```
int64:top = apeek();  // reads top without removing
```

3.0.0.112 Error Model Fatal errors (not Result): - Stack overflow on push → stderr diagnostic + `exit(1)` - Stack underflow on pop/peek → stderr diagnostic + `exit(1)` - Type mismatch on pop/peek → stderr diagnostic + `exit(1)`

3.0.0.113 Auto-Cleanup Stack is automatically destroyed on function return (explicit return or fallthrough). No manual cleanup needed.

3.0.0.114 Aria Reserved Words — Complete List

Generated from `src/frontend/lexer/lexer.cpp` keyword table (151 keywords) v0.16.11 — Authoritative reference

All words listed below are reserved by the Aria compiler and **cannot be used as variable names, function names, or identifiers** (except `stack` and `gc` which are contextual — see notes).

3.0.0.115 Mandatory Functions

`func`	`main`	`failsafe`

3.0.0.116 Control Flow

`if`	`else`	`while`	`for`	`loop`	`till`
`when`	`then`	`end`	`pick`	`fall`	`break`
`continue`	`return`	`pass`	`fail`	`exit`	

3.0.0.117 Result & Error Handling

`ok`	`drop`	`raw`	`defaults`	`Result`	`ERR`
`NIL`	`NULL`	`unknown`			

3.0.0.118 Memory Management

`wild`	`wildx`	`stack*`	`gc*`	`move`	`defer`

`*` `stack` and `gc` are **contextual** — valid as variable names outside allocation contexts.

3.0.0.119 Module System

`use`	`mod`	`pub`	`extern`	`in`	`as`	`cfg`

3.0.0.120 Type Declarations

`struct`	`enum`	`trait`	`impl`	`derive`	`opaque`
`Type`	`Rules`	`func`	`macro`		

3.0.0.121 Type Modifiers

`const`	`fixed`	`limit`	`dyn`	`obj`	`any`

3.0.0.122 Async & Concurrency

`async`	`await`	`catch`

3.0.0.123 Memory Ordering

`relaxed`	`acquire`	`release`	`acq_rel`	`seq_cst`

3.0.0.124 Borrow & Safety

`is`	`requires`	`ensures`	`invariant`	`result`

3.0.0.125 Compile-Time

`comptime`	`inline`	`noinline`	`prove`	`assert_static`

3.0.0.126 Integer Types

```
int1      int2      int4      int8      int16     int32
int64     int128    int256    int512    int1024   int2048   int4096
uint1     uint2     uint4     uint8     uint16    uint32
uint64    uint128   uint256   uint512   uint1024  uint2048  uint4096
```

3.0.0.127 Other Numeric Types

```
tbb8      tbb16     tbb32     tbb64
flt32     flt64     flt128    flt256    flt512
tfp32     tfp64
fix256
frac8     frac16    frac32    frac64
```

3.0.0.128 Boolean & String

```
bool      true      false     string
```

3.0.0.129 Ternary & Nonary

```
trit      tryte     nit       nyte
```

3.0.0.130 Vector, Matrix & Tensor

```
vec2      vec3      vec9
matrix    tmatrix
tensor    ttensor
```

3.0.0.131 Collection & I/O Types

```
array     binary    buffer    stream    pipe      process
```

3.0.0.132 Debug & Special

```
debug     apop      apush     apeek     astack    acap
asize     afits     atype     ahash     ahset     ahget
ahcount   ahsize    ahfits    ahtype
```

3.0.0.133 Not Reserved (Common Misconceptions)

Word	Status	Notes
`sync`	**Not a keyword**	Only `async`/`await` exist. Use atomics for sync patterns.
`atomic`	**Not a keyword**	`atomic<T>` is a generic type, not a reserved word.
`complex`	**Not a keyword**	`complex<T>` is a generic type, not a reserved word.
`simd`	**Not a keyword**	`simd<T,N>` is a generic type, not a reserved word.
`Handle`	**Not a keyword**	`Handle<T>` is a generic type, not a reserved word.
`void`	**Not a keyword**	Used in `extern` blocks only as a type annotation.

Word	Status	Notes
`free`	**Not a keyword**	Use `_release` or `_destroy` — `free` is shadowed.

3.0.0.134 Total Count: 151 reserved keywords

3.0.0.135 main / failsafe / exit() — Canonical Reference

3.0.0.136 This is the source of truth for how the two program endpoints work.

3.0.0.137 Design Intent Every Aria executable has exactly two code-path endpoints:

main — normal program entry and exit failsafe — abnormal/error exit (runtime errors, !!!, constraint violations)

Both terminate the process via `exit(code)`. Neither uses `pass()/fail()`. pass/fail are the Result return system for regular functions — they have no business in program endpoints.

3.0.0.138 Signatures

```
// main: normal entry point
// - returns int32 (C runtime compatibility)
// - no parameters (for now)
// - MUST contain at least one exit() call
func:main = int32() {
    // ... program logic ...
    exit(0);
};

// failsafe: error endpoint
// - returns int32 (C runtime compatibility)
// - takes tbb32:err - balanced ternary error code
//   (uses tbb32 so the ERR signal can represent "unknown error" from runtime)
// - MUST contain at least one exit(val) call where val > 0
func:failsafe = int32(tbb32:err) {
    // ... graceful shutdown / cleanup ...
    exit(1);
};
```

3.0.0.139 Why tbb32 for failsafe's error code?

- The runtime may invoke failsafe for errors that don't have a numeric code
- tbb32 has three states: positive integers, negative integers, and ERR
- ERR represents "unknown/unclassifiable error" — a state int32 cannot express
- This keeps the safety system coherent all the way to the final exit call
- User-invoked `!!! code` can pass a known tbb32 value
- Runtime-invoked failsafe passes ERR when the error category is not known

3.0.0.140 exit() rules

- Only valid inside `main` and `failsafe`
- Takes exactly one int32 argument (the process exit code)
- In main: any value is valid (`exit(0)` for success, `exit(1)` for failure, etc.)
- In failsafe: value MUST be > 0 (failsafe is an error path — exit(0) is wrong)
- exit() terminates the process immediately — it is noreturn
- At least one exit() call must be reachable in main and failsafe

3.0.0.141 What NOT to use in main/failsafe

- `pass(value)` — this is for regular functions, creates Result
- `fail(code)` — this is for regular functions, creates error Result
- `return` — Aria doesn't use this keyword

3.0.0.142 Invocation patterns

```
// Normal program exit
func:main = int32() {
    // ... do work ...
    exit(0);    // success
};

// Error exit from main
func:main = int32() {
    if (bad_condition) {
        exit(1);    // error
    }
    exit(0);
};

// Failsafe with cleanup
func:failsafe = int32(tbb32:err) {
    // Log error, close resources, etc.
    exit(1);
};

// Failsafe with error-code-dependent exit
func:failsafe = int32(tbb32:err) {
    int32:ret = 1;
    // ... handle different error categories ...
    ret = 32;
    exit(ret);
};

// Runtime triggers failsafe via !!! operator
func:some_function = int32() {
    if (critical_failure) {
        !!! 42;    // desugars to failsafe(42) → calls failsafe with tbb32 val
    }
    pass(0i32);
};
```

3.0.0.143 Compiler enforcement summary

1. main MUST exist (unless -c library mode)
2. main MUST return int32
3. main MUST take no parameters
4. main MUST contain at least one exit() call
5. failsafe MUST exist (unless -c library mode)
6. failsafe MUST return int32
7. failsafe MUST take exactly one parameter of type tbb32
8. failsafe MUST contain at least one exit() call with value > 0
9. exit() is only valid inside main and failsafe
10. exit() takes exactly one int32 argument

3.0.0.144 User Stack Reference

Feature: Per-Scope Implicit User Stack
Version: v0.4.3 (March 2026)
Status: Implemented, 22/22 tests passing

3.0.0.145 Summary

The user stack is a compiler-managed typed LIFO scratch pad for intra-function value passing. It provides type-safe push/pop operations with automatic type tagging, context-typed pop, and scope-based auto-cleanup.

3.0.0.146 API

3.0.0.147 `astack(capacity?)`

Property	Value
Arguments	0 or 1 (`int64` capacity, default 256)
Returns	Nothing meaningful (implicit handle)
Side Effects	Allocates mmap-backed storage
Errors	None (allocation failure is fatal)
Scope	One per function scope

3.0.0.148 `apush(value)`

Property	Value
Arguments	1 (any supported type)
Returns	Nothing
Side Effects	Stores value + type tag at stack top
Errors	Fatal `exit(1)` on overflow
Supported Types	int8, int16, int32, int64, flt32, flt64, bool, string, pointer

3.0.0.149 `apop()`

Property	Value
Arguments	0
Returns	Value of destination type (context-inferred)
Side Effects	Removes top element
Errors	Fatal `exit(1)` on underflow or type mismatch
Type Inference	From assignment target: `flt32:f = apop()` → pop as flt32

3.0.0.150 `apeek()`

Property	Value
Arguments	0

Property	Value
Returns	Value of destination type (context-inferred)
Side Effects	None (non-destructive)
Errors	Fatal `exit(1)` on underflow or type mismatch
Type Inference	Same as `apop()`

3.0.0.151 Type Tag Scheme

Tag	Type	Storage Method
0	int8	Zero-extended to int64
1	int16	Zero-extended to int64
2	int32	Zero-extended to int64
3	int64	Direct
4	flt32	Bitcast float → int32, zero-extended to int64
5	flt64	Bitcast double → int64
6	bool	Zero-extended to int64
7	string	Pointer stored as int64
8	pointer	Pointer stored as int64

3.0.0.152 Compiler Implementation

3.0.0.153 Files Modified (from v0.4.2 → v0.4.3)

File	Changes
`include/runtime/ustack.h`	Push changed to void return, removed error codes, added default capacity
`src/runtime/collections/ustack.cpp`	Push returns void with fatal exit on overflow
`include/backend/ir/codegen_expr.h`	Added `ustack_pop_dest_type` context variable
`include/backend/ir/ir_generator.h`	Added `ustack_pop_dest_type` for propagation
`src/frontend/sema/type_checker.cpp`	New signatures: astack(0-1 args), apush(1), apop/apeek(0 args, UnknownType return)
`src/backend/ir/codegen_expr.cpp`	Per-scope implicit stack via hidden `__aria_ustack_handle` alloca; type-aware pop/peek with bitcast chain
`src/backend/ir/codegen_stmt.cpp`	VarDecl sets dest type before initializer; auto-cleanup on return
`src/backend/ir/ir_generator.cpp`	VarDecl sets dest type; CALL dispatch propagates to fresh ExprCodegen

3.0.0.154 Key Implementation Detail The `ustack_pop_dest_type` context variable is the critical mechanism. When the compiler processes `int64:x = apop()`, the VarDecl handler sets `ustack_pop_dest_type = "int64"` before generating the initializer expression. The `apop()` codegen reads this context to know which type tag to validate and how to bitcast the returned value.

This required a fix in v0.4.3: `IRGenerator` creates a **fresh** `ExprCodegen` for each CALL node, so the context must be set on the `IRGenerator` and propagated to `ExprCodegen` in the CALL dispatch path.

3.0.0.155 Error Diagnostics All errors print to stderr with the format:

```
[ustack] <error description>
```

Then call `exit(1)`.

Example messages: - `[ustack] overflow: stack full (256/256)` - `[ustack] underflow: stack empty` - `[ustack] type mismatch: expected tag 3 (int64), got tag 5 (flt64)`

3.0.0.156 Tests

Test File	Count	Coverage
`tests/test_ustack.aria`	14	LIFO ordering, peek, multi-push, typed pop (int8/int32/int64/flt32/flt64), mixed types
`tests/test_ustack_strings.aria`	8	Scope independence across function calls, mixed int/float types

---

3.0.0.157 Breaking Changes from v0.4.2

v0.4.2 (Handle-Based)	v0.4.3 (Implicit)
`int64:h = astack(256i64)`	`astack(256i64)` or `astack()`
`apush(h, value)`	`apush(value)`
`apop(h)`	`apop()` — type from context
`apeek(h)`	`apeek()` — type from context
Push returns error code	Push is void (fatal on overflow)

3.0.0.158 Design Rationale The user stack fills a specific niche in Aria's "safe by default, opt-in to danger" philosophy:

1. **Why not just use variables?** Variables are faster but require knowing the shape at compile time. The user stack supports dynamic depth and mixed types.
2. **Why not Result?** Stack misuse (overflow, underflow, type mismatch) is a logic error, not a runtime condition. Fatal errors prevent silent corruption.
3. **Why implicit handles?** Explicit handles add ceremony without benefit when there's only one stack per scope. The compiler manages the handle internally.
4. **Why context-typed pop?** Requiring `apop::<int64>()` turbofish syntax would work but is verbose. Context typing from the assignment target is natural and catches mismatches at runtime.

3.0.0.159 See Also

- Spec — Line 6109: USER STACK section
- Guide — Tutorial with examples
- Memory Model — Memory perspective

3.0.0.160 Macro & Derive Authoring Guide

v0.8.4 — Aria Compiler Documentation

3.0.0.161 Overview Aria's macro system provides two mechanisms for code generation:

1. **Derive macros** — Auto-generate trait implementations for structs
2. **Attribute macros** — Modify or annotate declarations at compile time

3.0.0.162 Derive Macros

3.0.0.163 Built-in Derives Apply derive attributes to structs to auto-generate trait implementations:

```
#[derive(Eq, Ord, Clone, ToString, Debug, Hash)]
struct:Point = {
    int32:x;
    int32:y;
};
```

3.0.0.164 Available Traits

Trait	Method	Signature	Description
`ToString`	`to_string`	`string(Self:self)`	String representation
`Debug`	`debug`	`string(Self:self)`	Debug representation with types
`Eq`	`eq`	`bool(Self:self, Self:other)`	Field-by-field equality
`Ord`	`less_than`	`bool(Self:self, Self:other)`	Lexicographic less-than
`Hash`	`hash`	`uint64(Self:self)`	FNV-1a hash
`Clone`	`clone`	`Self(Self:self)`	Shallow copy

3.0.0.165 How Derive Works

1. The type checker expands `#[derive(Trait)]` into a synthetic `impl:Trait:for:StructName`
2. The generated impl contains methods that operate field-by-field
3. Synthetic impls are injected into the AST before IR generation
4. IR codegen generates the mangled function (e.g., `Point_less_than`)

3.0.0.166 Using Derived Methods Derived methods are called via UFCS (Uniform Function Call Syntax):

```
Point:a = Point{ x: 1, y: 2 };
Point:b = Point{ x: 3, y: 4 };

bool:is_equal = raw a.eq(b);
bool:is_less = raw a.less_than(b);
string:repr = raw a.to_string();
```

Note: Derived methods return `Result<T>`, so use `raw` to unwrap.

3.0.0.167 Attribute Macros

3.0.0.168 Available Attributes

Attribute	Target	Description
`#[inline]`	Functions	Hint to inline the function
`#[noinline]`	Functions	Prevent inlining
`#[align(N)]`	Structs/Vars	Set alignment in bytes
`#[derive(...)]`	Structs	Generate trait impls
`#[gpu_kernel]`	Functions	Mark as GPU kernel
`#[gpu_device]`	Functions	Mark as GPU device function
`#[comptime]`	Functions	Evaluate at compile time

3.0.0.169 SMT Solver Reference — Aria Compiler Working document. Will become a best practices guide later.

3.0.0.170 Architecture Overview Z3 is statically linked into `ariac`. Users never interact with it directly. The compiler uses it in two modes:

1. **Verification mode** (`--verify`) — prove constraints, contracts, overflow safety
2. **Optimization mode** (`--smt-opt`) — prove properties → emit faster codegen

3.0.0.171 Pipeline Location All Z3 work happens in **Phase 3.25** of compilation, between type checking and borrow checking, before IR generation.

```
Parse → Type Check → Z3 (3.25) →
  Borrow Check → Result Elision (3.75) →
  IR Gen → LLVM → Link
```

3.0.0.172 Files

File	LOC	Role
`include/analysis/z3_verifier.h`	208	Class definition, verification API
`src/analysis/z3_verifier.cpp`	1353	Core verification logic, Z3 queries
`src/main.cpp` (Phase 3.25)	~750	Orchestration, AST walks, optimization discovery
`src/backend/ir/ir_generator.cpp`	~20	Fast-mode flag activation per function
`src/backend/ir/codegen_expr.cpp`	~140	Fast-path code emission for ustack/uhash

3.0.0.173 Compiler Flags

Flag	Implies	Effect
`--verify`	—	Enable Z3 verification (Rules consistency, limit checks)
`--verify-contracts`	`--verify`	Verify requires/ensures function contracts
`--verify-overflow`	`--verify`	Verify integer arithmetic overflow safety
`--verify-report`	`--verify`	Emit detailed proof results (proven/disproven/unknown)
`--smt-opt`	`--verify`	Enable SMT-guided optimizations (fast paths)

Defined in `CompilerOptions` struct at main.cpp:132-137. Parsed at main.cpp:328-341.

3.0.0.174 Verification Passes (current as of v0.4.7)

3.0.0.175 1. Rules Consistency Check

- Verifies each `Rules` declaration is not self-contradictory
- Creates Z3 bitvector/real variable, asserts all conditions, checks SAT
- If UNSAT → error: rules can never be satisfied

3.0.0.176 2. Literal Limit Verification

- Walks AST for `limit<RulesName>` on variable declarations
- For integer/float literal initializers, proves value satisfies all conditions
- Uses `verifyLimitInt()` / `verifyLimitFloat()` with concrete values

3.0.0.177 3. Function Contract Verification (`--verify-contracts`)

- Walks functions with `requires`/`ensures` clauses
- Currently Phase 1 only: checks requires and ensures are individually satisfiable
- **NOT YET IMPLEMENTED: Phase 2** — proving ensures from requires + body

3.0.0.178 4. Overflow Verification (`--verify-overflow`)

- Walks binary ops (+, -, *) looking for literal-literal arithmetic
- Uses Z3 bitvector overflow builtins (`Z3_mk_bvadd_no_overflow`, etc.)
- **Limitation:** Only checks when both operands are known constants

3.0.0.179 5. SMT Optimization Discovery (`--smt-opt`)

- **User Stack (astack):** Collects type tags from all `apush()` calls in a function. If all tags are identical, Z3 proves homogeneity → function added to `ustack_opt_funcs`. Codegen emits `aria_ustack_push_fast` / `pop_fast` (no runtime type check).
- **User Hash (ahash):** Same pattern for `ahset()` value tags → `uhash_opt_funcs`. Codegen emits `aria_uhash_set_fast` / `get_fast`.

3.0.0.180 Z3 API Usage Patterns

3.0.0.181 Core Pattern (Negation-Based Proof)

```
1. Create Z3 sort (bitvector for ints, real for floats)
2. Translate Aria AST condition → Z3_ast
3. Assert NEGATION of the property
4. solver.check()
5. L_FALSE (UNSAT) → property PROVEN for all inputs
   L_TRUE (SAT) → DISPROVEN, extract counterexample from model
   L_UNKNOWN → timeout or too complex
```

3.0.0.182 Z3 Context

- Single `Z3_context` lives for entire compilation
- Per-query timeout: **5 seconds** (set in Z3Verifier constructor)
- Fresh solver created per query with push/pop scope

3.0.0.183 Type Mapping

Aria Type	Z3 Sort	Tag
int8	BitVec(8)	0

Aria Type	Z3 Sort	Tag
int16	BitVec(16)	1
int32	BitVec(32)	2
int64/int	BitVec(64)	3
flt32	Real	4
flt64/flt	Real	5
bool	—	6
string	—	7

3.0.0.184 Data Flow: Verifier → Codegen

```
main.cpp Phase 3.25:
  AST walk → collect push/set type tags per function
  Z3 proves homogeneity
  → std::set<std::string> ustack_opt_funcs
  → std::set<std::string> uhash_opt_funcs

main.cpp (post-verification):
  ir_gen.setUStackOptimizedFuncs(ustack_opt_funcs)
  ir_gen.setUHashOptimizedFuncs(uhash_opt_funcs)

ir_generator.cpp (FUNC_DECL entry):
  ustack_fast_mode = ustack_optimized_funcs.count(funcName) > 0
  uhash_fast_mode = uhash_optimized_funcs.count(funcName) > 0

codegen_expr.cpp (call emission):
  if (ustack_fast_mode) emit _fast variant (no type tag arg)
  else emit standard variant (with type tag arg)
```

3.0.0.185 Known Gaps & Limitations (v0.4.7)

3.0.0.186 Contract Verification

- Only Phase 1 (sanity check) — no body analysis
- Cannot prove ensures from requires + function body
- Needs symbolic execution or weakest precondition calculus

3.0.0.187 Overflow Detection

- Only works with literal operands (both sides must be constants)
- No range inference through assignments or control flow
- Would need abstract interpretation or interval analysis

3.0.0.188 Type Tag Inference

- Binary expressions: walks leftmost chain heuristically
- Complex expressions (function calls, casts) → tagged as unknown (-1)
- Unknown tags prevent optimization for entire function

3.0.0.189 Cross-Function Analysis

- Each function analyzed in isolation
- No interprocedural flows (callee results don't propagate to callers)
- No module boundary crossing

3.0.0.190 No Incremental Solving

- Fresh solver per query — no context reuse between queries
- Could cache Z3 context for related queries in same function

---

3.0.0.191 Bugs, Workarounds, & Notes (Log observations here as we work through v0.5.x)

3.0.0.192 v0.5.0 Work Log

3.0.0.193 Result Elision — IMPLEMENTED

What: Static analysis identifies functions that can never fail (no `fail`, no `sys`, all callees also infallible). These return raw T instead of `{T, ptr, i8}`, eliminating wrapping/unwrapping overhead at every call site.

Activation: `--smt-opt` flag (no Z3 needed — pure static analysis)

Phase: 3.75 (between Borrow Checker and IR Generation)

Analysis algorithm: 1. Collect all user functions with bodies (skip main, failsafe, generics, externs) 2. Walk each function's AST recursively, recording: - `has_fail`: contains any FAIL node - `has_sys_call`: calls sys/sys!!/sys!!! - `called_user_funcs`: set of user-function callees 3. Fixed-point iteration: a function is infallible iff `!has_fail && !has_sys_call && all callees are infallible`. Propagate until stable.

Files modified: - `ir_generator.h` — added `result_elide_funcs` set, `result_elide_mode` flag, `setResultElideFuncs()` setter - `main.cpp` — Phase 3.75 analysis pass (~200 lines), passes set to IR gen - `ir_generator.cpp` — Pre-pass forward decl checks elision, FUNC_DECL sets elide flag, Result wrapping decision uses elide flag, pass() returns raw T for elided funcs

Key insight: Call-site unwrap code needed NO modification — it detects Result by structural shape `{T, ptr, i8}`. Elided functions return raw T, so the struct check naturally short-circuits.

Bug found/fixed: Pre-pass forward declaration was creating functions with Result return type before main codegen. Added elision check to pre-pass too.

Also fixed: Default return for non-struct types used `ConstantInt::get(type, 0)` which only works for integers. Changed to `Constant::getNullValue(type)` which works for all types.

3.0.0.194 Future: Best Practices Guide Topics

- When to use `--verify` vs `--smt-opt` vs `--verify-all`
- Writing solver-friendly code (how to make your code provable)

- Understanding counterexamples
- Performance impact of verification (build time budgets)
- Dealing with UNKNOWN results
- Contract authoring patterns
- Rules design for verifiability

3.0.0.195 Garbage Collection Tuning Guide

v0.8.4 — Aria Compiler Documentation

3.0.0.196 Overview Aria uses a **hybrid generational garbage collector** with two regions:

- **Nursery (Young Generation)**: A copying (Cheney-style) semi-space for short-lived allocations.
- **Old Generation**: A mark-sweep region for long-lived objects promoted from the nursery.

3.0.0.197 GC Modes

3.0.0.198 Stop-the-World (STW) All mutator threads are paused during collection. Default for minor (nursery) collections.

3.0.0.199 Concurrent Major collections use **Snapshot-At-The-Beginning (SATB)** marking with incremental sweeping. Mutator threads continue running during the mark phase.

3.0.0.200 Safepoints Safepoints are injected automatically at: - Loop back-edges (every iteration boundary) - Function call sites

At safepoints, the runtime checks if a GC is pending and yields if needed.

3.0.0.201 Shadow Stack Root Scanning The GC uses a **shadow stack** to track root references. Every function that allocates GC-managed objects pushes roots onto the shadow stack and pops them on exit.

3.0.0.202 Coroutine Integration (v0.8.4) Suspended coroutine frames contain GC roots that must be scanned during collection. The `GCCoroAllocator::scan_frames()` method iterates all suspended frames and marks their roots during both minor and major GC cycles.

3.0.0.203 JIT Root Tracking (v0.8.4) JIT-compiled code can register GC roots via: - `aria_gc_register_jit_root(vo` `root_addr)` — Register a root location - `aria_gc_unregister_jit_root(void** root_addr)` — Unregister when no longer needed

JIT roots are scanned during all GC phases — minor evacuation, STW major marking, and concurrent major marking.

3.0.0.204 Object Pinning (Wild Interop) Objects passed to `wild` (unsafe FFI) blocks are **pinned** in place so external C code retains valid pointers. Pinned objects are: - Skipped during nursery evacuation (not moved) - Scanned in-place during old-gen marking

3.0.0.205 Write Barriers A **card table** tracks cross-generational references. When an old-gen object is updated to point to a nursery object, the corresponding card is dirtied. During minor GC, dirty cards are scanned for additional roots.

3.0.0.206 Tuning Parameters

Parameter	Default	Description
Nursery size	4 MB	Semi-space size for young generation
Promotion threshold	2	Survive N minor GCs before promotion
Major GC trigger	75%	Old-gen occupancy threshold
Concurrent mode	Auto	STW for small heaps, concurrent for large

3.0.0.207 RFC: Traits & Borrow Semantics for Aria

Status: Design Proposal (v0.2.4)
Implementation Target: v0.2.5 or v0.2.6
Author: AI-assisted design

3.0.0.208 Motivation

Two gaps in Aria's type system limit what real programs can express:

1. **No polymorphic dispatch.** You can't write a function that works on "anything with an `.encode()` method." Database backends, serialization formats, and generic algorithms all need this.
2. **No way to share handles.** Move-by-default consumes every variable on first use. Passing a DB handle to two functions requires the `last_db()` re-acquisition hack. Borrows exist in the checker but aren't user-facing.

Both are needed for Nikola: DB abstraction across SQLite/PostgreSQL/Redis, GPU handle passing, iterator patterns, and generic collections.

3.0.0.209 Part 1: Traits

3.0.0.210 Design Goals

- **Monomorphization by default** — zero runtime cost, the compiler stamps out a copy per concrete type
- **Vtable dispatch opt-in** — `dyn Trait` for cases where you truly need runtime polymorphism
- **No trait soup** — keep the common case simple, don't require 5 bounds on every function
- **Compose with UFCS** — `obj.method()` already desugars to `Type_method(obj)`, traits extend this naturally
- **Compose with Type declarations** — `Type:Person` already groups struct + methods, trait conformance plugs in

3.0.0.211 Syntax

3.0.0.212 Defining a trait

```
trait:Encodable = {
    func:encode = string(Self:self);
    func:size_hint = int32(Self:self);
};
```

- `Self` refers to the implementing type (resolved at compile time for monomorphization)
- Trait body contains function signatures — no default implementations in v1 (KISS)
- Traits can require other traits: `trait:Ordered = Equatable & { func:compare = int32(Self:a, Self:b); };`

3.0.0.213 Implementing a trait

```
impl Encodable for Vec3 {
    func:encode = string(Vec3:self) {
        pass("(" + int32_toString(self.x) + "," + int32_toString(self.y) + ")");
    };
    func:size_hint = int32(Vec3:self) {
        pass(20i32);
    };
};
```

- `impl Trait for Type { ... }` — explicit, clear, hard to confuse
- Each function must match the trait signature exactly (with `Self` replaced by the concrete type)
- An impl block can live in any file (not just where the type or trait is defined)

3.0.0.214 Integrating with Type declarations

```
Type:Person = {
    struct:internal = { int32:id; };
    struct:interface = { string:name; int32:age; };

    // Type methods (UFCS - already works)
    func:create = Person(string:name, int32:age) { ... };
    func:greet = NIL(Person:self) { print(self.name); pass(NIL); };

    // Trait conformance declared inline
    conforms Encodable;
    conforms Equatable;
};

// Implementations can be separate
impl Encodable for Person { ... };
```

- `conforms Trait;` inside a Type declaration is a compile-time assertion — the compiler checks that an `impl` block exists
- This keeps the Type declaration as a readable summary of what a type can do

3.0.0.215 Using trait bounds

```
func<T: Encodable>:write_all = NIL(T[]:items) {
    int32:i = 0i32;
    while (i < items.length) {
        print(items[i].encode());
        i = i + 1i32;
    }
    pass(NIL);
};
```

- Generic param constraints: `<T: Encodable>`, `<T: Encodable & Ordered>` (parser already handles this)
- The compiler monomorphizes: `write_all::<Vec3>` generates a concrete function with `Vec3_encode` calls
- UFCS glue: `items[i].encode()` → resolved to `Vec3_encode(items[i])` after monomorphization

3.0.0.216 Dynamic dispatch (opt-in)

```
func:write_any = NIL(dyn Encodable:item) {
    print(item.encode());
    pass(NIL);
};
```

- `dyn Trait` is a fat pointer (data ptr + vtable ptr)
- Runtime cost — only use when you need heterogeneous collections or plugin-style extensibility
- The compiler builds a vtable struct: `{ ptr encode_fn; ptr size_hint_fn; }`

3.0.0.217 Standard Traits (Proposed)

Trait	Methods	Notes
`Equatable`	`equals(Self, Self) -> bool`	`==` operator desugars to this
`Ordered`	`compare(Self, Self) -> int32`	`<`, `>`, `<=`, `>=` desugar
`Hashable`	`hash(Self) -> int64`	Required for HashMap keys
`Copyable`	`copy(Self) -> Self`	Opt-in value semantics (most types are move-only)
`Displayable`	`to_string(Self) -> string`	`print()` and string interpolation
`Addable`	`add(Self, Self) -> Self`	`+` operator, plus `Subtractable`, `Multipliable`, `Dividable`
`Defaultable`	`default() -> Self`	Zero-value construction
`Droppable`	`finalize(Self) -> NIL`	Custom cleanup (called by `drop()` or scope exit)

3.0.0.218 Implementation Strategy

1. **Parser** — `trait:Name = { ... };` and `impl Trait for Type { ... }` node types (NT_TRAIT_DECL already reserved)
2. **Type checker** — resolve trait bounds on generic params, verify impl blocks match trait signatures
3. **IR codegen** — monomorphization: stamp out concrete function for each `<T>` instantiation with known type
4. **Vtable codegen** (later) — for `dyn Trait`: generate vtable struct, emit indirect calls

---

3.0.0.219 Part 2: Borrow Semantics

3.0.0.220 Design Goals

- **Make the existing borrow checker user-facing** — the infrastructure is already there (`$` operator, AccessPath, loan tracking)
- **Rust's safety without Rust's learning curve** — Aria should guide, not punish
- **Explicit borrows** — no implicit reference creation, you always see `$`
- **Compatible with move-by-default** — borrows are a controlled exception to the consume-on-use rule

3.0.0.221 Syntax

3.0.0.222 Immutable borrow

```
int32:x = 42i32;
int32$:ref = $x;          // borrow x (immutable)
print(ref);                // use the borrow - x is still alive
print(x);                  // x is still usable - it wasn't moved
```

- `$x` creates an immutable borrow of `x`
- `int32$` is the borrow type (already in the type system as `SafeRefType`)
- Multiple immutable borrows allowed simultaneously

3.0.0.223 Mutable borrow

```
int32:x = 42i32;
int32$:ref = $mut x;       // mutable borrow
ref = 99i32;                // mutate through the borrow
```

```
// x is now 99
// cannot use x while ref is alive (enforced by borrow checker)
```

- `$mut x` creates a mutable borrow
- Only one mutable borrow at a time (borrow checker already enforces this)
- Original variable is frozen while mutably borrowed

3.0.0.224 Passing borrows to functions

```
func:print_value = NIL(int32$:val) {
    print(val);
    pass(NIL);
};

func:increment = NIL(int32$ mut:val) {
    val = val + 1i32;
    pass(NIL);
};

func:main = int32() {
    int32:x = 10i32;
    print_value($x);       // pass immutable borrow - x survives
    increment($mut x);     // pass mutable borrow - x is modified
    print(x);              // prints 11
    pass(0i32);
};
```

- `int32$` in a parameter = accepts an immutable borrow
- `int32$ mut` in a parameter = accepts a mutable borrow
- The caller uses `$x` or `$mut x` at the call site — always explicit

3.0.0.225 Structs and handles

```
// The Nikola use case: pass a DB handle to multiple functions
int32:db = aria_sqlite_open(":memory:");
setup_tables($mut db);      // borrows db, doesn't consume it
insert_data($mut db);       // borrows db again - previous borrow ended
query_data($db);            // immutable borrow - read-only access
aria_sqlite_close(db);      // final use - db is moved/consumed here
```

- No more `last_db()` hack
- The borrow checker ensures `db` isn't used after `close()`
- Mutable borrows are sequential (one at a time), immutable borrows can overlap

3.0.0.226 Split borrows (already implemented)

```
struct:Pair = { int32:a; int32:b; };
Pair:p = Pair{a: 1i32, b: 2i32};
int32$:ref_a = $p.a;       // borrow field a
int32$:ref_b = $p.b;       // borrow field b - disjoint, allowed
// Both refs alive simultaneously - the borrow checker tracks access paths
```

3.0.0.227 Lifetime Rules Aria uses **scope-based lifetimes** (no explicit lifetime annotations like Rust's `'a`):

1. A borrow cannot outlive the scope of the borrowed value
2. The borrow checker uses Appendage Theory: `Depth(Borrower)  Depth(Owner)`

3. Returned borrows are forbidden in v1 — return owned values instead
4. Borrows in structs are forbidden in v1 — use owned values or pointers

These restrictions eliminate the need for lifetime parameters while covering 90% of use cases. The remaining 10% (self-referential structs, borrowed iterators) can use `wild` pointers with explicit unsafe opt-in.

3.0.0.228 Two-Phase Borrows (already implemented)

```
Vector<int32>:vec = Vector{};
vec.push(vec.len());    // Works! push($mut vec, vec.len($vec))
                        // Phase 1: reserve mutable borrow for push
                        // Phase 2: activate after len() completes
```

3.0.0.229 Pin Semantics (# operator — existing)

```
wild int8[1024]:buffer = malloc(1024);
int8#:pinned = #buffer;    // pin: GC won't move this memory
ffi_read(pinned, 1024);    // safe to pass to C - address is stable
// pinned released when scope exits
```

- `#` pins a value in memory (prevents GC relocation)
- Critical for FFI: C functions expect stable pointers
- Borrow checker tracks pin lifetime

3.0.0.230 Part 3: Interaction Between Traits and Borrows

3.0.0.231 Trait methods and self borrowing

```
trait:Printable = {
    func:display = NIL(Self$:self);      // takes immutable borrow of self
};

trait:Resettable = {
    func:reset = NIL(Self$ mut:self);    // takes mutable borrow of self
};
```

- Trait methods declare whether they borrow self immutably (`Self$`) or mutably (`Self$ mut`)
- This lets the compiler enforce at the call site: `obj.display()` auto-borrows immutably, `obj.reset()` auto-borrows mutably
- If a method needs to consume self (like `drop`), it takes `Self:self` (owned)

3.0.0.232 Generic bounds with borrow-friendliness

```
func<T: Printable>:print_twice = NIL(T$:item) {
    item.display();     // first call borrows the borrow (re-borrow)
    item.display();     // second call - fine, immutable borrow
    pass(NIL);
};
```

- Passing `T$` (borrowed) means the function doesn't consume the value
- The caller retains ownership: `print_twice($my_value);`

3.0.0.233 Summary of Syntax Additions

Feature	Syntax	Example
Trait definition	`trait:Name = { ... };`	`trait:Encodable = { func:encode = string(Self:self); };`
Trait implementation	`impl Trait for Type { ... };`	`impl Encodable for Vec3 { ... };`
Trait conformance	`conforms Trait;`	Inside `Type:` declaration
Trait bound	`<T: Trait>`	`func<T: Ordered>:sort = ...`
Multiple bounds	`<T: A & B>`	`func<T: Hashable & Equatable>:lookup = ...`
Dynamic dispatch	`dyn Trait`	`func:f = NIL(dyn Encodable:x)`
Immutable borrow	`$x` / `Type$`	`int32$:ref = $x;`
Mutable borrow	`$mut x` / `Type$ mut`	`int32$ mut:ref = $mut x;`
Pin	`#x` / `Type#`	`int8#:pinned = #buffer;`

3.0.0.234 Open Questions

1. **Default trait methods?** Allowing `func:encode = string(Self:self) { ... };` with a body inside a trait definition. Useful but adds complexity. Propose: defer to v2.
2. **Associated types?** `trait:Iterator = { type Item; func:next = Item$(...); };` — needed for iterators but complex. Propose: defer to v2.
3. **Trait objects in collections?** `dyn Encodable[]:items` for heterogeneous arrays. Needs vtable + fat pointer arrays. Propose: defer to v2.
4. **Operator overloading via traits?** `+` → `Addable.add()`. Propose: yes, include in v1 (natural, expected).
5. **Auto-borrow at call sites?** Should `obj.method()` auto-create `$obj` if method takes `Self$`? Propose: yes, for ergonomics (Rust does this).

3.0.0.235 Implementation Phases

3.0.0.236 Phase 1 (v0.2.5): Core Traits + User-Facing Borrows

- `trait:` declaration parsing and AST
- `impl Trait for Type` parsing and AST
- Trait bound resolution in type checker
- Monomorphization in IR codegen
- User-facing `$x` / `$mut x` borrow syntax
- Standard traits: `Equatable`, `Displayable`, `Copyable`

3.0.0.237 Phase 2 (v0.2.6): Operator Traits + Dynamic Dispatch

- `Addable`, `Ordered`, `Hashable` with operator desugaring
- `dyn Trait` fat pointer support
- `conforms` assertion in Type declarations
- Generic collections (`Vector<T>`, `HashMap<K: Hashable, V>`)

3.0.0.238 Phase 3 (v0.2.7+): Advanced

- Default trait methods
- Associated types
- Trait objects in collections

- Derived traits (`#[derive(Equatable, Hashable)]`)

3.0.0.239 Aria WebAssembly Compilation Guide

3.0.0.240 Overview Aria v0.2.13 adds WebAssembly (WASM) as a compilation target. Compile Aria programs to `.wasm` files that run in WASI-compatible runtimes like `wasmtime`, `wasmer`, or `wasm3`.

3.0.0.241 Prerequisites

- **wasi-libc**: WASI sysroot with libc for wasm32

  ```
  sudo apt install wasi-libc
  ```

- **wasm-ld**: WebAssembly linker (part of LLVM/LLD)

  ```
  sudo apt install lld
  ```

- **wasmtime** (or wasmer): To run the compiled `.wasm` files

  ```
  curl https://wasmtime.dev/install.sh -sSf | bash
  ```

3.0.0.242 Usage

3.0.0.243 Basic Compilation

```
ariac program.aria --emit-wasm -o program.wasm
```

3.0.0.244 With Explicit Target

```
ariac program.aria --emit-wasm --target=wasm32-wasi -o program.wasm
```

3.0.0.245 Running

```
wasmtime program.wasm
```

3.0.0.246 Supported Features

Feature	WASM Support	Notes
Integer arithmetic (int32, int64)	[DONE] Full	
Float arithmetic (flt32, flt64)	[DONE] Full	
Strings	[DONE] Full	concat, compare, trim, case, search, pad
Control flow (if/else, while)	[DONE] Full	
Functions & recursion	[DONE] Full	
Structs	[DONE] Full	
Arrays	[DONE] Full	
Pattern matching	[DONE] Full	
Error handling (Result types)	[DONE] Full	
File I/O	[WARN] WASI only	Needs WASI runtime with filesystem access
stdin/stdout	[DONE] Full	Via WASI fd_read/fd_write
Math functions	[DONE] Full	Software implementations
Generics	[DONE] Full	

3.0.0.247 Unsupported Features (WASM Target) These features are **not available** when compiling to WASM. The compiler will emit warnings if your code uses them:

Feature	Reason
Async/await	No io_uring in WASM
Threading & thread pools	WASM is single-threaded (for now)
Mutex/atomics	No threading support
Process spawning (fork/exec)	WASM sandbox restriction
Signals	WASM has no signal model
mmap/mprotect	WASM uses linear memory
Raw syscalls	WASM sandbox restriction
FFI to native .so/.dll	WASM has its own import model

The compiler's compatibility checker scans your code and warns about unsupported features before linking.

3.0.0.248 Architecture

```
 program.aria   >    Aria Parser

                            AST

                      IR Generator

                            LLVM IR

                     WASM Codegen   (target: wasm32-wasi)

                            .o (WASM object)

                      wasm-ld
                  + libaria_runtime_wasm
                  + wasi-libc


                     program.wasm
```

The WASM backend: 1. Sets target triple to `wasm32-wasi` 2. Renames `main` → `__main_argc_argv` (WASI convention) 3. Emits a WASM object file via LLVM's WebAssembly backend 4. Links with `wasm-ld` against `libaria_runtime_wasm.a` and WASI libc

3.0.0.249 WASM Runtime

`libaria_runtime_wasm.a` is a stripped-down, WASI-compatible replacement for the native `libaria_runtime`. It provides:

- **String operations**: All `_simple` string functions (concat, trim, case, search, pad, substring, etc.)
- **I/O**: Print, file read/write (via WASI), stdin
- **Math**: All math functions (pow, sqrt, trig, abs, min/max)
- **Memory**: Arena, pool, slab allocators (simplified)
- **Maps**: Hash map with FNV-1a hashing
- **Compiler builtins**: `__multi3` (128-bit multiply for int64 arithmetic)

3.0.0.250 Examples

3.0.0.251 Hello World

```
func:main = int32() {
    out("Hello from WebAssembly!");
```

```
    return 0i32;
};

func:failsafe = void(int32:err_code) {};
```

```
ariac hello.aria --emit-wasm -o hello.wasm
wasmtime hello.wasm
#### Output: Hello from WebAssembly!
```

3.0.0.252 String Operations

```
func:main = int32() {
    str:greeting = "Hello, ";
    str:name = "World";
    str:message = greeting + name + "!";
    out(message);
    return 0i32;
};

func:failsafe = void(int32:err_code) {};
```

3.0.0.253 Integer Math & Control Flow

```
func:factorial = int64(int64:n) {
    if n <= 1i64 {
        return 1i64;
    };
    int64:sub = n - 1i64;
    int64:rec = factorial(sub);
    int64:result = n * rec;
    return result;
};

func:main = int32() {
    int64:fact5 = factorial(5i64);
    // fact5 == 120
    return 0i32;
};

func:failsafe = void(int32:err_code) {};
```

3.0.0.254 Binary Size Typical WASM binary sizes: - Hello world: ~15-20 KB - String operations: ~25-35 KB - Math-heavy programs: ~20-30 KB

These sizes include the WASM runtime and WASI libc stubs.

3.0.0.255 Troubleshooting

3.0.0.256 "undefined symbol" errors Your program may use a feature not available in the WASM runtime. Check the unsupported features table above.

3.0.0.257 "function signature mismatch" This typically indicates an ABI mismatch between your code and the WASM runtime. Ensure you're using the latest `libaria_runtime_wasm.a` built alongside the compiler.

3.0.0.258 wasmtime permission errors for file I/O WASI sandboxes filesystem access. Grant directory access:

```
wasmtime --dir=. program.wasm
```

3.0.0.259 Cross-Language Bindings Aria can interoperate with C, Python, and any language that supports C ABI calling conventions.

3.0.0.260 Overview Aria's compilation pipeline (Aria → LLVM IR → native code) produces symbols with standard C ABI linkage. This means Aria functions are callable from any language that can load shared libraries and call C functions.

Three binding directions:

Direction	Mechanism	Status
C → Aria	`extern` FFI blocks	Stable (v0.1.0+)
Aria → C	`--shared` flag produces `.so`	New in v0.2.6
Aria → Python	`ctypes.CDLL()` on `.so`	New in v0.2.6

3.0.0.261 Aria → C (Exporting Aria Functions) Compile Aria source to a shared library that C programs can link against.

3.0.0.262 Step 1: Write Aria Library

```
// mathlib.aria
func:add = int32(int32:a, int32:b) {
    pass(a + b);
};

func:multiply = int32(int32:x, int32:y) {
    pass(x * y);
};

func:square = int32(int32:n) {
    pass(n * n);
};
```

3.0.0.263 Step 2: Compile to Shared Library

```
ariac mathlib.aria --shared -o libmathlib.so
```

The `--shared` flag: - Skips the `failsafe` requirement (library mode) - Compiles with Position-Independent Code (PIC) - Links as a shared library (.so)

3.0.0.264 Step 3: Use from C

```
#include <stdio.h>

// Declare the Aria functions
extern int add(int a, int b);
extern int multiply(int x, int y);
extern int square(int n);

int main() {
    printf("add(3, 4) = %d\n", add(3, 4));
    printf("multiply(5, 6) = %d\n", multiply(5, 6));
```

```
    printf("square(9) = %d\n", square(9));
    return 0;
}
```

```
gcc main.c -L. -lmathlib -Wl,-rpath,. -o main
./main
```

3.0.0.265 Type Mapping (Aria → C)

Aria Type	C Type	Notes
`int8`	`int8_t` / `char`	
`int16`	`int16_t` / `short`	
`int32`	`int32_t` / `int`	
`int64`	`int64_t` / `long long`	
`float32`	`float`	
`float64`	`double`	
`bool`	`int`	0 = false, 1 = true
`string`	`char*`	Null-terminated C string

3.0.0.266 C → Aria (Consuming C Libraries)

Use `extern` blocks to declare C functions, then call them from Aria.

3.0.0.267 Example: Using a C Math Library

```
extern "m" {
    func:sqrt = float64(float64:x);
    func:pow = float64(float64:base, float64:exp);
    func:sin = float64(float64:x);
    func:cos = float64(float64:x);
};

func:failsafe = NIL(int32:code) { pass(NIL); };

func:main = NIL() {
    var:result = sqrt(144.0);
    emit(result);
};
```

```
ariac main.aria -o main -lm
```

3.0.0.268 Using Custom C Shared Libraries

```
extern "mylib" {
    func:do_work = int32(int32:input);
    func:get_version = int32();
};
```

```
ariac main.aria -o main -L./libs -lmylib
```

The `extern "name"` string corresponds to the library name: `libname.so`.

3.0.0.269 Aria → Python (via ctypes)

Python can call Aria shared libraries using the built-in `ctypes` module.

3.0.0.270 Step 1: Compile Aria to Shared Library

```
ariac mathlib.aria --shared -o libmathlib.so
```

3.0.0.271 Step 2: Call from Python

```
import ctypes

#### Load the Aria shared library
lib = ctypes.CDLL('./libmathlib.so')

#### Declare function signatures
lib.add.restype = ctypes.c_int
lib.add.argtypes = [ctypes.c_int, ctypes.c_int]

lib.multiply.restype = ctypes.c_int
lib.multiply.argtypes = [ctypes.c_int, ctypes.c_int]

lib.square.restype = ctypes.c_int
lib.square.argtypes = [ctypes.c_int]

#### Call Aria functions from Python
print(lib.add(10, 20))        # 30
print(lib.multiply(7, 8))     # 56
print(lib.square(9))          # 81
```

3.0.0.272 Python Type Mapping

Aria Type	ctypes Type
`int8`	`ctypes.c_int8`
`int16`	`ctypes.c_int16`
`int32`	`ctypes.c_int`
`int64`	`ctypes.c_longlong`
`float32`	`ctypes.c_float`
`float64`	`ctypes.c_double`
`bool`	`ctypes.c_int`
`string`	`ctypes.c_char_p`

3.0.0.273 Aria → Other Languages

Any language that supports C FFI can call Aria shared libraries:

Language	Mechanism
Rust	`extern "C"` + `#[link]`
Go	`cgo` with `// #cgo LDFLAGS: -lmathlib`
Java	JNI or JNA
Node.js	`ffi-napi` package
Ruby	`fiddle` or `ffi` gem
C#	`[DllImport]` P/Invoke
Zig	`@cImport` or `extern`

Language	Mechanism

Since Aria produces standard C ABI symbols, any language with C interop works.

3.0.0.274 Compiler Flags Reference

Flag	Output	Use Case
(default)	Executable	Standalone programs
`-c`	Object file (.o)	Intermediate compilation
`--shared`	Shared library (.so)	Cross-language bindings
`--emit-llvm`	LLVM IR (.ll)	Debugging, inspection
`--emit-asm`	Assembly (.s)	Low-level inspection

3.0.0.275 Symbol Naming Aria functions use their declared name directly as the symbol name. There is no name mangling.

```
func:my_function = int32(int32:x) { pass(x); };
```

Produces symbol: `my_function` (visible via `nm -D libname.so`).

3.0.0.276 Limitations

- **Strings**: Aria string values passed across the boundary are C-style `char*` pointers. The caller is responsible for memory management of string arguments.
- **Complex types**: Structs and arrays are not yet supported across the FFI boundary. Use scalar types or pointers.
- **Async functions**: Async Aria functions cannot be called from C/Python directly (they return coroutine handles).
- **No header generation**: C header files must be written manually. Planned for future release.

3.0.0.277 Undefined State Prevention: A Multi-Layered Approach

Design Goal: Never reach an undefined state. Ever.

Key Insight: No single mechanism can prevent all undefined states. Different kinds of "wrongness" require different safety layers.

3.0.0.278 The Three Safety Layers

3.0.0.279 Layer 1: Unknown (Computational Uncertainty)

Purpose: Handle indeterminate mathematical values

Scope: Operations that have no well-defined mathematical answer - Division by zero: `5/0`, `0/0` - Infinity arithmetic: `∞ - ∞`, `∞ / ∞` - Complex number edge cases: `sqrt(-1)` (in real mode) - Invalid operations on special numbers

Behavior: - Operation produces `Unknown` value instead of crashing - Unknown propagates through calculations: `(5 + Unknown) → Unknown` - Type system tracks: `ok(value)` returns `false` for Unknown - Compiler enforces checking before use in critical contexts

Philosophy: These aren't errors - they're valid mathematical states representing "genuinely unknowable." The computation continues safely until the Unknown is checked.

Example:

```
func:safe_divide = int32(int32:a, int32:b) {
    int32:result = a / b;  // If b==0, result becomes Unknown

    if (!ok(result)) {
        pass(-1i32);  // Handle the Unknown case
    }

    pass(result);
}
```

Implementation: Division, modulo, and other unsafe operations inject runtime checks that produce Unknown sentinel values instead of undefined behavior.

3.0.0.280 Layer 2: Result (Expected Failures)

Purpose: Handle operations that can legitimately fail

Scope: I/O, parsing, resource allocation, validation - File operations: file not found, permission denied - Network operations: timeout, connection refused - Parsing: invalid format, out of range - Optional computations: lookup not found

Behavior: - Functions return `Result<T>` wrapper - Caller must handle both success and error cases - Unwrap operators provide ergonomic extraction: - `?` - casual unwrap with default - `??` - null coalesce for pointers/optionals - `?!` - emphatic unwrap (semantically stronger intent) - `_?` - shorthand for `drop()` — discard Result without checking - `_!` - shorthand for `raw()` — extract value without checking - Compiler enforces "no checky no val" - can't access `.value` without checking `.is_error`

Philosophy: Failure is a normal part of the operation's contract, not an exceptional circumstance. The type system makes this explicit.

Example:

```
func:load_config = Result<Config>() {
    // Might fail - that's expected
    pass(readFile("config.json"));
```

```
}

func:main = int32() {
    // LOCAL handling: use default if loading fails
    Config:cfg = load_config() ?! getDefaultConfig();
    pass(0i32);
}
```

3.0.0.281 Layer 3: Failsafe (Catastrophic Failures)

Purpose: Emergency handler for unrecoverable errors

Scope: System-level failures that cannot be locally handled - Out of memory - Corrupted program state - Assertion failures - Stack overflow - Hardware faults

Behavior: - Every program **must** define `failsafe(int32:err_code)` function - Compiler enforces presence - won't compile without it - Explicit escalation via `!!!` operator - Empty failsafe is valid - documents conscious decision - Last line of defense before program termination

Philosophy: "Life preserver on the boat even if you can swim." You might never need it, but when the ship sinks, you'll be glad it's there. The requirement creates accountability - you cannot claim "I forgot."

Example:

```
// REQUIRED - every program must have this
func:failsafe = int32(int32:err_code) {
    // Log error, cleanup resources, notify operators
    // Even empty is valid - it documents your choice
    logCriticalError(err_code);
    cleanupResources();
    pass(err_code);
};

func:check_invariants = int32() {
    if (critical_system_invariant_violated) {
        // EMERGENCY: trigger global handler
        !!! INVARIANT_VIOLATION;
    }
    pass(0i32);
}
```

3.0.0.282 Why Multiple Layers? Single-layer approaches fail:

1. **"Just use exceptions"**:
   - Hidden control flow
   - Performance overhead
   - Doesn't distinguish expected vs catastrophic
   - Can be ignored
2. **"Just use Result everywhere"**:
   - Verbose for simple math
   - Wrong abstraction for "unknowable" values
   - Doesn't handle system-level failures
3. **"Just crash on errors"**:
   - Unsafe for production systems
   - No graceful degradation

- Lost opportunity to handle recoverable cases

Our approach: Each layer handles what it's designed for

```
Layer 3: Failsafe (catastrophic)     System-level, unrecoverable
 !!!  operator, mandatory function    "Ship is sinking"

Layer 2: Result<T> (expected fail)   Normal error conditions
 ?, ??, ?! operators                  "Operation might not work"

Layer 1: Unknown (indeterminate)     Mathematical uncertainty
 ok() checker, propagation            "Answer is unknowable"
```

3.0.0.283 The ! Connection The exclamation mark semantically means "not right" across all uses:

- `!` - logical NOT: "this is not true"
- `?!` - emphatic unwrap: "this isn't right, but I have a fallback"
- `!!!` - failsafe call: "VERY MUCH NOT RIGHT - EMERGENCY"

This creates intuitive escalation from local to global error handling.

3.0.0.284 Design Guarantees No undefined states possible because:

1. **Mathematical operations** → Unknown (never undefined)
2. **Expected failures** → Result (type system enforces handling)
3. **Catastrophic failures** → failsafe() (compiler enforces presence)
4. **Null pointers** → Eliminated via optional types and null coalescing
5. **Uninitialized variables** → Compiler enforces initialization
6. **Type confusion** → Static type system prevents

The programmer cannot: - Forget to implement failsafe (compiler error) - Access Result value without checking (compile error) - Use Unknown in critical context unchecked (type system) - Dereference null (no null in type system) - Reach undefined behavior (all paths covered)

3.0.0.285 Implementation Status

☒ Layer 1: Unknown literal and propagation
☒ Layer 2: Result with unwrap operators (?, ??, ?!)
☒ Layer 3: Mandatory failsafe() and !!! operator
☐ Divide-by-zero → Unknown conversion
☐ Overflow → Unknown conversion
☐ Integration testing across all layers

3.0.0.286 Usage Guidelines When to use Unknown: - Mathematical operations that can be indeterminate - Computations that might not have answers - Continue execution with "I don't know" values

When to use Result: - File I/O, network operations - Parsing and validation - Resource allocation - Any operation where failure is a normal possibility

When to use failsafe (!!!): - Assertion violations - Invariant corruption - Out of memory - Unrecoverable system errors - "This should never happen" scenarios

---

3.0.0.287 The Life Preserver Analogy **Unknown** = Swim training - Learn to handle rough conditions - Part of normal operations - Expected to encounter sometimes

Result = Life jacket - Comfortable to wear always - Expected to need occasionally - Normal safety equipment

Failsafe = Emergency life raft - Mandatory on every vessel - Hope to never use - Critical when needed - Required by regulation (compiler)

You can be an expert swimmer, but the boat still needs emergency equipment. The compiler enforces this because lives (or at least data) depend on it.

3.0.0.288 Direct Syscalls and Safety (v0.4.0) The `sys()` builtin extends Aria's escalation model to kernel-level operations:

Construct	Danger	Analogy
`sys(WRITE, ...)`	Safe	Life jacket — curated whitelist, Result-wrapped
`sys!!(KILL, ...)`	Medium	Emergency flare — all syscalls, still Result-wrapped
`sys!!!(nr, ...)`	High	Abandoning ship — bare int64, no safety net

The safe tier cannot be bypassed with variables or expressions — only named constants from the compiler's whitelist. This prevents accidental escalation and makes dangerous syscall usage grep-able and auditable.

Author's Note: "I don't think one approach can cover it all. The unknown was the original idea, and I think it can cover most if we can somehow make things like divide by zero turn into unknown rather than undefined behind the scenes. Then they can actually handle that instead of crashing you know. But the failsafe was like… requiring people to have a life preserver on the boat even if they can swim. It's for when things don't go as planned."

3.0.0.289 Evaluation of the Aria Programming Language for Safety-Critical and Physics-Based AI Systems The engineering of safety-critical software—spanning domains such as aerospace flight control, nuclear reactor management, autonomous vehicular navigation, and the emerging field of Artificial General Intelligence (AGI)—demands programming paradigms that systematically eradicate undefined behavior, memory corruption, and non-deterministic execution.1 Historically, the software industry has addressed these stringent requirements by relying on heavily constrained subsets of legacy programming languages. For instance, C and C++ are frequently utilized under strict adherence to MISRA or AUTOSAR guidelines to mitigate their inherent vulnerabilities, while specialized languages such as Ada and its formally verifiable subset, SPARK, have long served as the gold standard for high-integrity systems.2 In recent years, Rust has emerged as a formidable alternative, introducing compile-time memory safety through its rigorous ownership model and borrowing semantics, thereby eliminating the need for garbage collection while preventing data races.5

The Aria programming language enters this highly rigorous ecosystem with a specialized focus on physics-based artificial intelligence models, wave-based computation, and AGI consciousness substrates.1 Aria attempts to reconcile the draconian safety requirements mandated by certification standards like DO-178C (for civil aviation) and ISO 26262 (for automotive functional safety) with the highly connected, graph-heavy, and mathematically intense memory patterns required by advanced physics simulations and neural architectures.1 By codifying a layered safety system, deterministic twisted floating-point arithmetic, generational memory handles, and compile-time dimensional analysis, Aria presents a novel and exhaustive architecture.1 If fully implemented and verified according to its specification, Aria represents a significant evolutionary branch in systems programming. It diverges from Rust's static lifetime analysis in favor of structural isolation, emphasizes sticky error propagation over exception handling, and prioritizes absolute mathematical determinism across heterogeneous hardware.1 The following analysis evaluates the language's specified features against contemporary safety-critical standards and competing programming languages.

3.0.0.290 The Epistemology of Error: Layered Safety and the Elimination of Undefined Behavior The defining characteristic of any programming language intended for safety-critical environments is its treatment of anomalous states and edge cases. In standard C and C++, operations such as signed integer overflow, division by zero, or out-of-bounds memory access result in undefined behavior (UB).1 Modern optimizing compilers, such as those based on the LLVM infrastructure, utilize undefined behavior as an aggressive optimization license; the compiler assumes UB will never occur and routinely eliminates safety branches or fabricates values to minimize the generated instruction count.17 In safety-critical contexts, this compiler behavior leads to catastrophic, silent divergences between the source code's logical intent and the executed machine code.16

Aria's design philosophy fundamentally rejects the concept of undefined behavior, operating under the strict doctrine that software must never reach an undefined state that causes an immediate, uncontrollable crash.1 To achieve this, the language constructs a defense-in-depth architecture composed of three distinct and non-negotiable safety layers: the Result<T> system for hard errors, the unknown sentinel state for soft errors, and the mandatory failsafe() routine for unrecoverable systemic failures.1

3.0.0.291 The Result System and Explicit Accountability Aria's primary error-handling layer mandates that all functions implicitly return a Result<T> struct, which physically encapsulates a success value, a void pointer to an error message, and a boolean error flag.1 This mechanism is conceptually analogous to Rust's Result<T, E> enum, forcing the developer to explicitly acknowledge and handle the possibility of failure before the underlying data can be accessed.5 However, Aria streamlines the cognitive overhead by unifying the error type into a standardized structure, integrating it directly with language-level operators.1

The divergence from mainstream languages lies in Aria's compiler-enforced accountability and syntactic integration. Languages like Java or Python rely on exceptions, which introduce hidden control flow paths, significant stack-unwinding overhead at runtime, and the persistent danger of unhandled exceptions crashing the host process.1 Conversely, C relies on integer return codes, which lack type safety and are effortlessly ignored by developers.1 By making the Result type pervasive and syntactically binding it to sticky error propagation, Aria ensures that recoverable errors—such as file I/O failures, network timeouts, or parsing

errors—are handled explicitly at the boundary of their occurrence.1

Aria provides a suite of unwrap operators to interact with this system seamlessly. The safe unwrap operator (?) allows the developer to provide a default fallback value if the operation fails, ensuring continuous execution.1 The null coalescing operator (??) provides similar fallback functionality specifically for NIL optional values, differentiating between an explicit error state and a benign absence of data.1 Furthermore, Aria guarantees zero-overhead on the success path; checking the Result struct's boolean flag requires only a single branch instruction, which modern CPU branch predictors optimize to negligible latency.1

3.0.0.292 The Unknown State: Enabling Fail-Operational Design The second layer of Aria's safety architecture addresses operations that traditionally trigger hardware traps, panics, or undefined behavior, such as division by zero, invalid type conversions, or out-of-bounds array accesses.1 Instead of crashing the application or triggering a controlled panic (as Rust would in safe mode), Aria maps these anomalous operations to a defined unknown sentinel value.1

This design choice aligns flawlessly with the concept of "fail-operational" systems, a critical architectural requirement in modern aerospace design and highly automated driving frameworks.24 A fail-operational system must continue to function, albeit potentially in a degraded state, when a subsystem fails, rather than simply shutting down.26 By allowing an unknown value to propagate systematically through a chain of calculations—similar to how a Not-a-Number (NaN) value propagates in floating-point math, but highly controlled and type-checked across all data types—the software avoids abrupt termination.1

The developer is required to use the ok() function to explicitly verify the data's validity before it can be passed back into the strict Result<T> system or used in a critical control path.1 This mechanism provides the overall system a crucial operational window to apply analytical redundancy, execute sensor fusion algorithms, or trigger fallback logic before a complete functional collapse occurs.1

3.0.0.293 The Mandatory Failsafe: Enforcing the Safe State When fail-operational mitigation strategies are exhausted, or when an error is fundamentally unrecoverable, safety-critical systems must seamlessly transition to a "fail-safe" mode, achieving a state where no harm can occur to human life or physical property.24 Aria codifies this transition at the compiler level by requiring every single program to define a global failsafe(int32:err_code) function.1 This function acts as the ultimate, un-bypassable sink for unrecoverable errors, out-of-memory conditions, stack overflows, and assertions.1

This language-level mandate is a paradigm shift. While MISRA C guidelines and ISO 26262 standards strictly require systems to possess a safe state transition protocol—such as de-energizing industrial motors, dropping nuclear control rods, or severing fuel lines—these are traditionally implemented as architectural design patterns rather than compiler-enforced language semantics.10 Aria's strict refusal to compile without a defined, non-empty failsafe() routine forces developers to codify their emergency termination protocols at the inception of the project.1

Aria introduces the emphatic unwrap operator (?!) and the direct failsafe operator (!!!) to interface with this layer. The ?! operator attempts to unwrap a Result, but immediately invokes the failsafe() routine if an error is present, signaling that the software cannot safely continue without the requested data.1 The !!! operator provides unconditional emergency termination, bypassing the Result system entirely to assert that an impossible state has been reached and the system must be halted.1

Error Handling Paradigm	C / C++	Rust	Ada / SPARK	Aria
Expected Errors	Integer Return Codes	Result<T, E> Enum	Exceptions / Out Params	Result<T> Struct (Mandatory)
Math Anomalies / OOB	Undefined Behavior	panic! (Controlled Crash)	Exceptions / Formal Proof	unknown Sentinel (Fail-Operational)
Terminal State	abort(), exit()	panic!, abort	Exception Propagation	failsafe() (Compiler Mandated)

Error Handling Paradigm	C / C++	Rust	Ada / SPARK	Aria
Safety Bypass	Unrestricted	unsafe block	Restricted Features	Explicit TOS Operators (raw, drop)

3.0.0.294 Mathematical Determinism and Twisted Numeric Representation Aria's primary target domain includes distributed Artificial General Intelligence substrates and hyper-accurate physics simulations. In these environments, identical mathematical operations must yield perfectly bit-exact results across heterogeneous hardware architectures (e.g., x86, ARM, RISC-V) to maintain network consensus and prevent the corruption of complex physical state manifolds.1 Standard floating-point arithmetic, governed by the IEEE 754 standard, catastrophically fails to provide this absolute guarantee.39

3.0.0.295 The Rejection of IEEE 754 IEEE 754 floating-point calculations are notoriously non-deterministic across different central processing units.39 Depending on the specific hardware and the compiler's optimization flags, instructions may be fused into Multiply-Add (FMA) operations, intermediate calculations may be temporarily stored in extended 80-bit precision registers, and rounding modes may be altered dynamically by shared mutable state within the floating-point environment.40 Furthermore, IEEE 754 introduces mathematically problematic artifacts such as negative zero (-0.0) and multiple binary representations of Not-a-Number (NaN).1 These artifacts can propagate silently through millions of operations, corrupting logical branching and destroying zero-neutrality.1 In the context of an AGI physics engine—where quantum states or neural topologies are iteratively calculated over sustained periods—even sub-atomic floating-point drift inevitably cascades into systemic divergence, fracturing the simulation.1

Aria completely abandons hardware-accelerated IEEE floats for critical calculations, providing its own strictly deterministic software-implemented types: fix256 (deterministic fixed-point) and tfp64 (Twisted Floating Point).1 The fix256 type utilizes a massive Q128.128 representation, allocating 128 bits for the integer component and 128 bits for the fractional component.1 This provides astronomical precision down to 2^{-128} (significantly finer than the Planck length) while ensuring that all operations execute under the hood as pure, deterministic integer arithmetic.1 Fixed-point arithmetic avoids the non-associativity of floating-point math, making it the required standard for deterministic systems in aerospace telemetry and lockstep physics engines.39

For scientific applications requiring a massive dynamic range that fixed-point cannot provide, Aria introduces the tfp32 and tfp64 types. These construct a floating-point representation using Aria's specialized integer components for both the mantissa and the exponent, executing the arithmetic entirely in software.1 While this incurs a steep performance penalty (estimated at 10x to 50x slower than hardware FPU execution), it guarantees absolute cross-platform bit-exactness, collapses all mathematical error states into a single unified ERR sentinel, and entirely eliminates the -0 anomaly.1

3.0.0.296 Twisted Balanced Binary (TBB) and Sticky Error Propagation Aria addresses the vulnerabilities of standard integer arithmetic through its proprietary Twisted Balanced Binary (TBB) types, available in tbb8, tbb16, tbb32, and tbb64 variants.1 Standard two's complement integers exhibit a dangerous asymmetry; for example, an 8-bit signed integer ranges from -128 to +127. Because the absolute value of -128 is +128, which exceeds the maximum positive limit, attempting to negate the minimum value results in a silent wraparound or an overflow panic in languages like Rust.1

Aria's TBB types structurally eliminate this vulnerability by sacrificing the absolute minimum value (e.g., -128 in a tbb8) to serve as a hardware-level ERR sentinel.1 This yields a perfectly symmetric valid numeric range (-127 to +127), making absolute value and negation operations mathematically flawless.1

More importantly, this architectural decision enables "sticky error propagation" at the bare-metal level.1 Whenever an overflow, underflow, or invalid operation occurs, the result collapses into the ERR sentinel.1 Any

subsequent mathematical operation involving an ERR sentinel natively evaluates to ERR.1 This eliminates the need for the compiler to inject expensive branching checks after every single arithmetic instruction—a common and severe performance bottleneck in safe C++ implementations or Rust's checked-math modes.49 The application can execute a massive, vectorized pipeline of calculations and simply check for the ERR sentinel at the termination of the sequence, dramatically increasing throughput without sacrificing safety.1

3.0.0.297 Exact Rational Arithmetic and LBIM Cryptographic Types Complementing its deterministic floats, Aria provides Fraction types (frac8 through frac64) to enable mathematically exact rational arithmetic.1 A frac struct maintains a whole number, a numerator, and a denominator, constantly reducing to canonical form via Greatest Common Divisor (GCD) algorithms to entirely prevent the accumulation of rounding errors common in recursive division.1

For high-security consensus algorithms and communications, Aria embeds Large BigInt Math (LBIM) types directly into the language primitives, scaling from int1024 up to a strictly enforced limit of int4096 and uint4096.1 These massive types are specifically designed for post-quantum and quantum-resistant cryptography (e.g., RSA-16384, lattice-based cryptography).1 By defining these cryptographic types as fundamental language primitives rather than relying on external, third-party libraries (which are historically frequent sources of vulnerabilities via side-channel attacks or memory mismanagement), Aria ensures that cryptographic operations inherit the exact same sticky error propagation, division-by-zero protection, and deterministic constraints as native integers.1

Feature	IEEE 754 / Standard Int	Aria TBB / TFP / Fix	Safety & Physics Impact
Symmetry	Asymmetric (e.g., -128 to 127)	Symmetric (e.g., -127 to 127)	Prevents absolute value and negation overflow panics.
Error State	NaN (Float), Wraparound (Int)	Unified ERR Sentinel	Halts silent data corruption; explicitly tracked by compiler.
Determinism	Hardware-dependent, FMA drift	Bit-exact cross-platform	Ensures identical simulations across heterogeneous AGI nodes.
Propagation	NaN poisoning (silent)	Sticky ERR propagation	Allows batch processing without branch-per-instruction overhead.

3.0.0.298 Memory Architecture: Generational Handles vs. The Borrow Checker Memory safety remains the paramount challenge in systems programming. Vulnerabilities such as use-after-free (UAF), double-free, and buffer overflows account for the vast majority of critical security exploits and system crashes worldwide.21 Rust revolutionized this space by introducing the borrow checker, a compile-time mechanism that enforces strict single-mutable ownership and tracks lifetimes to guarantee memory safety without relying on a garbage collector.5

However, the strict, tree-based ownership hierarchy demanded by Rust's borrow checker is notoriously hostile to the highly connected, self-referential graph data structures required in AGI neural topologies, real-time physics simulations, and Entity Component Systems (ECS).57 To build complex graphs in Rust, developers are often forced to wrap data in runtime reference counting constructs (Rc or Arc), which introduce significant performance overhead and thread contention. Alternatively, they fall back to using integer indices into arrays, which circumvents the borrow checker but merely masks the memory management problem without guaranteeing temporal safety against stale indices.56

3.0.0.299 Generational Handles and Arena Allocation Aria aggressively solves the graph-structure problem by adopting Generational Handles paired with Arena allocators as a fundamental language construct.1 A Handle<T> is a lightweight, 12-byte structure comprising an index (uint64) and a generation counter (uint32).1 It acts as a safe reference to memory, completely replacing raw pointers for dynamic, interconnected data.1

When an object—such as a wave node in Aria's physics engine or a neuron in its Sparse Hyper-Voxel Octree (SHVO)—is allocated in a typed arena, the arena issues a handle containing the slot's current generation number.1 If the arena is forced to reallocate (e.g., during exponential memory expansion or "neurogenesis"), or if the specific object is explicitly freed, the arena increments the generation counter for that memory slot.1 When the program subsequently attempts to access the data using the old handle, the generation mismatch is immediately detected.1 Instead of allowing a catastrophic use-after-free read or write, the arena.get() function safely returns an ERR via the Result<T> system.1

This paradigm fundamentally shifts memory safety from a static compile-time proof (as seen in Rust) to an $O(1)$ dynamic runtime check.1 While this theoretically introduces runtime overhead, modern CPU branch predictors optimize the generation comparison check to near-zero latency on the hot path.1 Generational handles provide deterministic, rapid, and completely memory-safe graph representations, granting Aria a vital architectural advantage over Rust for building scalable AGI memory substrates without wrestling with lifetime annotations.1

3.0.0.300 Explicit Allocation Modes and RAII Semantics To accommodate diverse real-time constraints and prevent hidden performance latency, Aria mandates explicit allocation modes, removing the dangerous ambiguity of implicit heap allocations found in many high-level languages.1

- **stack**: The default mode, offering ultra-fast $O(1)$ allocation via pointer bumping, constrained strictly by lexical scope lifetimes.
- **gc**: Opt-in garbage collection for non-critical code with complex shared ownership, though heavily discouraged in deterministic physics loops due to unpredictable pause times.
- **wild**: Manual, unmanaged heap allocation (akin to C's malloc) designed for absolute real-time determinism where the developer takes full responsibility.
- **wildx**: Executable memory allocation strictly isolated for Just-In-Time (JIT) compilation requirements.1

When utilizing wild memory, Aria mitigates the severe risk of memory leaks through the defer keyword.1 The defer statement guarantees that designated cleanup code executes in Last-In-First-Out (LIFO) order upon scope exit.1 Crucially, this execution is guaranteed regardless of whether the scope exits normally, via an early return, or through an error/failsafe trigger.1 This provides the safety benefits of C++ Resource Acquisition Is Initialization (RAII) without the immense complexity of hidden constructor/destructor control flows.1

3.0.0.301 Borrow Semantics and Pinning While generational handles dominate dynamic graph allocations, later updates to the Aria specification (v0.2.35) incorporated an opt-in compile-time borrow checker to provide localized memory safety and prevent data races.1 Utilizing the

$$m(mutable, exclusive) and$$

i (immutable, shared) qualifiers, Aria enforces standard aliasing rules to ensure safe concurrent access during parallel physics computations.1 Furthermore, Aria introduces the pinning operator (#), which guarantees

that a variable's memory address remains absolutely stable. This prevents the garbage collector from relocating the data during optimization passes, an essential feature for seamless, zero-copy Foreign Function Interface (FFI) integration with external C libraries.1

Memory Management Strategy	C / C++	Rust	Aria
Primary Safety Mechanism	Manual / Smart Pointers	Borrow Checker / Lifetimes	Generational Handles / Arenas
Temporal Safety (Use-After-Free)	Unsafe (Developer Responsibility)	Statically Proven at Compile-Time	Dynamically Checked (Handles)
Graph / Tree Structures	Trivial but entirely unsafe	Difficult (Requires Rc / unsafe)	Trivial and fully safe (Handles)
Cleanup Guarantees	Manual / RAII	RAII / Drop trait	defer keyword (LIFO execution)
Allocation Visibility	Implicit / Library calls	Implicit (Box, Vec)	Explicit keywords (stack, gc, wild)

3.0.0.302 Semantic Correctness: Contracts, Limits, and Dimensional Analysis Beyond securing memory and preventing arithmetic overflow, safety-critical software must verify that its operations logically align with physical reality and business requirements.38 A structurally flawless program that attempts to add velocity to mass still produces a fundamentally flawed physics simulation, potentially leading to catastrophic real-world outcomes.1

3.0.0.303 Compile-Time Dimensional Analysis Aria elevates physical units of measurement to the type system level, instituting strict, compile-time dimensional analysis.1 By parameterizing numeric types with physical units (e.g., fix256<Joules>, fix256<Meters>, tfp64<Newtons>), the Aria compiler mechanically verifies thermodynamic and physical equations prior to execution.1 Attempting to add time to energy, or equating mass to energy without explicitly applying the c^2 conversion coefficient, triggers an immediate compile-time error.1

While dimensional analysis is relatively rare in mainstream, general-purpose languages, it is highly prized in scientific computing and aerospace engineering. F# successfully implements units of measure, verifying the logic and then stripping the metadata at compile time to incur zero runtime performance overhead.69 The Ada 2012 specification similarly introduced dimensionality checking specifically to prevent multi-million-dollar disasters akin to the Mars Climate Orbiter unit-conversion failure.72 By mandating this at the core of the language rather than relying on external libraries, Aria prevents entire categories of physics errors—a rigorous paradigm the language specification refers to as "Thermodynamic Constitutionalism".1

3.0.0.304 Design by Contract and Refinement Types To further enforce semantic correctness, Aria integrates Design by Contract (DbC) natively into its function syntax using the requires (preconditions) and ensures (postconditions) clauses.1 If an input parameter violates a requires clause at runtime, the function immediately intercepts the execution and automatically returns an error Result, intertwining contractual constraints directly with the sticky error propagation system.1

Aria extends this philosophy with "Rules" and "Limits," effectively implementing Refinement Types.1 Developers can define logical boundaries—such as Rules:r_safe_temp = { $ >= -40, $ <= 120 }—and apply them to variable declarations using the limit keyword.1 If a literal value violates the limit, the compiler throws an error; if a dynamic runtime value breaches the boundary, it triggers the failsafe.1

It is critical to contrast Aria's approach to contracts with the formal methods used by Ada/SPARK. SPARK utilizes deductive formal verification, employing powerful SMT solvers (like Z3, Alt-Ergo, or CVC4) to mathematically *prove* the Absence of Runtime Errors (AoRTE) and verify contractual compliance before the

code is ever compiled into an executable.65 SPARK can formally guarantee that a buffer overflow or contract violation is impossible across all conceivable execution paths.74

As of v0.3.4, Aria has integrated the Z3 SMT solver directly into its compiler pipeline, enabling static formal verification of `requires/ensures` contracts and integer arithmetic overflow at compile time.1 The `--verify-contracts` flag instructs the compiler to translate function preconditions and postconditions into Z3 assertions and mathematically prove their validity across all possible inputs. The `--verify-overflow` flag uses Z3's bitvector overflow intrinsics (`bvadd_no_overflow`, `bvsub_no_underflow`) to prove that integer arithmetic operations cannot overflow for bounded input ranges. A `--verify-report` flag emits detailed proof results per function (proven, unproven, or skipped).

This places Aria's contractual verification power in the same class as SPARK's formal methods—using the same Z3 solver backend—while integrating the proofs directly into Aria's existing Result and failsafe error propagation systems. Aria's Z3 integration also extends to its Rules/limit refinement types (since v0.2.45), where the solver verifies the consistency and satisfiability of compile-time constraints using bitvector-accurate proofs that match Aria's exact integer widths. Combined with the runtime enforcement of contracts and limits as a fallback for cases the solver cannot statically prove, Aria now provides both static and dynamic assurance for contractual compliance.65

3.0.0.305 Concurrency, Polymorphism, and Performance High-performance physics engines and AGI substrates require massively parallel execution across multi-core architectures with minimal synchronization overhead.1 Aria addresses this by mandating Sequential Consistency (SeqCst) as the default memory ordering for all atomic operations.1 This prioritizes developer reasoning and state safety over the raw, unpredictable speed of relaxed memory models.1 The language provides robust, safe primitives for Lock-Free Queues, Seqlocks, and Transactional Locks, requiring developers to explicitly mark weaker memory orderings (using suffixes like _acquire, _release, _relaxed) to alert security auditors to expert-level hardware optimizations.1

For hardware acceleration, Aria introduces explicit SIMD (Single Instruction, Multiple Data) vector types (e.g., simd<fix256, 16>) tailored for modern processor extensions like AVX-512 and ARM NEON.1 Crucially, SIMD operations in Aria utilize masked operations to handle sticky errors without relying on CPU branching.1 This ensures that safety checks do not destroy vectorization throughput—a pervasive limitation in auto-vectorizing C++ compilers where the introduction of a safety branch forces the compiler to abandon SIMD optimizations.1

3.0.0.306 Monomorphization vs. Dynamic Dispatch Aria achieves zero-cost abstraction for generic programming via monomorphization, a technique identical to C++ templates and Rust generics.1 When a generic function or struct is utilized, the compiler generates a specialized, hard-coded copy of that function for the specific type being processed.1 This completely eliminates the need for virtual function tables (vtables) and dynamic dispatch overhead at runtime.1

While monomorphization drastically improves execution speed and allows the compiler to aggressively inline code, it risks "binary bloat"—a legitimate and significant concern for flash-constrained embedded systems in aerospace and automotive hardware.80 However, the sheer execution speed and cache-locality afforded by monomorphization are absolutely essential for Aria to achieve the strict 1-millisecond simulation timesteps demanded by advanced, multi-dimensional AGI physics grids.1

To complement its memory and concurrency models, recent updates to the Aria specification (v0.2.37) introduced Channels and Actors to facilitate thread-safe message passing.1 By providing buffered, unbuffered, and oneshot channels alongside isolated Actor entities, Aria embraces structured concurrency paradigms heavily inspired by Go and Erlang, preventing the catastrophic state corruption that occurs when multiple threads attempt to mutate shared global memory.1

3.0.0.307 Expressive Paradigms and Architectural Syntax While heavily focused on bare-metal safety, Aria provides modern, expressive paradigms to maintain developer productivity without compromising its core tenets. The language treats functions as first-class citizens, supporting closures and lambda

expressions.1 To maintain strict scoping safety, Aria differentiates between safe value capture (which copies data into the closure) and unsafe reference capture (which requires explicit pointer dereferencing and lifetime management by the developer).1

Aria's syntax is meticulously designed to reduce cognitive load and prevent ambiguity. It rejects the overloaded operators common in C++ in favor of distinct, blueprint-style operators that visually indicate the direction of data flow.1 For instance, int64->:ptr denotes a pointer pointing *to* an integer type, @var represents the address *at* a variable, and <-ptr clearly indicates data being pulled *from* a pointer during dereferencing.1

Furthermore, Aria supports advanced string interpolation via template literals (e.g., 'Value: &{x}'), allowing complex expressions and function calls to be evaluated safely within strings, complete with automatic type conversion and multiline support.1 The module system relies on explicit use, mod, and pub keywords, enforcing strict namespace isolation and facilitating clean dependency management in large-scale codebases.1

3.0.0.308 Regulatory Compliance and the Explicit TOS Bypass When evaluating a programming language for commercial safety-critical deployment, technical features must translate into verifiable regulatory compliance capabilities, specifically targeting rigorous frameworks like RTCA DO-178C (Civil Aviation) and ISO 26262 (Automotive Functional Safety).9 Aria's design presents several major advantages for certification:

1. **Zero Implicit Conversion:** Aria strictly enforces literal type suffixes and completely eliminates implicit type coercions.1 This natively enforces at the compiler level one of the most critical and frequently violated rules in the MISRA C coding standard, drastically reducing the required static analysis burden prior to certification audits.1

2. **Absence of Unintended Behavior:** DO-178C places a heavy emphasis on predictable data flow, control coupling, and hardware determinism.13 Aria's systemic removal of undefined behavior, combined with the deterministic fixed-point (fix256) and twisted floating-point (tfp64) math, provides profound assurances that the software will execute exactly as modeled across all operational states.1

3. **Safe State Transitions:** ISO 26262 requires systems to identify hazards and guarantee a transition to a defined Safe State upon failure.10 Aria's language-mandated failsafe() function serves as the ultimate, un-bypassable conduit for this exact requirement, ensuring that Architectural Safety Integrity Level (ASIL) D targets for runtime monitoring and emergency response are structurally satisfied from day one.1

3.0.0.309 The TOS Vocabulary vs. Rust Unsafe In systems programming, developers inevitably require mechanisms to bypass high-level safety constraints to interact directly with hardware or optimize critical paths. Rust utilizes the monolithic unsafe keyword to unlock raw pointer dereferencing, mutable statics, and unchecked array access.54 However, the broad scope of unsafe has drawn scrutiny from security researchers regarding poor developer risk assessment, the lack of granular security policies, and the difficulty of auditing precisely *which* safety rule is being bypassed within an unsafe block.54

Aria categorizes its explicit bypasses into a discrete, highly auditable "Terms of Service" (TOS) vocabulary.1 Instead of a single unsafe block, developers must use specific keywords for specific actions: raw() forces a Result unwrap without checking the error flag; drop() evaluates an expression but throws away the safety Result entirely; wild allocates unmanaged memory; and ok() deliberately strips an unknown sentinel taint from a variable.1 This extreme granularity heavily favors compliance certification, allowing auditors to precisely track and justify where error checking was bypassed versus where memory management was handled manually.1

Despite these immense architectural strengths, Aria currently lacks the decades of ecosystem maturity, certified compiler toolchains, and established developer familiarity that C, C++, and Ada possess.7 To achieve DAL A (DO-178C) or ASIL D (ISO 26262) certification in commercial settings, the Aria compiler itself must undergo rigorous qualification, and structural coverage analysis tools (such as MC/DC analyzers) must be developed specifically for the language.10

3.0.0.310 Conclusion The Aria programming language represents a highly specialized, intensely rigorous approach to safety-critical software architecture. By synthesizing the graph-friendly memory safety of generational handles with the structural accountability of a mandatory failsafe system, and underpinning it all with the absolute determinism of fixed-point and twisted-binary arithmetic, Aria addresses the precise failure modes that plague C/C++ and Rust in highly concurrent, physics-based AI simulations.1

While it presently lacks the static formal verification ecosystem of Ada/SPARK 74, its architectural refusal to allow undefined behavior, its fail-operational unknown state, and its strict thermodynamic constraints position Aria as an exceptionally potent language for its intended domain.1 By making safety an inescapable structural requirement rather than a developer convention, Aria possesses the foundational semantics necessary to become a dominant language in the development of fault-tolerant AGI, aerospace control systems, and high-fidelity physics substrates.

Update (v0.3.4, March 2026): Aria has since integrated the Z3 SMT solver into its compiler pipeline, adding static formal verification of `requires/ensures` contracts and integer arithmetic overflow proofs. This directly addresses the formal verification gap identified above, placing Aria alongside Ada/SPARK in providing compile-time mathematical correctness guarantees for safety-critical code.

3.0.0.311 Works cited

1. aria_specs.txt

2. Should I choose Ada, SPARK, or Rust over C/C++? (2024) | Hacker News, accessed March 28, 2026, https://news.ycombinator.com/item?id=45486829

3. Fixed-Point Arithmetic Unit with a Scaling Mechanism for FPGA-Based Embedded Systems, accessed March 28, 2026, https://www.mdpi.com/2079-9292/10/10/1164

4. IAEA Safety Standards — Deterministic Safety Analysis for Nuclear Power Plants, accessed March 28, 2026, IAEA PUB1851

5. Should I choose Ada, SPARK, or Rust over C/C++? - AdaCore, accessed March 28, 2026, https://www.adacore.com/blog/should-i-choose-ada-spark-or-rust-over-c-c

6. Bringing Rust to Safety-Critical Systems in Space - arXiv, accessed March 28, 2026, https://arxiv.org/html/2405.18135v1

7. What's The Best Language For Safety Critical Software? — Reddit, accessed March 28, 2026, r/programming

8. Programming Languages in Safety-Critical Applications - adesso SE, accessed March 28, 2026, https://www.adesso.de/en/news/blog/programming-languages-in-safety-critical-applications.jsp

9. Rust is DO-178C Certifiable - Pictorus, accessed March 28, 2026, https://blog.pictor.us/rust-is-do-178-certifiable/

10. Functional Safety in Automotive: ISO 26262 Testing Best Practices | QA Systems, accessed March 28, 2026, https://www.qa-systems.com/blog/iso-26262-testing-best-practices/

11. Meeting ISO 26262 Guidelines - Black Duck, accessed March 28, 2026, https://www.blackduck.com/resources/white-papers/ISO26262-guidelines.html

12. Master Thesis — Comparison Ada C Rust, accessed March 28, 2026, ILS Stuttgart

13. DO-178C Compliance in Static Analysis — Parasoft, accessed March 28, 2026, Parasoft

14. The Most Memory Safe Native Programming Language, accessed March 28, 2026, https://vale.dev/memory-safe

15. Ask HN: What if a language's structure determined memory lifetime? | Hacker News, accessed March 28, 2026, https://news.ycombinator.com/item?id=46488290

16. The Compiler as Sentinel: A Comprehensive Analysis of Compiler-Oriented Software Security | by Vipul Kumar | Medium, accessed March 28, 2026, https://medium.com/@vipulkc/the-compiler-as-sentinel-a-comprehensive-analysis-of-compiler-oriented-software-security-f75428dbe820

17. Taming Undefined Behavior in LLVM - Microsoft Research, accessed March 28, 2026, https://www.microsoft.com/en-us/research/publication/taming-undefined-behavior-llvm/

18. A production bug that made me care about undefined behavior - Hacker News, accessed March 28, 2026, https://news.ycombinator.com/item?id=46423521

19. Failure is Required: Understanding Fail-Safe and Fail-Fast Strategies - Medium, accessed March 28, 2026, https://medium.com/javarevisited/failure-is-required-understanding-fail-safe-and-fail-fast-strategies-ac9112fe056d

20. What's your opinion on exceptions? : r/ProgrammingLanguages - Reddit, accessed March 28, 2026, https://www.reddit.com/r/ProgrammingLanguages/comments/o1ye66/whats_your_opinion_on_exceptions/

21. What Is the Most Secure Coding Language? Top Options in 2026, accessed March 28, 2026, https://www.securityjourney.com/post/what-is-the-most-secure-coding-language

22. Software Error Incident Categorizations in Aerospace, accessed March 28, 2026, https://arc.aiaa.org/doi/10.2514/1.I011240

23. Safety measures : r/AskProgramming - Reddit, accessed March 28, 2026, https://www.reddit.com/r/AskProgramming/comments/16pgxfy/safety_measures/

24. On Safety-Critical Software Development - Increment, accessed March 28, 2026, https://increment.com/reliability/safety-critical-software-development/

25. Safe Software for Automated Driving - Functional Safety for High Performance ECUs - Vector, accessed March 28, 2026, https://cdn.vector.com/cms/content/know-how/_technical-articles/Safety_Automated_Driving_ATZelektronik_202007_PressArticle_EN.pdf

26. A Taxonomy to Unify Fault Tolerance Regimes for Automotive Systems: Defining Fail-Operational, Fail-Degraded, and Fail-Safe, accessed March 28, 2026, https://d-nb.info/1268108790/34

27. What Does It Mean to Fail Safe?. "Failure is inevitable, but surprise is… | by Tobe Onyema | Medium, accessed March 28, 2026, https://medium.com/@tobe.onyema/what-does-it-mean-to-fail-safe-ba6e33db2479

28. Technical Report Fail-Operational and Fail-Safe Vehicle Platooning in the Presence of Transient Communication Errors - Diva-portal.org, accessed March 28, 2026, https://www.diva-portal.org/smash/get/diva2:1646357/FULLTEXT01.pdf

29. Software Reliability and Safety in Nuclear Reactor Protection Systems, accessed March 28, 2026, https://www.nrc.gov/reading-rm/doc-collections/nuregs/contract/cr6101/cr6101.pdf

30. Is it safe to „let it crash”? : r/programming - Reddit, accessed March 28, 2026, https://www.reddit.com/r/programming/comments/1mdlbw/is_it_safe_to_let_it_crash/

31. Reinforcement Learning for Fail-Operational Systems with Disentangled Dual-Skill Variables - MDPI, accessed March 28, 2026, https://www.mdpi.com/2227-7080/13/4/156

32. The Safe State: Design Patterns and Degradation Mechanisms for Fail- Operational Systems - Elektrobit, accessed March 28, 2026, https://www.elektrobit.com/wp-content/uploads/2015/11/The_safe_state-Design_patterns_and_degradation_mechanisms_for_fail-operational_systems.pdf

33. Fail-safe - Wikipedia, accessed March 28, 2026, https://en.wikipedia.org/wiki/Fail-safe

34. Understanding Fail-Safe PLC Systems: Hardware and Programming Guide, accessed March 28, 2026, https://industrialmonitordirect.com/blogs/knowledgebase/understanding-fail-safe-plc-systems-hardware-and-programming-guide

35. What to Expect with Version 3 of ISO 26262 - UL Solutions, accessed March 28, 2026, https://www.ul.com/sis/blog/what-to-expect-with-version-3-of-iso-26262

36. Functional Design Specifications (FDS) - IACS Engineering, accessed March 28, 2026, https://www.iacsengineering.com/functional-specifications/

37. Multicore processors and critical embedded systems: WCET, interference research, and other challenges - LDRA, accessed March 28, 2026, https://ldra.com/capabilities/mcp/

38. SUMMARY OF TOPICS - Electrical and Computer Engineering, accessed March 28, 2026, https://users.ece.cmu.edu/~koopman/pubs/191213_UL4600_VotingVersion.pdf

39. The most useful feature of fixed point calculations is its deterministic nature…. - Hacker News, accessed March 28, 2026, https://news.ycombinator.com/item?id=10061759

40. What are the pros and cons of fixed-point arithmetics vs floating-point arithmetics?, accessed March 28, 2026, https://langdev.stackexchange.com/questions/665/what-are-the-pros-and-cons-of-fixed-point-arithmetics-vs-floating-point-arithmet

41. How is the development process for mission critical software? : r/cpp - Reddit, accessed March 28, 2026, https://www.reddit.com/r/cpp/comments/5k2v3v/how_is_the_development_process_for_mission/

42. Does flight software use floating point or fixed points for its calculations? Why? - Reddit, accessed March 28, 2026, https://www.reddit.com/r/aerospace/comments/acsn0n/does_flight_software_use_floating_point_or_fixed/

43. Valori: A Deterministic Memory Substrate for AI Systems - arXiv, accessed March 28, 2026, https://arxiv.org/html/2512.22280v1

44. Automated Fixed-Point Precision Optimization for FPGA Synthesis - IEEE Xplore, accessed March 28, 2026, https://ieeexplore.ieee.org/iel8/8784029/10830546/11039693.pdf

45. Deterministic Atomic Buffering - IEEE/ACM International Symposium on Microarchitecture, accessed March 28, 2026, https://www.microarch.org/micro53/papers/738300a981.pdf

46. Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications

This manuscript has been authored by UT-Battelle, LLC under Contract No. DE-AC05-00OR22725 with the U.S. Department of Energy. The United States Government retains and the publisher, by accepting the article for publication, acknowledges that the United - arXiv, accessed March 28, 2026, https://arxiv.org/html/2408.05148v1

47. Arithmetic with binary-encoded balanced ternary numbers | Request PDF - ResearchGate, accessed March 28, 2026, https://www.researchgate.net/publication/269308357_Arithmetic_with_binary-encoded_balanced_ternary_numbers

48. Diff - 49b07690a93a813585047d8265e326de8efd95c6^! - gofrontend - Git at Google - Google Git, accessed March 28, 2026, https://go.googlesource.com/gofrontend/+/49b07690a93a813585047d8265e326de8efd95c6%5E%21/

49. Towards Smart Contract Fuzzing on GPUs - About Me, accessed March 28, 2026, https://chapering.github.io/pubs/sp24weimin.pdf

50. The Architecture of Reliability: A Comprehensive Treatise on CUDA Error Handling and Debugging Methodologies | Uplatz Blog, accessed March 28, 2026, https://uplatz.com/blog/the-architecture-of-reliability-a-comprehensive-treatise-on-cuda-error-handling-and-debugging-methodologies/

51. Finding And Preventing Bugs In c++ - Volt Software, accessed March 28, 2026, https://volt-software.nl/posts/finding-and-preventing-bugs-in-cpp/

52. Analysis of Error Propagation in Safety Critical Software Systems: An Approach Based on UGF | Request PDF - ResearchGate, accessed March 28, 2026, https://www.researchgate.net/publication/315066106_Analysis_of_Error_Propagation_in_Safety_Critical_Software_Systems_An_Approach_Based_on_UGF

53. Comparative Analysis of Programming Languages in Cryptography - Diva-portal.org, accessed March 28, 2026, http://www.diva-portal.org/smash/get/diva2:1885509/FULLTEXT01.pdf

54. "I wouldn't want my unsafe code to run my pacemaker": An Interview Study on the Use, Comprehension, and Perceived Risks of - USENIX, accessed March 28, 2026, https://www.usenix.org/system/files/usenixsecurity23-holtervennhoff.pdf

55. I've heard that "Rust's borrow checker is necessary to ensure memory safety without a GC" usually also implying it's the only way, but I've done the same without the borrow checker. Am I just clueless/confused? - Reddit, accessed March 28, 2026, https://www.reddit.com/r/rust/comments/16gb7pc/ive_heard_that_rusts_borrow_checker_is_necessary/

56. Borrow checking, RC, GC, and the Eleven (!) Other Memory Safety Approaches, accessed March 28, 2026, https://verdagon.dev/grimoire/grimoire

57. r/gamedev on Reddit: Rant: Entity systems and the Rust borrow checker … or something., accessed March 28, 2026, https://www.reddit.com/r/gamedev/comments/9fq828/rant_entity_systems_and_the_rust_borrow_checker/

58. Jonathan Blow: Entity Systems and the Rust Borrow Checker… or something - Reddit, accessed March 28, 2026, https://www.reddit.com/r/rust/comments/9fqget/jonathan_blow_entity_systems_and_the_rust_borrow/

59. I like Odin | Brian Lovin, accessed March 28, 2026, https://brianlovin.com/hn/32626543

60. Memory safety without borrow checking, reference counting, or garbage collection | Hacker News,

accessed March 28, 2026, https://news.ycombinator.com/item?id=36351415

61. I like Odin | Hacker News, accessed March 28, 2026, https://news.ycombinator.com/item?id=32626543

62. All | Search powered by Algolia, accessed March 28, 2026, https://hn.algolia.com/?query=%E2%80%9CIt%20is%20neve

63. Who Owns the Memory? Part 3: How Big Is your Type? | Luca Lombardo, accessed March 28, 2026, https://lukefleed.xyz/posts/who-owns-the-memory-pt3/

64. RKOS: UNIKERNEL DESIGN FOR SAFETY AND PERFORMANCE - Index of /, accessed March 28, 2026, https://media.taricorp.net/rkos.pdf

65. SPARK Overview - learn.adacore.com, accessed March 28, 2026, https://learn.adacore.com/courses/intro-to-spark/chapters/01_Overview.html

66. Assuring Functional Safety in Open Systems of Systems - kluedo, accessed March 28, 2026, https://kluedo.ub.rptu.de/files/4422/_Assuring+Functional+Safety+in+Open+Systems+of+Systems.pdf

67. Programming languages and dimensions - Department of Computer Science and Technology | - University of Cambridge, accessed March 28, 2026, https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-391.pdf

68. Dimensional analysis and types, accessed March 28, 2026, https://www.johndcook.com/blog/2015/12/01/dimensional-analysis-and-types/

69. Is F# suitable for Physics applications? - Stack Overflow, accessed March 28, 2026, https://stackoverflow.com/questions/167909/is-f-suitable-for-physics-applications

70. Units of measure - F# for fun and profit, accessed March 28, 2026, https://fsharpforfunandprofit.com/posts/units-of-measure/

71. Units of Measure - F# | Microsoft Learn, accessed March 28, 2026, https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/units-of-measure

72. Physical Units Pass the Generic Test - AdaCore, accessed March 28, 2026, https://www.adacore.com/blog/physical-units-pass-the-generic-test

73. Dimensional Analysis in Programming Languages - Reddit, accessed March 28, 2026, https://www.reddit.com/r/programming/comments/8lwfis/dimensional_analysis_in_programming_languages/

74. SPARK: Formal Verification and Proving Program Correctness in Ada | by Jordan Rowles, accessed March 28, 2026, https://jordansrowles.medium.com/spark-formal-verification-and-proving-program-correctness-in-ada-3105cc82694d

75. The SPARK Programming Language - AdaCore, accessed March 28, 2026, https://www.adacore.com/languages/spark

76. 8. Applying SPARK in Practice - Documentation - AdaCore, accessed March 28, 2026, https://docs.adacore.com/spark2014-docs/html/ug/en/usage_scenarios.html

77. Why isn't design by contract more common? : r/ProgrammingLanguages - Reddit, accessed March 28, 2026, https://www.reddit.com/r/ProgrammingLanguages/comments/l238o5/why_isnt_design_by_contract_more_common/

78. Is Rust faster than C? - Hacker News, accessed March 28, 2026, https://news.ycombinator.com/item?id=46569175

79. Rust Performance Optimizations Compared to Other Programming Languages - Medium, accessed March 28, 2026, https://medium.com/@kaly.salas.7/rust-performance-optimizations-compared-to-other-programming-languages-c2e3685163e2

80. A Compiler-Driven Approach for Static Dependency Injection in Embedded Software, accessed March 28, 2026, https://sol.sbc.org.br/index.php/sblp/article/download/36944/36729/

81. Tighten Rust's Belt: Shrinking Embedded Rust Binaries - Stanford Sing, accessed March 28, 2026, https://sing.stanford.edu/site/assets/publications/rust-lctes22.pdf

82. DO-178C Certification - Your Complete Verification Journey - LDRA, accessed March 28, 2026, https://ldra.com/do-178/

83. SEI CERT C Coding Standard: Rules for Developing Safe, Reliable, and Secure Systems (2016 Edition), accessed March 28, 2026, https://abougouffa.github.io/awesome-coding-standards/sei-cert-c-2016.pdf

84. From ASIL D to ASIL E: A Unified Framework for Driver-Out Functional Safety, accessed March 28, 2026, https://www.researchgate.net/publication/395910500_From_ASIL_D_to_ASIL_E_A_Unified_Framework_for_Driver-Out_Functional_Safety

85. Improving Operating System Security, Reliability, and Performance through Intra-Unikernel Isolation, Asynchronous Out-of-kernel - VTechWorks, accessed March 28, 2026, https://vtechworks.lib.vt.edu/bitstreams/ae3b79d1-c54b-467d-be64-42d34dcd6b99/download

86. Key Automotive Functional Safety Challenges: ISO 26262 | Synopsys Blog, accessed March 28, 2026, https://www.synopsys.com/blogs/chip-design/auto-functional-safety-iso-26262-key-challenges.html

87. ISL-ESRD-TR-14-03, "Evaluation of Guidance for Tools Used to Develop Safety-Related Digital Instrumentation and Control Sof - Nuclear Regulatory Commission, accessed March 28, 2026, https://www.nrc.gov/docs/ML1504/ML15043A206.pdf